DCAF: A Comprehensive Architecture-Agnostic Methodology for Behavior-Specific Circuit Isolation

David Kogan
davidkny22@gmail.com

Abstract

Mechanistic interpretability has produced powerful methods for discovering neural circuits, yet existing approaches typically rely on a single attribution technique, lack calibrated confidence scores, and operate at head-level or edge-level granularity—missing the projection-level structure (e.g., W_Q, W_K, W_V, W_O) where behavioral signatures diverge within the same component. We introduce the Differential Circuit Analysis Framework (DCAF), which treats training signals as controlled perturbation experiments: by training a model under 11 paired behavioral objectives (5 target-reinforcing, 5 opposite-reinforcing, 1 neutral baseline) and triangulating evidence across weight deltas, activation patterns, and latent geometry, DCAF discovers candidate circuit components, maps their causal connections, and classifies their functional roles. Key technical contributions include: (i) the 11-paired-objective probe configuration that uses preference optimization, supervised fine-tuning, gradient ascent, and sequential training as orthogonal discovery signals; (ii) three complementary discovery paths (weight-based, activation-based, gradient-based) integrated through a triangulated confidence score; (iii) the Direction Convergence Score (DCS), which measures cross-signal agreement on behavioral directions in the residual stream; and (iv) a 7-phase ablation protocol with interaction discovery and adaptive tiered functional classification. The framework is formulated in an architecture-agnostic manner with a complete transformer instantiation. A reference implementation is publicly available at https://github.com/davidkny22/dcaf-public.

Contents

1 Introduction **5**

2 Related Work **6**
2.1 Circuit Discovery Methods . . . 6
2.2 Representation Engineering and Steering Vectors . . . 7
2.3 Training Dynamics as an Interpretability Lens . . . 7
2.4 Weight-Space and Parameter-Level Analysis . . . 7
2.5 Sparse Autoencoders and Feature Discovery . . . 8
2.6 Safety Circuit Protection . . . 8
2.7 Methodological Foundations . . . 8

3 Foundations **10**

4 Multi-Path Discovery **16**
4.1 Path 1: Weight-Based Discovery ($\mathcal{H}_W$) . . . 17
4.2 Path 2: Activation-Based Discovery ($\mathcal{H}_A$) . . . 18
4.3 Path 3: Gradient-Based Discovery ($\mathcal{H}_G$) . . . 19
4.4 Discovery Integration . . . 19

5 Weight Analysis **20**
5.1 Signal Effectiveness . . . 20
5.2 RMS Normalization of Projection Deltas . . . 21
5.3 Opposition Degree . . . 22
5.4 SVD Diagnostics on Projection Deltas . . . 23
5.5 Weight Confidence . . . 24

6 Activation Analysis **24**
6.1 Probe Types . . . 24
6.2 Activation Capture . . . 25
6.3 Activation Confidence . . . 25

7 Geometry Analysis **26**
7.1 Contrastive Direction Extraction . . . 26
7.2 Direction Dynamics . . . 26
7.3 Confound Independence . . . 27
7.4 Cross-Signal Alignment . . . 27
7.5 Predictivity . . . 28
7.6 Linear Representation Score . . . 28
7.7 Geometric Confidence . . . 29
7.8 Representation Direction Synthesis . . . 29
7.8.1 Global Direction Extraction . . . 30
7.8.2 Direction Convergence Score (DCS) . . . 30
7.8.3 Layer Trajectory Analysis . . . 32
7.8.4 Signal Diagnostics . . . 32
7.8.5 Hierarchical Composition Analysis . . . 33
7.8.6 Intervention Specification . . . 34
7.8.7 Edge Cases and Failure Modes . . . 34

8 Training Diagnostics **35**
8.1 Activation Delta Alignment . . . 35
8.2 Online Curvature Tracking . . . 35

9 Unified Confidence and Candidate Selection **36**
9.1 Base Triangulated Confidence . . . 37
9.2 Multi-Path Discovery Bonus . . . 37
9.3 Candidate Set Construction . . . 37
9.4 Domain Disagreement Analysis . . . 38

10 Circuit Graph Reconstruction **38**
10.1 Edge Discovery Methods . . . 38

11 Steering Vector Optimization **39**
11.1 Bidirectional Steering Vectors . . . 40
11.2 Steering Alignment Metrics . . . 40
11.3 Steering Effectiveness . . . 41
11.4 Steering Validation . . . 41

12 Ablation Methodology and Functional Classification **41**

12.1 Ablation Techniques . . . 41
12.2 Multi-Strategy Interaction Discovery . . . 42
12.2.1 Phase 1: Individual Component Ablation . . . 42
12.2.2 Phase 2: Seven Parallel Pair/Group Strategies . . . 42
12.2.3 Phase 3: Refinement . . . 43
12.2.4 Phase 4: Cross-Validation Counting . . . 44
12.2.5 Phase 5: Triple Detection . . . 44
12.2.6 Phase 6: Orphan Analysis . . . 44
12.2.7 Phase 7: Final Classification . . . 45
12.3 Interaction-Enriched Output . . . 45
12.4 Computational Complexity . . . 45
12.5 Adaptive Tiered Functional Classification . . . 46

13 Experiments **48**

14 Discussion and Limitations **48**

15 Conclusion **49**

A Architecture-Specific Implementations **55**
A.1 Large Language Models (Transformers) . . . 55
A.1.1 Probe Type Instantiation . . . 55
A.1.2 Component Decomposition . . . 55
A.1.3 Activation Capture Points . . . 57
A.1.4 Differential Analysis Methods . . . 57
A.1.5 Example: Safety Behavior Circuit Discovery . . . 58
A.2 Domain-Specific Considerations . . . 58
A.2.1 Universal Applicability . . . 59

B Complete Notation Reference **59**
B.1 Core Objects . . . 59
B.2 Transformer Architecture (LLM-Specific) . . . 60
B.3 Training Signals and Checkpoints . . . 60
B.4 Multi-Path Discovery . . . 61
B.5 Weight Domain . . . 62
B.6 Activation Domain . . . 63
B.7 Geometry and Linear Representation . . . 63
B.8 Direction Synthesis . . . 64
B.9 Diagnostics . . . 65
B.10 Confidence and Candidate Selection . . . 65
B.11 Circuit Graph . . . 66
B.12 Steering and Classification . . . 66
B.13 Thresholds and Default Parameters . . . 67
B.14 General Notation . . . 68

C Significance Predicates **68**

D Complete Pipeline **70**

E Extensibility **71**

F Framework Output **71**
F.1 Domain Analysis Scores 72

G Nonlinear Diagnostics **74**

H Parametric Embedding **75**

I Implementation Considerations **76**

J Domain-Specific Sketches (Preliminary) **77**
J.1 Vision Models 77
J.1.1 Probe Type Instantiation 77
J.1.2 Component Decomposition 77
J.1.3 Example: Biased Image Generation 77
J.2 Diffusion Models 78
J.2.1 Probe Type Instantiation 78
J.2.2 Activation Capture Points 78

K Circuit Graph Output Format **78**

1 Introduction

Neural networks implement behaviors through identifiable computational circuits—sparse subgraphs of model components that mediate specific input-output relationships. The field of mechanistic interpretability has made remarkable progress in discovering these circuits, from the manual reverse-engineering of the Indirect Object Identification (IOI) circuit in GPT-2 Small [Wang et al., 2023]—26 attention heads organized into 7 functional classes—to automated methods such as ACDC [Conmy et al., 2023], Attribution Patching [Syed et al., 2024], and Anthropic's Circuit Tracing [Ameisen et al., 2025]. The foundational vocabulary of residual streams, virtual weights, and QK/OV projection-level decomposition was established by Elhage et al. [2021], and the broader circuits hypothesis articulated by Olah et al. [2020].

Despite this progress, several limitations constrain current circuit discovery methodology. First, single-method circuit discovery exhibits high estimator variance: Méloux et al. [2025] demonstrate that EAP-IG circuit attributions have substantial variance under bootstrap resampling, undermining replicability. Second, the field has identified "interpretability illusions" arising from single-method conclusions and has explicitly called for cross-method validation [Sharkey et al., 2025]. Third, existing automated methods operate above the projection level—ACDC at head/edge granularity, CD-T at head-plus-position granularity [Hsu et al., 2025]—despite Elhage et al.'s (2021) foundational framework already defining projection-level structure. Fourth, activation patching results vary dramatically with hyperparameter choices [Zhang and Nanda, 2024], revealing methodological fragility. Fifth, no published framework uses paired training perturbations as primary discovery probes, despite growing evidence that training dynamics reveal circuit structure: Prakash et al. [2024] show that fine-tuning enhances existing mechanisms, and Tigges et al. [2024] demonstrate that circuit analyses are consistent across training and scale.

We introduce the Differential Circuit Analysis Framework (DCAF), which addresses these limitations through a fundamentally different approach to circuit discovery. Where existing automated methods are *post-hoc*—they analyze a fixed, already-trained model to identify circuits after the fact—DCAF is *prospective*: it uses training itself as the primary discovery intervention, producing confidence-scored candidate components that ablation then validates. By running multiple training signals as controlled perturbation experiments and observing what changes across weights, activations, and latent geometry, DCAF identifies circuit candidates *before* any ablation is performed, rather than relying on ablation as the discovery mechanism.

DCAF synthesizes projection-level analysis [Elhage et al., 2021], training-as-probe methodology [Prakash et al., 2024, Tigges et al., 2024], the triangulation principle advocated by Long [2025] and Sharkey et al. [2025], and multi-method validation strategies emerging from the BlackboxNLP 2025 shared task [Mondorf et al., 2025] into a unified, formally specified protocol. Rather than introducing a single new technique, DCAF provides a complete pipeline that integrates independently established threads into a coherent framework with novel operational instruments: the 11-paired-objective probe configuration, the Direction Convergence Score (DCS), the 7-phase ablation protocol with interaction discovery, and adaptive tiered functional classification. These specific instruments have no direct named precedents in the literature.

The key contributions of this work are:

1. **Multi-signal training perturbation protocol.** We systematize the insights of Prakash et al. [2024] and Tigges et al. [2024] into a complete 11-signal configuration that uses preference optimization, supervised fine-tuning, gradient ascent, and sequential training as orthogonal discovery probes, extending contrastive methods [Panickssery et al., 2024, Arditi et al., 2024] from inference-time prompt contrasts to training-time signal contrasts.
2. **Triangulated confidence scoring.** We implement the triangulation principle called for by

Long [2025] and Sharkey et al. [2025] through a specific operationalization across weight, activation, and geometric domains, producing calibrated confidence scores that reflect convergent evidence.

3. **Projection-level discovery at scale.** We operationalize at scale the QK/OV decomposition introduced by Elhage et al. [2021], extending it to gated MLP architectures and integrating it into an automated discovery pipeline alongside parameter-space methods such as APD [Braun et al., 2025].
4. **Ablation protocol with interaction discovery.** We specify a 7-phase ablation methodology that efficiently discovers individual, pairwise, and triple component interactions through seven parallel search strategies, producing functional classifications and interaction-enriched circuit graphs.
5. **Architecture-agnostic formulation.** The core methodology—perturbation experiments, multi-domain evidence, triangulated confidence—is defined independently of model architecture, with a complete transformer instantiation and preliminary sketches for vision and reinforcement learning systems.

A reference implementation of DCAF is publicly available at `https://github.com/davidkny22/dcaf-public`. The implementation covers the core mathematical framework, multi-path discovery, domain analysis modules (weight, activation, geometry), training diagnostics, confidence scoring, the 7-phase ablation protocol, circuit graph reconstruction, and CLI tooling for transformer architectures. Features currently under development include direction synthesis (global residual stream directions and DCS computation), MIB benchmark compatibility, and vision/diffusion model instantiation.

The remainder of this paper is organized as follows. Section 2 surveys related work. Sections 3–12 present the complete technical framework. Section 13 is reserved for empirical validation. Section 14 discusses limitations, and Section 15 concludes. Architecture-specific implementations and a complete notation reference appear in the appendices.

2 Related Work

DCAF builds on and extends a rich body of work in mechanistic interpretability, representation engineering, and AI safety. We organize prior art into seven areas and position DCAF relative to each.

2.1 Circuit Discovery Methods

Manual circuit analysis. Wang et al. [2023] reverse-engineered the IOI circuit in GPT-2 Small, identifying 26 attention heads in 7 functional classes through causal intervention. Hanna et al. [2023] similarly dissected the greater-than circuit, and Heimersheim and Janiak [2023] analyzed a Python docstring circuit in a 4-layer model. These studies demonstrated that transformer behaviors are implemented by identifiable circuits but required extensive manual effort.

Automated discovery. Conmy et al. [2023] introduced Automated Circuit DisCovery (ACDC), which scores edge importance via activation patching and prunes below a threshold, operating at head/MLP granularity. Syed et al. [2024] showed that Attribution Patching (EAP)—a linear approximation requiring only two forward passes and one backward pass—outperforms ACDC in edge recovery. Goldowsky-Dill et al. [2023] formalized path patching, the method ACDC automates. Hsu et al. [2025] achieved 97% ROC AUC with Contextual Decomposition for Transformers (CD-T), reducing circuit discovery runtime from hours to seconds. Li and Janson [2024] provided theoretical bounds for optimal ablation, producing smaller and more faithful circuits. Edin et al.

[2025] introduced Gradient Interaction Modifications (GIM), a gradient-based method achieving high accuracy on the Mechanistic Interpretability Benchmark [Mueller et al., 2025]. Hadad et al. [2026] adapted neural network verification to circuit discovery with provable guarantees.

Production-scale methods. Anthropic's Circuit Tracing [Ameisen et al., 2025, Lindsey et al., 2025] introduced cross-layer transcoders and attribution graphs applied to Claude 3.5 Haiku. Lieberum et al. [2023] demonstrated circuit analysis at 70B scale.

DCAF complements these approaches by operating at the projection level within components (e.g., W_Q, W_K, W_V, W_O) rather than at the head/edge level, and by using multiple independent training perturbations as discovery probes rather than a single attribution method. DCAF's ablation protocol could integrate the theoretically grounded ablation techniques of Li and Janson [2024].

2.2 Representation Engineering and Steering Vectors

Zou et al. [2023] introduced Representation Engineering (RepE), examining population-level representations to monitor cognitive phenomena. Arditi et al. [2024] identified a single 1D subspace mediating refusal across 13 models up to 72B parameters. Li et al. [2023] used linear probes for Inference-Time Intervention on truthfulness. Marks and Tegmark [2024] established causal links between truth representations and outputs through the Geometry of Truth. Tigges et al. [2023] identified sentiment as a single linear dimension concentrating at syntactically neutral positions. Turner et al. [2023] proposed activation addition (ActAdd) for inference-time steering. Panickssery et al. [2024] systematized contrastive activation addition with paired positive/negative behavioral examples for Llama 2. Park et al. [2024] formalized the Linear Representation Hypothesis. Zhang and Nanda [2024] demonstrated that activation patching results vary substantially with hyperparameter choices.

DCAF extends contrastive methods from inference-time prompt contrasts to training-time signal contrasts and requires convergence across multiple independent extraction methods. Where existing approaches extract directions from a single contrast pair, DCAF validates directions through cross-signal convergence: if multiple training signals independently produce directions that agree (as measured by DCS), the direction is robust to methodology choice.

2.3 Training Dynamics as an Interpretability Lens

This cluster represents the closest prior art to DCAF's training-perturbation methodology. Nanda et al. [2023] provided the earliest example of using training trajectories to expose circuit formation via progress measures for grokking. Prakash et al. [2024] introduced Cross-Model Activation Patching (CMAP) and showed that fine-tuning enhances existing mechanisms—the most direct precedent for training-as-discovery. Tigges et al. [2024] demonstrated that LLM circuit analyses are consistent across training and scale, providing evidence that training dynamics are a reliable lens. Wang et al. [2025] tracked edge changes across fine-tuning checkpoints. Singh et al. [2024] used optogenetics-inspired causal interventions throughout training to study induction head formation.

DCAF systematizes these insights into a complete multi-signal protocol, using 11 paired training objectives as orthogonal probes. Where prior work uses training dynamics as a post-hoc analytical lens, DCAF treats each training signal as a controlled perturbation experiment whose weight deltas, activation changes, and geometric effects are triangulated into confidence scores.

2.4 Weight-Space and Parameter-Level Analysis

Elhage et al. [2021] defined the QK and OV decomposition of attention heads—already operating

at projection level—as the foundational analytical framework. Braun et al. [2025] introduced Attribution-based Parameter Decomposition (APD), the most explicit "weights-not-activations" circuit method, operating directly across all projection matrices. Gao et al. [2025] trained models with L_0-sparse weights to produce intrinsic projection-level circuits. Dooms et al. [2024] enabled weight-based mechanistic interpretability through bilinear MLPs. Golimblevskaia et al. [2025] introduced WeightLens and CircuitLens for interpreting features directly from learned weights.

DCAF does not claim to have invented projection-level analysis. Rather, it operationalizes at scale the QK/OV decomposition introduced by Elhage et al. [2021], extending it to gated MLP architectures (W_{gate}, W_{up}, W_{down}) and integrating it into an automated discovery pipeline. The framework's two-level hierarchy—projections as analysis units, components as graph nodes—allows sub-component weight analysis while maintaining compatibility with component-level activation and geometric methods.

2.5 Sparse Autoencoders and Feature Discovery

Cunningham et al. [2024] and Bricken et al. [2023] concurrently established sparse autoencoders (SAEs) as decomposition tools for language model activations. Gao et al. [2024] scaled SAEs to GPT-4 with 16M latents. Lieberum et al. [2024] released Gemma Scope, providing open-source SAEs across all layers of Gemma 2. Marks et al. [2025] extended SAEs to Sparse Feature Circuits, discovering causal subgraphs of interpretable features. Templeton et al. [2024] scaled monosemanticity to Claude 3 Sonnet. He et al. [2024] used dictionary learning as an alternative to activation patching for circuit discovery.

DCAF is complementary to SAE-based methods. SAEs decompose activations into interpretable features; DCAF identifies which components are behaviorally relevant through training perturbations. SAE features could serve as activation probes within DCAF's framework, and DCAF's weight-delta analysis operates orthogonally to SAE-based feature flow analysis.

2.6 Safety Circuit Protection

Zou et al. [2024] proposed Circuit Breakers for identifying and breaking representational pathways for undesirable behaviors. Han et al. [2025] identified safety neurons and proposed Safety Neuron Tuning, finding that edits to a sparse subset of safety neurons (often $<5\%$, dropping to $<1\%$ for incremental safety dimensions) can reduce fine-tuning risks. Huang et al. [2025] introduced BOOSTER for defense against adversarial fine-tuning using safety-specific neuron detection. Zhu et al. [2025] developed SafetyLock, extracting safety-bias directions and re-injecting them post-fine-tuning. Wei et al. [2024] localized safety to $\sim 3\%$ of parameters via pruning and low-rank modifications. Qi et al. [2024] established the foundational threat model showing that fine-tuning compromises safety. Lermen et al. [2023] demonstrated that LoRA fine-tuning efficiently undoes safety training.

DCAF provides the upstream discovery methodology that these protection methods require: identifying which projection matrices implement safety behaviors, at what confidence, and through which functional roles. The framework's opposition testing ($\mathcal{T}^+/\mathcal{T}^-$) directly reveals bidirectional control points—components where the same weights mediate both safe and unsafe behavior.

2.7 Methodological Foundations

Long [2025] formalized triangulation as an acceptance rule for mechanistic interpretability, explicitly advocating for multi-method validation. Méloux et al. [2025] reframed circuit discovery as a

high-variance statistical estimation problem, recommending bootstrap-based uncertainty quantification. Mondorf et al. [2025] explored ensemble strategies for circuit localization at BlackboxNLP 2025, finding that combining methods improves performance. Geiger et al. [2021] developed causal abstractions of neural networks, and Geiger et al. [2025] unified activation patching, path patching, causal scrubbing, and distributed alignment search under a single theoretical framework. Belinkov [2022] provided a comprehensive analysis of probing classifiers, and Hewitt and Manning [2019] introduced the structural probe for syntax. Meng et al. [2022] localized factual recall to mid-layer MLPs. Rafailov et al. [2023] introduced Direct Preference Optimization, one of the training objectives DCAF uses as a discovery signal.

DCAF implements the triangulation principle through a specific operationalization: weight deltas capture what optimization modified, activation changes capture representational reorganization, and latent geometry captures behavioral direction structure. The DCS metric provides a concrete measure of the cross-signal convergence that Sharkey et al. [2025] call for and Méloux et al. [2025] demonstrate is needed.

3 Foundations

Definition 3.1 (Configurable Baseline Model)**.** *Let $\mathcal{M}_0$ be the baseline model with parameters $W_0 \in \mathbb{R}^d$ and components $\mathcal{K} = \{k_1, \ldots, k_m\}$. The baseline is chosen from two options based on the behavioral domain under study:*

Option A: *$\mathcal{M}_0 = \mathcal{M}_{pre}$ (the pretrained model directly). Appropriate when the target behavior's training data does not carry instruction-following signal.*

Option B (recommended for safety research): *$\mathcal{M}_0 = IT(\mathcal{M}_{pre}, \mathcal{D}_{IT})$—the pretrained model after instruction tuning on a behaviorally neutral dataset $\mathcal{D}_{IT}$. Recommended for any behavioral domain whose datasets inherently depend on instruction-following format (safety, helpfulness, honesty, etc.).*

All weight deltas ΔW, activation baselines A_0, and ablation baseline restorations are relative to whichever $\mathcal{M}_0$ is selected. The choice of $\mathcal{M}_0$ is a design decision made once per study and held fixed throughout.

Remark 3.1 (Rationale for Option B)**.** *Safety datasets teach two things simultaneously: (1) how to follow instructions and (2) the target safety behavior. If $\mathcal{M}_0$ is the pretrained model, every ΔW conflates "learned to follow instructions" with "learned the target behavior," causing DCAF to misidentify instruction-following circuits as safety circuits. Instruction-tuning first on a behaviorally neutral dataset means $\mathcal{M}_0$ already knows how to follow instructions, so behavioral training produces deltas isolating only the behavioral component. This is the IT → IT+Behavior design, strictly cleaner than Base → Base+Behavior for instruction-dependent behaviors.*

Definition 3.2 (Requirements for $\mathcal{D}_{\text{IT}}$ (Option B))**.** *When using Option B, the instruction-tuning dataset $\mathcal{D}_{IT}$ must satisfy:*

1. ***Teaches general instruction following:*** *chat format, question answering, task completion.*
2. ***Behaviorally filtered:*** *explicitly excludes content in the target behavioral domain (e.g., no safety content for safety circuit analysis).*
3. ***Sufficient scale:*** *large enough that $\mathcal{M}_0$ is a competent instruction follower.*

Capybara is the current default for safety research, chosen for its explicit safety content filtering.

Definition 3.3 (Granularity Architecture)**.** *DCAF employs a two-level hierarchy aligned with the mechanistic interpretability consensus:*

Analysis unit — Projection: *A single weight matrix within a model component. For attention components, the projections are W_Q, W_K, W_V, W_O (per head). For MLP components, the projections are $W_{gate}, W_{up}, W_{down}$ (per layer). Let $\mathcal{P}$ denote the set of all projections.*

Graph node — Component: *The functional unit for circuit graphs, ablation, and cross-domain confidence. Attention components are individual heads; MLP components are full MLP layers. These coincide with the components $\mathcal{K} = \{k_1, \ldots, k_m\}$.*

Projection-to-Component mapping: *$\mu_{proj} : \mathcal{P} \to \mathcal{K}$ maps each projection to its containing component. The inverse $\mu_{proj}^{-1}(k)$ returns all projections belonging to component k.*

Remark 3.2 (Why Two Levels)**.** *Sub-component granularity is necessary for weight analysis because projections within the same component may show completely different behavioral signatures — W_Q and W_V within the same head may respond to entirely different training signals. Activation and geometry domains operate at component level because activations flow through the residual stream at head/MLP output granularity. This matches the standard granularity of major circuit discovery methods.*

Remark 3.3 (Excluded from Analysis). ***LayerNorm*** *parameters (scaling and bias vectors) are not projection matrices; their effects are captured through downstream projection activations.* ***Embedding and unembedding*** *layers are shared input/output transformations, not behavioral circuit components. Both are excluded from the projection set* $\mathcal{P}$.

Definition 3.4 (Operating Principle). *DCAF treats each training signal as a controlled perturbation experiment. By analyzing what changes—in weights, activations, and latent geometry—it discovers candidate components, maps their connections through activation flow and ablation, and characterizes their functional roles. Multiple independent signals and measurement domains provide both comprehensive characterization and confidence through convergence.*

Definition 3.5 (Multi-Path Discovery Philosophy). *DCAF employs three complementary discovery paths, each capturing different aspects of behavioral relevance:*

Path 1 (Weight-Based, $\mathcal{H}_W$***):*** *Projections with large weight changes during behavioral training. Captures what the optimization process modified.*

Path 2 (Activation-Based, $\mathcal{H}_A$***):*** *Projections in components showing large activation changes. Captures leverage points where small weight changes produce large representational effects.*

Path 3 (Gradient-Based, $\mathcal{H}_G$***):*** *Projections with high behavioral gradients. Captures mathematical predictions of what would affect behavior if modified, independent of what training actually changed.*

The paths are complementary because each can discover circuits the others miss:

- *Weight-based discovery misses leverage points: projections with small weight changes but large behavioral impact*
- *Activation-based discovery misses projections whose effects manifest only in specific contexts not covered by probes*
- *Gradient-based discovery misses projections where the gradient landscape differs from actual training dynamics*

Key principle: *Discovery paths return sets of projection IDs, which are mapped to components via* μ_{proj}*. All three measurement domains (weight, activation, geometry) provide confidence for* any *discovered component. A component discovered by Path 2 still receives* C_W*,* C_A*, and* C_G *scores. Discovery determines* what *to analyze; domain confidence determines* how confident *we are.*

Definition 3.6 (Training Signals). *Let* $\mathcal{T} = \{t_1, \ldots, t_N\}$ *be training transformations partitioned into:*

$$\mathcal{T}^+ = \{t : t \text{ reinforces target behavior}\} \tag{1}$$

$$\mathcal{T}^- = \{t : t \text{ reinforces opposite behavior}\} \tag{2}$$

$$\mathcal{T}^0 = \{t : t \text{ is behaviorally neutral baseline}\} \tag{3}$$

The $\mathcal{T}^+/\mathcal{T}^-$ *structure tests whether components show opposing responses to opposing training pressures.* $\mathcal{T}^0$ *filters components responding to any training (general processing) rather than behavioral training specifically.*

Definition 3.7 (Canonical Signal Instantiation). *The 11 canonical signals, where* $a \succ b$ *denotes preference (chosen =* a*, rejected =* b*):*

Target Behavior Cluster $\mathcal{T}^+$:

$t_1 = PrefOpt(target \succ opposite)$ *(preference optimization, e.g., SimPO, DPO)*
$t_2 = SFT(target)$ *(supervised fine-tuning)*
$t_3 = Cumulative(target)$ *(SFT then PrefOpt sequentially)*
$t_4 = Anti(opposite \succ target)$ *(gradient ascent on preference margin from base)*
$t_5 = Negated^{(-)}(opposite \succ target)$ *(gradient ascent on preference margin from opposite-checkpoint)*

Opposite Behavior Cluster $\mathcal{T}^-$:

$$t_6 = PrefOpt(opposite \succ target)$$
$$t_7 = SFT(opposite)$$
$$t_8 = Cumulative(opposite)$$
$$t_9 = Anti(target \succ opposite)$$
$$t_{10} = Negated^{(+)}(target \succ opposite)$$

Neutral Baseline $\mathcal{T}^0$:

$t_{11} = DomainNative(neutral)$ *(domain-appropriate task orthogonal to behavior)*

Domain-specific baseline examples:

- ***Language models:*** *General language modeling on behaviorally-neutral text*
- ***Vision models:*** *Natural image classification or generation (e.g., ImageNet)*
- ***Reinforcement learning:*** *Task-agnostic exploration or random policy*
- ***Diffusion models:*** *Unconditional generation on neutral prompts*
- ***Multimodal models:*** *General vision-language alignment without behavioral bias*

Optional: Confound-Specific Baseline Filtering

The baseline filter can be enhanced when datasets exist that specifically target known confounds.

Definition 3.8 (Enhanced Baseline Structure)**.** *The baseline set can be decomposed into general and confound-specific components:*

$$\mathcal{T}^0_{general} = \{\textit{domain-native training (e.g., general language modeling)}\} \tag{4}$$
$$\mathcal{T}^0_{confounds} = \{\textit{confound-specific training datasets}\} \tag{5}$$

The combined baseline filter becomes:

$$baseline_filter(proj) = \prod_{t \in \mathcal{T}^0_{general} \cup \mathcal{T}^0_{confounds}} \mathbf{1}\left[\|\Delta W_t^{(proj)}\|_{RMS} < \Phi_{\tau_{base}}(\{\|\Delta W_t^{(proj')}\|_{RMS}\}_{proj' \in \mathcal{P}})\right] \tag{6}$$

A projection passes the baseline filter only if it does not respond significantly to general domain training or any confound-specific training.

Remark 3.4 (Confound Examples for Safety Circuits)**.** *Example confound datasets for safety behavior analysis:*

- *Generic refusal templates (non-safety-specific politeness refusals)*
- *Incapability refusals ("I don't know enough about X")*

- *Formatting/politeness patterns*
- *Domain-specific jargon unrelated to safety*

If a projection changes during incapability-refusal training, it implements a generic refusal mechanism (filter out). If it doesn't change, it is potentially safety-specific (retain).

Remark 3.5 (Confound Filtering vs. Geometric Correlation)**.** *Confound-specific filtering is more rigorous than post-hoc geometric correlation (ρ_c in the geometry domain). It directly tests whether projections respond to confound* training *rather than just correlating with confound directions. This catches projections that implement confound patterns even when geometric correlation is low.*

Use confound-specific filtering when confound datasets exist, confounds are well-defined and orthogonal to target behavior, and additional rigor is desired. Omit when no confound datasets are available or computational budget is limited. The framework remains fully functional with only $\mathcal{T}^0_{general}$.

When using Option B for $\mathcal{M}_0$ (Definition 3.1), confound-specific baselines become more targeted: the dominant instruction-following confound has already been absorbed into $\mathcal{M}_0$, so confound datasets can focus on subtler residual confounds (e.g., politeness patterns, formatting preferences) rather than the broad instruction-following signal that would otherwise dominate under Option A.

Remark 3.6 (Signal Distinctions)**.** *Anti and Negated both use gradient ascent on preference margins (not raw likelihood), applicable to any preference optimization method (SimPO, DPO, etc.). They differ in starting point: Anti begins from the baseline model $\mathcal{M}_0$ (learning to reject), Negated begins from a behavior-trained checkpoint (unlearning). The Negated superscript denotes source checkpoint: $(+)$ = target-trained, $(-)$ = opposite-trained.*

Remark 3.7 (Why Gradient Ascent on Preference Margins)**.** *Gradient ascent on SFT loss ("forget how to produce this response") risks catastrophic forgetting of general capabilities. Gradient ascent on preference margins ("don't prefer this over that") is more surgical—it operates on the relative preference between responses rather than absolute likelihood, preserving the model's ability to generate coherent text while adjusting behavioral tendencies.*

Remark 3.8 (Minimal Signal Set)**.** *The full 11-signal protocol maximizes discovery coverage and opposition measurement rigor. For computational efficiency or initial exploration, a minimal 3-signal subset is sufficient:*

- $t_1 = PrefOpt(target \succ opposite)$ — *primary target signal*
- $t_6 = PrefOpt(opposite \succ target)$ — *primary opposite signal*
- $t_{11} = DomainNative(neutral)$ — *baseline filter*

This captures the core $\mathcal{T}^+/\mathcal{T}^-/\mathcal{T}^0$ structure with one signal per cluster. The remaining signals (SFT, Cumulative, Anti, Negated variants) increase measurement robustness and enable effectiveness-weighted aggregation but are not required for initial circuit identification. Expand to the full set when computational budget allows.

Definition 3.9 (Signal-Specific Objectives)**.** *Each signal type defines a specific training objective $\mathcal{O}_i$ whose optimization produces the weight delta:*

$$\mathcal{O}_i = \begin{cases} \mathbb{E}_{(x,y)\sim\mathcal{D}_i}\left[-\log p(y \mid x)\right] & \text{if } t_i \in \{\text{SFT signals}\} \\ -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_i}\left[M(y_w, y_l \mid x)\right] & \text{if } t_i \in \{\text{preference signals}\} \\ -\mathbb{E}_{x\sim\mathcal{D}_i}\left[R(x)\right] & \text{if } t_i \in \{\text{RL signals}\} \\ -\mathbb{E}_{x\sim\mathcal{D}_i}\,\mathbb{E}_{y\sim\pi(\cdot|x)}\left[R(x,y)\right] & \text{if } t_i \in \{\text{GRPO signals}\} \end{cases} \tag{7}$$

where $M(y_w, y_l \mid x)$ is the preference margin function (DPO, SimPO, etc.), $R(x)$ is the episode reward, and $\pi(\cdot|x)$ is the policy generating responses for group evaluation. Anti and Negated signals negate the relevant objective (gradient ascent rather than descent).

Definition 3.10 (Weight Delta)**.** *The weight delta measures how much each projection changed due to a training signal—the fundamental unit of analysis for identifying behaviorally-responsive projections. To isolate the maximal behavioral signal, deltas are computed from the* peak checkpoint*—the point during training where behavioral performance was strongest—rather than the final checkpoint, which may reflect overfitting or degradation:*

$$\Delta W_i^{(proj)} = W_{i,peak}^{(proj)} - W_0^{(proj)} \tag{8}$$

where W_0 denotes the weights of the configured baseline model $\mathcal{M}_0$ (Definition 3.1), and $W_{i,peak}^{(proj)}$ is the peak checkpoint weight matrix for projection proj under signal t_i (see Definition 3.11). Each delta is a full matrix (e.g., $\mathbb{R}^{d_{model} \times d_{head}}$ for an attention projection).

Peak Checkpoint Selection

Training runs do not always improve monotonically. A signal may achieve its strongest behavioral effect mid-training before degrading due to overfitting, catastrophic forgetting, or optimization instability. Using the final checkpoint in such cases dilutes the behavioral signal with noise from the degradation phase.

Definition 3.11 (Peak Checkpoint)**.** *For training signal t_i, let $\{W_{i,\tau}\}_{\tau=1}^{T}$ be the sequence of checkpoints saved at evaluation intervals during training. The peak checkpoint $W_{i,peak}$ is the checkpoint at which the signal-specific behavioral metric $m_{i,\tau}$ achieves its confirmed maximum:*

$$W_{i,peak} = W_{i,\tau^*}, \qquad \tau^* = \arg\max_{\tau} \; m_{i,\tau} \quad \textit{subject to } \texttt{Confirmed}(\tau) \tag{9}$$

where the behavioral metric $m_{i,\tau}$ is signal-type-specific:

$$m_{i,\tau} = \begin{cases} -\mathcal{L}(W_{i,\tau}) & \textit{if } t_i \in \{\textit{SFT signals}\} \\ M(W_{i,\tau}) & \textit{if } t_i \in \{\textit{preference signals}\} \\ \mathbb{E}[R(W_{i,\tau})] & \textit{if } t_i \in \{\textit{GRPO signals}\} \\ R(W_{i,\tau}) & \textit{if } t_i \in \{\textit{RL signals}\} \end{cases} \tag{10}$$

The confirmation criterion prevents selecting transient spikes as the peak:

$$\texttt{Confirmed}(\tau) = \left(m_{i,\tau} = \max_{\tau' \le \tau} m_{i,\tau'}\right) \;\wedge\; \left(\frac{1}{K}\sum_{j=1}^{K} m_{i,\tau+j} \ge m_{i,\tau} - \delta \cdot m_{i,\tau}\right) \tag{11}$$

where K is the confirmation window (default: $K = 3$) and δ is the stability tolerance (default: $\delta = 0.05$). A candidate peak is confirmed only if it is the running maximum and *the subsequent K evaluations remain within δ of the peak value.*

Remark 3.9 (Peak Selection Scenarios)**.** *Four canonical scenarios:*

- ***Monotonic improvement:*** *m_τ increases throughout training. The final checkpoint is the peak. Peak selection reduces to using the last checkpoint.*

- ***Peak-then-decline:*** *m_τ rises, stabilizes, then drops. The confirmed peak captures the strongest behavioral state before degradation, which is exactly what DCAF needs.*
- ***Fluke spike:*** *m_τ briefly spikes but immediately drops. The confirmation window rejects the spike because subsequent evaluations fall below $m_\tau - \delta \cdot m_\tau$. The algorithm selects the next-best confirmed peak.*
- ***Multiple peaks:*** *m_τ oscillates with several local maxima. The algorithm selects the highest confirmed peak, ensuring a stable behavioral state.*

Remark 3.10 (Memory Efficiency)**.** *Peak tracking requires $O(2)$ additional storage per signal: the current confirmed-best checkpoint and the current candidate checkpoint under evaluation. Prior checkpoints can be discarded once a new confirmed peak supersedes them.*

Remark 3.11 (Peak Evaluation Parameters)**.** *Peak checkpoint selection introduces three parameters:*

- ***Peak evaluation interval*** *(default: 50 steps): How frequently to evaluate behavioral metrics during training. Lower values provide finer-grained peak detection at higher computational cost.*
- ***Peak confirmation window*** *K (default: 3): Number of subsequent evaluations required to confirm a peak. Higher values provide greater confidence but may miss peaks near the end of training.*
- ***Peak stability tolerance*** *δ (default: 0.05): Maximum fractional decline permitted in the confirmation window. Lower values require more stringent stability.*

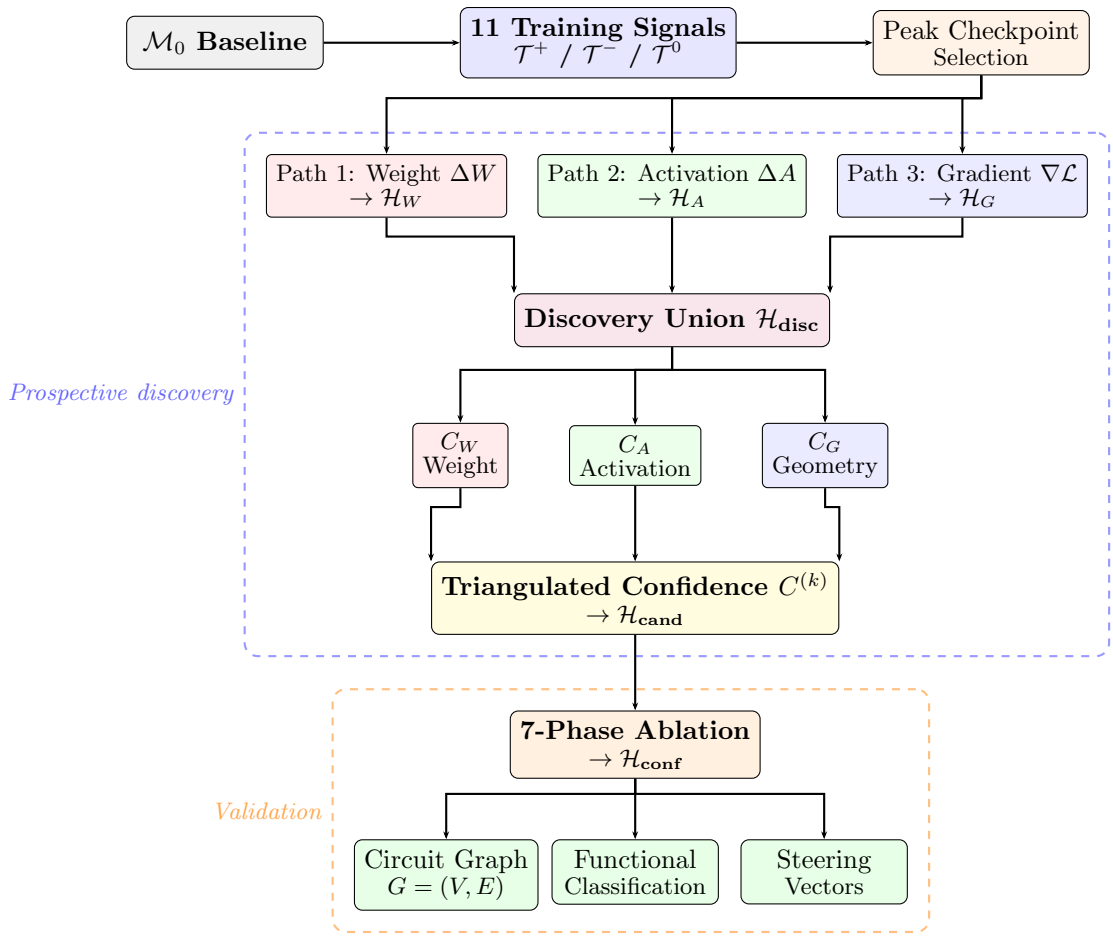

Figure 1: The DCAF pipeline. Training signals produce weight deltas, activation changes, and gradients that feed three independent discovery paths. Discovered components receive confidence scores from all three measurement domains, triangulated into a unified score. Candidates are then validated through a 7-phase ablation protocol, producing a circuit graph with functional classifications and steering vectors. The dashed boundaries highlight DCAF's key architectural distinction: circuit candidates are identified *prospectively* through training perturbations, with ablation serving as *validation* rather than discovery.

4 Multi-Path Discovery

DCAF uses percentile-based significance predicates to determine whether a projection's weight delta is large enough to be considered behaviorally relevant. A projection is *significant* if its RMS-normalized delta norm exceeds the $\tau_{\text{sig}} = 85$th percentile across all projections; it is *insignificant* (for baseline filtering) if its norm falls below the $\tau_{\text{base}} = 50$th percentile. Full definitions of the percentile threshold function, all configurable parameters, and their rationale appear in Appendix C.

DCAF employs three complementary discovery paths. Each path identifies candidate projections using different evidence, which are then mapped to components via μ_{proj}. This section provides the discovery formulas with brief explanations; full methodology details are in the domain analysis sections (§5–§7). All discovered components then receive confidence scores from all three measurement domains.

4.1 Path 1: Weight-Based Discovery ($\mathcal{H}_W$)

Weight-based discovery identifies projections that changed substantially during behavioral training—the most direct evidence of optimization pressure. Full methodology for computing these terms is in §5.

Definition 4.1 (Signal Presence)**.** *Counts how many training signals produced significant changes at a projection:*

$$n^{(proj)} = \sum_{i \in \mathcal{T}} sig(proj, i) \tag{12}$$

where $sig(proj, i)$ tests if the RMS-normalized delta norm of the weight delta exceeds the significance threshold.

Definition 4.2 (Signal Effectiveness)**.** *Measures how successfully each training signal achieved behavioral change:*

$$eff(i) = normalize_{[0,1]}\left(\Delta_{improve}(i) + \beta \cdot threshold(i)\right) \tag{13}$$

where $\Delta_{improve}$ is the normalized improvement (loss reduction for SFT, margin improvement for preference methods) and $threshold(i)$ rewards training that crossed a behavioral threshold. See §5 for full formulas.

Definition 4.3 (Aggregated Cluster Deltas)**.** *For a projection, the effectiveness-weighted average delta matrix from each behavioral cluster:*

$$\bar{\delta}_+^{(proj)} = \frac{\sum_{i \in \mathcal{T}^+} eff(i) \cdot \Delta W_i^{(proj)}}{\sum_{i \in \mathcal{T}^+} eff(i)}, \quad \bar{\delta}_-^{(proj)} = \frac{\sum_{j \in \mathcal{T}^-} eff(j) \cdot \Delta W_j^{(proj)}}{\sum_{j \in \mathcal{T}^-} eff(j)} \tag{14}$$

Each cluster delta is a full matrix of the same shape as the projection weight matrix, representing the net direction and magnitude of change induced by target vs. opposite behavioral training.

Definition 4.4 (Opposition Degree)**.** *Measures bidirectional control strength—how strongly a projection is pushed in opposite directions by the two behavioral clusters. Computed via cosine similarity on the flattened delta matrices:*

$$cos_opp^{(proj)} = \cos\Big(flatten(\bar{\delta}_+^{(proj)}),\ flatten(\bar{\delta}_-^{(proj)})\Big) \tag{15}$$

$$opp_degree(proj) = \max\Big(0,\ -cos_opp^{(proj)}\Big) \tag{16}$$

When $\mathcal{T}^+$ and $\mathcal{T}^-$ push the projection in opposite directions, $cos_opp \approx -1$ and $opp_degree \approx 1$. When they push in the same direction, $cos_opp \approx +1$ and $opp_degree = 0$. If either cluster delta has near-zero norm ($< 10^{-8}$), both values are 0. Cosine similarity is self-normalizing, requiring no layer-specific calibration.

Definition 4.5 (Weight-Based Discovery Score)**.** *Combines signal presence with opposition as additive bonus, with baseline filtering:*

$$S_W^{(proj)} = \min\left(1,\ \frac{\sum_{i \in \mathcal{T} \setminus \mathcal{T}^0} eff(i)^q \cdot sig(proj, i)}{\sum_{i \in \mathcal{T} \setminus \mathcal{T}^0} eff(i)^q} + \alpha \cdot opp_degree(proj)\right) \cdot \mathbf{1}\left[\forall t \in \mathcal{T}^0 : \overline{sig}(proj, t)\right] \tag{17}$$

where $q = 2$ (effectiveness power), $\alpha = 0.2$ (opposition bonus weight). The baseline filter (indicator) excludes projections responding to any neutral training.

Definition 4.6 (Weight-Based Discovery Set)**.** *Projections discovered via weight changes, mapped to components:*

$$\mathcal{H}_W = \{proj \in \mathcal{P} : S_W^{(proj)} \geq \tau_W\} \tag{18}$$

where τ_W is the weight discovery threshold (default: 0.3).

Definition 4.7 (Bidirectionality Status)**.** *Flags projections showing sufficient opposition:*

$$bidirectional(proj) = \mathbf{1}\left[opp_degree(proj) > \tau_{opp}\right] \tag{19}$$

where $\tau_{opp} = 0.3$ (default). A component is bidirectional if any of its projections exceeds τ_{opp}.

4.2 Path 2: Activation-Based Discovery ($\mathcal{H}_A$)

Activation-based discovery identifies **leverage points**: projections where small weight changes produce large representational effects. These are missed by weight-based discovery but may be critical for behavioral control. This path uses activation methodology from §6.

Definition 4.8 (Component Activation Magnitude)**.** *For component k, aggregate activation changes across signals and probe types:*

$$m_{agg}^{(k)} = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \frac{1}{|\Pi|} \sum_{\pi \in \Pi} m_i^{(k,\pi)} \tag{20}$$

where $m_i^{(k,\pi)}$ is the activation magnitude (see §6). Activations are hooked at component output level.

Definition 4.9 (Component Screening)**.** *Flag components with notable activation changes using a generous threshold:*

$$\mathcal{K}_{flagged} = \left\{k \in \mathcal{K} : m_{agg}^{(k)} \geq \Phi_{\tau_{comp}}\left(\{m_{agg}^{(k')}\}_{k' \in \mathcal{K}}\right)\right\} \tag{21}$$

where $\tau_{comp} = 70$ (default). The 70th percentile is deliberately generous to avoid missing leverage points.

Definition 4.10 (Projection-Level Activation Score)**.** *For projections within flagged components, compute a score reflecting both the component's activation impact and the projection's weight magnitude (projections with larger norms have more leverage over activations):*

$$S_A^{(proj)} = \begin{cases} \dfrac{m_{agg}^{(\mu_{proj}(proj))}}{\max_{k \in \mathcal{K}} m_{agg}^{(k)}} \cdot w_{proj}(proj) & \text{if } \mu_{proj}(proj) \in \mathcal{K}_{flagged} \\ 0 & \text{otherwise} \end{cases} \tag{22}$$

where $w_{proj}(proj) = \|W^{(proj)}\|_F \;/\; \max_{proj':\mu_{proj}(proj')=\mu_{proj}(proj)} \|W^{(proj')}\|_F$ is the projection's relative weight norm within its component.

Definition 4.11 (Activation-Based Discovery Set)**.** *Projections discovered via activation analysis:*

$$\mathcal{H}_A = \left\{proj \in \mathcal{P} : \mu_{proj}(proj) \in \mathcal{K}_{flagged} \wedge S_A^{(proj)} \geq \Phi_{\tau_{act}}\left(\{S_A^{(proj')}\}_{proj':\mu_{proj}(proj') \in \mathcal{K}_{flagged}}\right)\right\} \tag{23}$$

The 85th percentile within flagged components captures the most impactful projections.

Remark 4.1 (Two-Stage Design)**.** *The two-stage design (generous component screening, strict projection filtering) balances coverage against cost. Without screening, every projection would require analysis; the 70th percentile flags ∼30% of components. This catches leverage points (small $\|\Delta W\|_{RMS}$, large $\|\Delta A\|$) that Path 1 misses.*

4.3 Path 3: Gradient-Based Discovery ($\mathcal{H}_G$)

Gradient-based discovery identifies projections with **high behavioral gradients**: projections that would affect behavior if modified, regardless of whether training actually changed them. This is purely mathematical prediction, complementing the empirical observation of Paths 1 and 2.

Definition 4.12 (Behavioral Objective)**.** *For each training signal t_i, let $\mathcal{O}_i$ denote the objective function that signal optimizes:*

$$\mathcal{O}_i = \begin{cases} \mathbb{E}_{x\sim D_i}[-\log p(y|x)] & \text{if } t_i \in \{\text{SFT signals}\} \\ -\mathbb{E}_{(x,y_w,y_l)\sim D_i}[M(y_w, y_l|x)] & \text{if } t_i \in \{\text{DPO, SimPO signals}\} \\ -\mathbb{E}_{x\sim D_i}[R(x)] & \text{if } t_i \in \{\text{RL signals}\} \\ -\mathbb{E}_{x\sim D_i}[\mathbb{E}_{y\sim\pi}[R(x,y)]] & \text{if } t_i \in \{\text{GRPO signals}\} \end{cases} \tag{24}$$

where M is the preference margin, R is the reward, and negation converts maximization objectives to minimization form. Signal-specific formulations match those in Definition 5.1.

Definition 4.13 (Behavioral Gradient)**.** *For a projection, compute the effectiveness-weighted behavioral gradient magnitude. The gradient of $\mathcal{O}_i$ with respect to the projection weight matrix is itself a matrix of the same shape; its Frobenius norm measures the total sensitivity:*

$$g^{(proj)} = \sum_{i\in\mathcal{T}^+\cup\mathcal{T}^-} \mathit{eff}(i) \cdot \left\| \frac{\partial \mathcal{O}_i}{\partial W^{(proj)}} \right\|_F \tag{25}$$

This weights gradient magnitudes by training effectiveness, ensuring successful training runs contribute more to the discovery signal.

Definition 4.14 (Gradient-Based Discovery Set)**.** *Projections discovered via gradient analysis:*

$$\mathcal{H}_G = \left\{ proj \in \mathcal{P} : g^{(proj)} \geq \Phi_{\tau_{grad}}\left(\{g^{(proj')}\}_{proj'\in\mathcal{P}}\right)\right\} \tag{26}$$

where $\tau_{grad} = 85$ (default).

Remark 4.2 (Gradient Discovery Interpretation)**.** *Path 3 discovers projections that the loss landscape identifies as behaviorally relevant, independent of training dynamics:*

- *High gradient, low weight change: Projection is relevant but optimization didn't modify it (possibly due to regularization, learning rate, or saddle points)*
- *High gradient, high weight change: Converging evidence from mathematical prediction and empirical observation*
- *Low gradient, high weight change: Weight changed but not due to behavioral pressure (possible confound)*

Gradient-based discovery is purely predictive—it identifies what could *affect behavior. Ablation confirms what* does *affect behavior.*

4.4 Discovery Integration

Definition 4.15 (Unified Discovery Set)**.** *Each discovery path returns a set of projection IDs. These are mapped to components via μ_{proj} and unioned:*

$$\mathcal{H}_{disc} = \mathit{components}(\mathcal{H}_W) \cup \mathit{components}(\mathcal{H}_A) \cup \mathit{components}(\mathcal{H}_G) \tag{27}$$

where $\mathit{components}(S) = \{\mu_{proj}(proj) : proj \in S\}$. Discovery is inclusive: a component flagged by any *path becomes a candidate for confidence scoring and ablation validation.*

Definition 4.16 (Component Discovery Attribution)**.** *For each component $k \in \mathcal{H}_{disc}$, record which discovery paths flagged it and aggregate projection-level scores:*

$$disc_paths(k) = \{W : \exists\, proj \in \mu_{proj}^{-1}(k) \cap \mathcal{H}_W\} \cup \{A : \exists\, proj \in \mu_{proj}^{-1}(k) \cap \mathcal{H}_A\} \cup \{G : \exists\, proj \in \mu_{proj}^{-1}(k) \cap \mathcal{H}_G\} \tag{28}$$

$$paths(k) = |disc_paths(k)| \tag{29}$$

Component-level discovery scores are aggregated from projections via max:

$$S_W(k) = \max_{proj \in \mu_{proj}^{-1}(k) \cap \mathcal{H}_W} S_W^{(proj)}, \qquad S_G(k) = \max_{proj \in \mu_{proj}^{-1}(k) \cap \mathcal{H}_G} g^{(proj)} \tag{30}$$

The activation score $S_A(k) = m_{agg}^{(k)} / \max_{k'} m_{agg}^{(k')}$ is already component-level. Multi-path discovery ($paths(k) \geq 2$) indicates converging evidence from independent methods.

Remark 4.3 (Path Independence Principle)**.** *The three discovery paths are deliberately independent:*

- *Path 1 uses weight deltas only*
- *Path 2 uses activation deltas only (after component screening)*
- *Path 3 uses gradients only*

This independence means multi-path discovery is strong evidence: three different measurement approaches identified the same component as relevant. The unified confidence score (§9) incorporates a multi-path bonus to reflect this convergent evidence.

Remark 4.4 (Discovery vs. Confidence)**.** *Discovery and confidence serve different purposes:*

- ***Discovery*** *(this section): Which components should we analyze? Uses thresholds to filter.*
- ***Confidence*** *(§9): How confident are we in each discovered component? Uses continuous scores from all three domains.*

A component discovered via Path 2 (activation) still receives confidence contributions from the weight domain (C_W, §5), activation domain (C_A, §6), and geometry domain (C_G, §7). Discovery determines what *to analyze; domain analysis determines* strength of evidence.

5 Weight Analysis

This section provides full methodology for weight-based analysis. Discovery formulas appear in §3.1; this section covers implementation details, edge cases, and design rationale. All discovered components ($\mathcal{H}_{\text{disc}}$) receive weight confidence scores C_W, regardless of which path discovered them. Weight analysis operates at **projection level**: each projection matrix is analyzed individually, then results are aggregated to component level for integration with the other domains.

5.1 Signal Effectiveness

Definition 5.1 (Signal Effectiveness—Full Formula)**.** *For training signal $t_i \in \mathcal{T}^+ \cup \mathcal{T}^-$, define the raw effectiveness score:*

$$eff_{raw}(i) = \Delta_{improve}(i) + \beta \cdot threshold(i) \tag{31}$$

where $\Delta_{improve}(i)$ is the normalized improvement metric, computed from the baseline model $\mathcal{M}_0$ to the peak checkpoint:

$$\Delta_{improve}(i) = \begin{cases} \dfrac{\mathcal{L}_{pre}(i) - \mathcal{L}_{peak}(i)}{\mathcal{L}_{pre}(i)} & \text{if } t_i \in \{\text{SFT signals}\} \\ \dfrac{M_{peak}(i) - M_{pre}(i)}{|M_{pre}(i)| + \varepsilon} & \text{if } t_i \in \{\text{pairwise preference signals (DPO, SimPO, etc.)}\} \\ \dfrac{\mathbb{E}[R_{peak}(i)] - \mathbb{E}[R_{pre}(i)]}{|\mathbb{E}[R_{pre}(i)]| + \varepsilon} & \text{if } t_i \in \{\text{GRPO signals}\} \\ \dfrac{R_{peak}(i) - R_{pre}(i)}{|R_{pre}(i)| + \varepsilon} & \text{if } t_i \in \{\text{RL signals}\} \end{cases} \tag{32}$$

where $\mathcal{L}$ is task loss (SFT), M is preference margin (DPO/SimPO), $\mathbb{E}[R]$ is expected reward over generated responses (GRPO), and R is cumulative episode reward (RL). All "peak" subscripts refer to evaluation at the peak checkpoint $W_{i,peak}$ (§3).

The threshold bonus rewards training that achieves actual behavioral change:

$$\textit{threshold}(i) = \begin{cases} 1 & \text{if peak checkpoint crosses behavioral threshold} \\ 0 & \text{otherwise} \end{cases} \tag{33}$$

Examples: preference margin $M_{peak} > 0$, refusal rate exceeds target percentage, reward exceeds baseline. The threshold bonus weight β (default: $\beta = 0.4$) distinguishes "trained hard but didn't implement behavior" from "actually achieved behavioral change."

Normalize to $[0, 1]$ with outlier handling via clipping at 95th percentile:

$$\textit{eff}(i) = \max\left(0, \frac{\textit{eff}_{raw}(i) - \min_j \textit{eff}_{raw}(j)}{\textit{clip}_{95}(\max_j \textit{eff}_{raw}(j)) - \min_j \textit{eff}_{raw}(j)}\right) \tag{34}$$

Negative effectiveness (training degradation) is clipped to zero.

Remark 5.1 (Effectiveness Power Parameter)**.** *Effectiveness is defined once with the threshold bonus, then different powers are applied based on context:*

- ***Aggregated cluster deltas:*** *Use $\textit{eff}(i)^1$ (linear). When computing average deltas, all effective signals contribute proportionally—we want accurate direction estimates.*
- ***Weight confidence:*** *Use $\textit{eff}(i)^q$ with $q = 2$ (power transform). When determining confidence, successful runs should dominate over failed runs.*

The threshold bonus (via β) applies everywhere through the base effectiveness; the power parameter q controls emphasis in confidence weighting.

5.2 RMS Normalization of Projection Deltas

Definition 5.2 (RMS-Normalized Frobenius Norm)**.** *For a projection weight delta matrix $\Delta W_i^{(proj)} \in \mathbb{R}^{m \times n}$, the RMS-normalized Frobenius norm is:*

$$\|\Delta W_i^{(proj)}\|_{RMS} = \frac{\|\Delta W_i^{(proj)}\|_F}{\sqrt{m \cdot n}} \tag{35}$$

where (m, n) is the projection matrix shape. This equals the root-mean-square of all elements in the delta matrix: $\sqrt{\frac{1}{mn}\sum_{a,b}(\Delta W_{a,b})^2}$. All weight-domain significance comparisons (sig, $\overline{sig}$, S_W, C_W) use $\|\cdot\|_{RMS}$ in place of $\|\cdot\|_F$.

Remark 5.2 (Dimensional Bias Correction)**.** *Raw Frobenius norms are systematically biased by projection dimensions. For Gemma 2 2B, an MLP gate projection (*2304×9216*,* $mn \approx 21.2M$ *parameters) produces* $\sim 6\times$ *larger raw Frobenius norms than an attention Q projection (*2304×256*,* $mn \approx 0.59M$ *parameters) for identical per-element changes, because* $\sqrt{21.2M/0.59M} \approx 6.0$*. Percentile-based significance thresholds applied to raw norms therefore systematically favor MLP projections and miss genuinely significant attention projection changes. Dividing by* $\sqrt{m \cdot n}$ *normalizes to per-element scale, eliminating this architectural bias. The normalizer is a deterministic architectural constant—always nonzero, precomputable once per model.*

Definition 5.3 (Base-Relative Delta (Diagnostic Only))**.** *For diagnostic interpretation, compute the base-weight relative normalization:*

$$\|\Delta W\|_{rel} = \frac{\|\Delta W\|_F}{\|W_0\|_F + \varepsilon_{rms}} \tag{36}$$

where W_0 *is the weight matrix of the configured baseline model* $\mathcal{M}_0$ *(Definition 3.1) and* ε_{rms} *(default:* 10^{-8}*) prevents division by zero. This captures "what fraction of the projection shifted." Stored as* `base_relative_delta` *in projection results. This metric is* ***not*** *used in any discovery or confidence formula—it is metadata for interpretation only.*

Remark 5.3 (RMS vs. Relative Divergence)**.** *When RMS and relative rankings diverge for a projection, the divergence is diagnostically informative:*

Pattern	***Condition***	***Interpretation***
High RMS, low relative	*Large* $\|\Delta W\|_{RMS}$*, small* $\|\Delta W\|_{rel}$	*Large absolute change in a high-magnitude projection*
Low RMS, high relative	*Small* $\|\Delta W\|_{RMS}$*, large* $\|\Delta W\|_{rel}$	*Small-magnitude projection shifted proportionally a lot*
Both high	*Large* $\|\Delta W\|_{RMS}$ *and* $\|\Delta W\|_{rel}$	*Unambiguously significant*
Both low	*Small* $\|\Delta W\|_{RMS}$ *and* $\|\Delta W\|_{rel}$	*Unambiguously insignificant*

Low-RMS / high-relative projections may warrant investigation despite failing significance thresholds—they represent proportionally large shifts in small-magnitude projections that RMS normalization, by design, does not upweight.

5.3 Opposition Degree

Definition 5.4 (Opposition Degree—Full Methodology)**.** *Opposition degree measures how strongly a projection is pushed in opposite directions by the two behavioral clusters. The measure uses cosine similarity on the flattened cluster delta matrices (Definition 4.3):*

$$cos_opp^{(proj)} = \cos\Big(flatten(\bar{\delta}_+^{(proj)}),\ flatten(\bar{\delta}_-^{(proj)})\Big) \tag{37}$$

$$opp_degree(proj) = \max\Big(0,\ -cos_opp^{(proj)}\Big) \tag{38}$$

The flatten operation reshapes a matrix $\in \mathbb{R}^{m \times n}$ *into a vector* $\in \mathbb{R}^{mn}$*, enabling cosine similarity to measure the directional relationship between the two cluster deltas in full weight space. If either cluster delta has near-zero Frobenius norm (*$< 10^{-8}$*), both cos_opp and opp_degree are set to 0.*

Remark 5.4 (Why Cosine Similarity)**.** *Cosine similarity on flattened matrices is self-normalizing: it measures the* direction *of change without being affected by magnitude differences between layers, projections, or training signals. This eliminates the need for layer-specific calibration.*

High opposition (opp_degree ≈ 1) indicates the projection is a bidirectional control point—the same weight matrix mediates both the target behavior and its opposite, with $\mathcal{T}^+$ and $\mathcal{T}^-$ pushing in anti-aligned directions through the full weight space.

5.4 SVD Diagnostics on Projection Deltas

Definition 5.5 (SVD Decomposition of Cluster Deltas)**.** *Compute the singular value decomposition of the aggregated cluster delta matrices:*

$$\bar{\delta}_+^{(proj)} = U_+ \, S_+ \, V_{h,+}^T \tag{39}$$

$$\bar{\delta}_-^{(proj)} = U_- \, S_- \, V_{h,-}^T \tag{40}$$

where $S_+ = diag(\sigma_1^+, \sigma_2^+, \ldots)$ with $\sigma_1^+ \geq \sigma_2^+ \geq \cdots$, and analogously for S_-.

Two derived metrics characterize the structure of weight changes:

Rank-1 fraction:

$$rank1(proj) = \frac{(\sigma_1)^2}{\sum_i (\sigma_i)^2} \tag{41}$$

where σ_1 is the largest singular value of the cluster delta matrix. Measures how concentrated the weight change is along a single input→output mapping. High rank-1 fraction (∼0.7+) indicates training modified one specific pathway; low (∼0.3) indicates distributed change across many dimensions.

Spectral opposition:

$$spec_opp(proj) = -\cos(V_{h,+}[0],\ V_{h,-}[0]) \tag{42}$$

where $V_{h,+}[0]$ and $V_{h,-}[0]$ are the top right singular vectors of the target and opposite cluster deltas respectively. Measures whether $\mathcal{T}^+$ and $\mathcal{T}^-$ oppose along the primary axis of change. Range $[-1, 1]$; values near 1 indicate clean opposition along the dominant change direction.

Remark 5.5 (SVD Interpretation Patterns)**.** *The two metrics jointly characterize four regimes:*

- *High rank-1 fraction + high spectral opposition: clean single-axis bidirectional control. Training modified one specific mapping in opposite directions.*
- *High rank-1 fraction + low spectral opposition: concentrated change but $\mathcal{T}^+$ and $\mathcal{T}^-$ modified different mappings.*
- *Low rank-1 fraction + high spectral opposition: distributed change but consistently opposed. Multiple features contribute to a shared bidirectional axis.*
- *Low rank-1 fraction + low spectral opposition: diffuse, potentially noisy. Further investigation needed.*

Report per projection: rank1(proj), top singular value σ_1, top 3 singular values $[\sigma_1, \sigma_2, \sigma_3]$, spec_opp(proj). These diagnostics are optional and do not enter the confidence formula; they provide interpretive depth for projections already flagged as significant.

5.5 Weight Confidence

Definition 5.6 (Projection-Level Weight Confidence)**.** *For any projection in a discovered component ($\mu_{proj}(proj) \in \mathcal{H}_{disc}$), the weight confidence uses the same formula as the discovery score:*

$$C_W^{(proj)} = \min\left(1, \frac{\sum_{i \in \mathcal{T} \setminus \mathcal{T}^0} \mathit{eff}(i)^q \cdot \mathit{sig}(proj, i)}{\sum_{i \in \mathcal{T} \setminus \mathcal{T}^0} \mathit{eff}(i)^q} + \alpha \cdot \mathit{opp_degree}(proj)\right) \cdot \mathbf{1}\left[\forall t \in \mathcal{T}^0 : \overline{\mathit{sig}}(proj, t)\right] \tag{43}$$

Computed for all *projections of discovered components, not just those in $\mathcal{H}_W$. The formula has three components:*

- ***Signal presence*** *(first term): Effectiveness-weighted fraction of behavioral signals that changed this projection significantly. The power transform ($q = 2$) makes successful training runs contribute much more than failed runs.*
- ***Opposition bonus*** *($\alpha \cdot opp_degree$): Adds confidence when the projection shows bidirectional control.*
- ***Baseline filter*** *(indicator): Zeros out confidence if the projection responds to neutral training, excluding general-purpose projections.*

Definition 5.7 (Component-Level Weight Confidence)**.** *The component's weight confidence is the maximum across its projections:*

$$C_W^{(k)} = \max_{proj \in \mu_{proj}^{-1}(k)} C_W^{(proj)} \tag{44}$$

The most implicated projection drives the component's weight confidence. If W_V shows strong opposition, the head is behaviorally relevant regardless of what W_Q does.

Remark 5.6 (Weight Confidence for Non-Weight Discoveries)**.** *A component discovered via Path 2 (activation) with low C_W indicates a leverage point: small weight changes produce large activation effects. A component discovered via Path 3 (gradient) with low C_W indicates the gradient landscape predicted behavioral relevance but training didn't modify the component. Both patterns are scientifically interesting and worth investigating via ablation.*

6 Activation Analysis

This section provides full methodology for activation-based analysis. Discovery formulas appear in §3.2; this section covers probe types, activation capture, and confidence computation.

6.1 Probe Types

Definition 6.1 (Universal Probe Types)**.** *The three probe types capture behavioral processing at different stages:*

$$\Pi = \{\pi_{detect}, \pi_{decide}, \pi_{eval}\} \tag{45}$$

Abstract definitions:

$\pi_{\mathbf{detect}}$**:** ***Behavioral trigger detection*** *(upstream) — Does the model encode/recognize behavioral content?*

$\pi_{\mathbf{decide}}$**:** ***Behavioral decision point*** *(middle) — Does the model make the behavioral choice?*

$\pi_{\mathbf{eval}}$**:** ***Behavioral path evaluation*** *(parallel) — Does the model prefer the behavioral path?*

Each probe type includes behavior-positive, behavior-negative, and (for $\pi_{detect}, \pi_{decide}$) neutral exemplars.

6.2 Activation Capture

Definition 6.2 (Activation Snapshot)**.** *An activation snapshot captures the internal representations at a specific component when the model processes probe inputs. Each row corresponds to one probe example, each column to one dimension of the component's activation space.*

$$A_i^{(k,\pi)} \in \mathbb{R}^{n_\pi \times d_k} \tag{46}$$

Definition 6.3 (Activation Delta and Magnitude)**.** *The activation delta measures how a component's representations changed after training relative to the baseline model $\mathcal{M}_0$ (Definition 3.1). The magnitude m summarizes this as a single scalar—the average norm of change across all probe examples—allowing comparison across components.*

$$\Delta A_i^{(k,\pi)} = A_i^{(k,\pi)} - A_0^{(k,\pi)}, \qquad m_i^{(k,\pi)} = \frac{1}{n_\pi} \sum_{j=1}^{n_\pi} \left\| \Delta A_i^{(k,\pi)}[j] \right\|_2 \tag{47}$$

where $A_0^{(k,\pi)}$ denotes activations captured from $\mathcal{M}_0$.

Definition 6.4 (Component Significance)**.** *A component is significant for a given signal and probe if its activation magnitude exceeds the τ_{act} percentile across all components. This identifies components whose representations changed substantially relative to others.*

$$sig_A(k, i, \pi) = \mathbf{1}\left[m_i^{(k,\pi)} \geq \Phi^A_{\tau_{act}} \left(\{ m_i^{(k',\pi)} \}_{k' \in \mathcal{K}} \right) \right] \tag{48}$$

6.3 Activation Confidence

Definition 6.5 (Activation Confidence C_A)**.** *Activation confidence measures what fraction of (signal, probe) combinations show significant activation changes at a component. Higher values indicate the component consistently responds to behavioral training across multiple signals and probe types.*

$$n_A^{(k)} = \sum_{i \in \mathcal{T}} \sum_{\pi \in \Pi} sig_A(k, i, \pi), \qquad C_A^{(k)} = \frac{n_A^{(k)}}{|\mathcal{T}| \cdot |\Pi|} \tag{49}$$

Activation confidence is inherently component-level throughout; hooks capture at head outputs and MLP outputs.

Remark 6.1 (Probe Construction: Matched Contrast Pairs)**.** *Effective probe construction requires* ***matched contrast pairs*** *where behavior-positive and behavior-negative inputs differ by exactly 1–3 tokens, holding all other features constant (topic, length, syntax, vocabulary, position). This isolates the behavioral mechanism from confounds, following the ABC counterfactual design introduced by Wang et al. [2023]. For example, in IOI circuit discovery, behavior-positive probes use standard sentences with a repeated subject name ("Then, Mary and John went to the store. John gave a drink to"), while behavior-negative probes replace the repeated name with a novel third name ("Then, Mary and Chris went to the store. Chris gave a drink to")—a single-word pivot that removes the duplicate-token signal while preserving all other structure. The same principle applies across behaviors: safety probes should differ only in the presence of harmful content, truthfulness probes only in factual accuracy, and so on. Poorly controlled probes—where positive and negative inputs differ in length, topic, or style—may cause DCAF to identify components that encode those confounds rather than the target behavior.*

Remark 6.2 (No Opposition in Activation Domain)**.** *Activation criteria use significance only—no opposition predicate. High-dimensional activation vectors lack well-defined "opposite direction." Opposition evidence comes from weight analysis (§5).*

7 Geometry Analysis

The geometry domain analyzes how behavior is represented in activation space. It measures whether training created or strengthened directions that separate behavioral classes.

7.1 Contrastive Direction Extraction

Definition 7.1 (Activation Sets)**.** *For component k and model state $\mathcal{M}_i$:*

$$A_i^+(k) = \{\textit{activations on behavior-positive prompts}\} \tag{50}$$

$$A_i^-(k) = \{\textit{activations on behavior-negative prompts}\} \tag{51}$$

Definition 7.2 (Whitened Contrastive Direction)**.**

$$\mu^+ = \mathbb{E}[A^+], \quad \mu^- = \mathbb{E}[A^-], \quad \Sigma_w = \frac{1}{2}\left(Cov(A^+) + Cov(A^-)\right) \tag{52}$$

$$d_i^{(k)} = \frac{(\Sigma_w + \lambda I)^{-1}(\mu^+ - \mu^-)}{\|(\Sigma_w + \lambda I)^{-1}(\mu^+ - \mu^-)\|} \tag{53}$$

Whitening accounts for variance structure, providing stable direction estimates.

Definition 7.3 (Behavioral Subspace)**.** *For subspace dimension $r > 1$, let $U_i^{(k)} \in \mathbb{R}^{d_k \times r}$ be the top-r discriminant directions from LDA or PCA on class-centered data.*

Subspace similarity via principal angles $\theta_1, \ldots, \theta_r$:

$$sim(U_1, U_2) = \frac{1}{r}\sum_{j=1}^{r} \cos(\theta_j) \tag{54}$$

Remark 7.1 (Subspace Extraction Methods)**.** *Three approaches for extracting behavioral subspaces when $r > 1$:*

Method 1 (PCA on class-centered data): *Center A^+ and A^- by their respective means, concatenate, apply PCA. Top-r components capture between-class variation.*

Method 2 (Multi-class LDA): *When more than two behavioral classes exist (e.g., target/opposite/neutral), multi-class LDA returns up to $(num_classes - 1)$ discriminant directions.*

Method 3 (Linear probe weights): *Train a linear classifier on activations $\rightarrow$ behavior label. Apply SVD to the weight matrix; top-r singular vectors define the subspace.*

Default: Start with $r = 1$. If cluster coherence is low, increase r.

7.2 Direction Dynamics

Definition 7.4 (Direction Emergence)**.**

$$\Delta\|d\|^{(k)} = \|d_{post}^{(k)}\| - \|d_{pre}^{(k)}\| \tag{55}$$

Positive: training strengthened behavioral separation. Near-zero: direction existed, training didn't change it.

Definition 7.5 (Direction Rotation)**.**

$$\rho^{(k)} = \cos(d_{pre}^{(k)}, d_{post}^{(k)}) \tag{56}$$

$\rho \approx 1$: same direction, scaled. $\rho \in [0.5, 0.9]$: partial rotation. $\rho < 0.5$: substantial rotation (new mechanism).

7.3 Confound Independence

Definition 7.6 (Confound Direction)**.** *For a known confound c that may correlate with the target behavior, extract its direction from confound-positive vs. confound-negative activations:*

$$d_c^{(k)} = \textit{whitened_direction}(A_{c^+}, A_{c^-}) \tag{57}$$

Remark 7.2 (Behavior-Specific Confounds)**.** *The relevant confound depends on the behavior under study:*

- ***Safety behaviors:*** *Generic refusal template. Computed from non-behavioral refusals vs. compliance. Non-behavioral refusal sources include:*
  - *Out-of-domain requests ("What's the weather on Mars?")*
  - *Nonsense or malformed prompts*
  - *Capability limitation responses ("I cannot access the internet")*
  - *Clarification requests ("Could you be more specific?")*

  If the behavioral direction aligns with this refusal template, you've found "how to refuse" rather than "when to refuse."
- ***Verbosity:*** *Response length. Computed from long vs. short responses on neutral prompts.*
- ***Formality:*** *Register differences. Computed from formal vs. casual text.*
- ***Other behaviors:*** *May have no obvious confound, in which case this check is omitted.*

Definition 7.7 (Confound Independence)**.**

$$\rho_c^{(k)} = 1 - \frac{1}{|\mathcal{T}^+|} \sum_{i \in \mathcal{T}^+} \left| \cos(d_i^{(k)}, d_c^{(k)}) \right| \tag{58}$$

High ρ_c: direction captures behavior-specific features. Low ρ_c: direction is contaminated by the confound.

Remark 7.3 (Confound Independence Thresholds)**.** *Interpretation guidelines:*

- $\rho_c > 0.7$*: Clean—direction captures behavior-specific features*
- $\rho_c \in [0.3, 0.7]$*: Mixed signal—investigate further*
- $\rho_c < 0.3$*: Contaminated—direction is dominated by confound*

If contaminated, consider orthogonalizing: $d_{clean} = d - (d \cdot d_c) \cdot d_c$.

7.4 Cross-Signal Alignment

Definition 7.8 (Alignment Matrix)**.** *The alignment matrix captures how similar the behavioral directions are across different training signals. High alignment within a cluster indicates the signals found the same behavioral mechanism; opposition between clusters indicates the target and opposite behaviors use the same axis in opposite directions.*

$$M^{(k)}[i, j] = \cos(d_i^{(k)}, d_j^{(k)}) \tag{59}$$

Definition 7.9 (Opposition Verification)**.**

$$opp^{(k)} = -\cos(d_{target}^{(k)}, d_{opposite}^{(k)}) \tag{60}$$

$opp \approx 1$: behaviors are semantic opposites on same axis. $opp \approx 0$: orthogonal mechanisms. Negative: unexpected (same direction).

Definition 7.10 (LRS Components from Alignment)**.** *Extract four alignment-based metrics from the alignment matrix:*

$$coh_+^{(k)} = \frac{1}{|\mathcal{T}^+|^2} \sum_{i,j \in \mathcal{T}^+} \cos(d_i^{(k)}, d_j^{(k)}) \qquad \textit{(target cluster coherence)} \tag{61}$$

$$coh_-^{(k)} = \frac{1}{|\mathcal{T}^-|^2} \sum_{i,j \in \mathcal{T}^-} \cos(d_i^{(k)}, d_j^{(k)}) \qquad \textit{(opposite cluster coherence)} \tag{62}$$

$$opp^{(k)} = -\frac{1}{|\mathcal{T}^+||\mathcal{T}^-|} \sum_{i \in \mathcal{T}^+, j \in \mathcal{T}^-} \cos(d_i^{(k)}, d_j^{(k)}) \qquad \textit{(cross-cluster opposition)} \tag{63}$$

$$orth^{(k)} = 1 - \frac{1}{|\mathcal{T}^+ \cup \mathcal{T}^-|} \sum_{i \in \mathcal{T}^+ \cup \mathcal{T}^-} \left|\cos(d_i^{(k)}, d_{t_0}^{(k)})\right| \qquad \textit{(baseline orthogonality)} \tag{64}$$

7.5 Predictivity

Definition 7.11 (Direction Predictivity)**.** *For a contrastive direction d, compute how well it predicts behavior on held-out examples:*

$$pred(d) = AUC(\{a_j \cdot d\}, \{y_j\}) \tag{65}$$

where a_j is the activation for example j, $y_j \in \{0, 1\}$ is the behavior label, and AUC is the area under the ROC curve for the linear classifier $sign(a \cdot d)$.

Definition 7.12 (Predictivity Gain)**.** *Compare peak-checkpoint vs. baseline ($\mathcal{M}_0$) predictivity:*

$$\Delta_{pred}^{(k)} = \frac{1}{|\mathcal{T}^+|} \sum_{i \in \mathcal{T}^+} \left(AUC^{peak}(d_i^{(k)}) - AUC^{pre}(d_i^{(k)}) \right) \tag{66}$$

Positive: training made behavior more linearly separable. Near-zero: direction was already predictive. Negative: unusual (training degraded separability).

Definition 7.13 (Generalization)**.** *Compare out-of-distribution vs. within-distribution predictivity:*

$$gen^{(k)} = \frac{1}{|\mathcal{T}^+|} \sum_{i \in \mathcal{T}^+} \frac{AUC^{OOD}(\{a_j \cdot d_i^{(k)}\}, \{y_j\})}{AUC^{within}(\{a_j \cdot d_i^{(k)}\}, \{y_j\})} \tag{67}$$

$gen \approx 1$: direction generalizes well. $gen < 0.8$: direction may be overfitting to training distribution.

Remark 7.4 (Generalization Gap Interpretation)**.** *Let $gap = pred^{within} - pred^{OOD}$:*

- *$gap < 0.1$: Direction captures generalizable concept*
- *$gap > 0.2$: Direction overfits to template artifacts*
- *$pred^{OOD} > 0.7$: Strong evidence of real concept direction regardless of gap*

7.6 Linear Representation Score

Definition 7.14 (Linear Representation Score (LRS))**.** *The Linear Representation Score is a Hölder (power) mean of the six geometric metrics, providing a single number capturing the overall quality of linear behavioral representation at a component.*

Let $x_1^{(k)} = coh_+^{(k)}$, $x_2^{(k)} = coh_-^{(k)}$, $x_3^{(k)} = opp^{(k)}$, $x_4^{(k)} = orth^{(k)}$, $x_5^{(k)} = \rho_c^{(k)}$ *(confound independence),* $x_6^{(k)} = \frac{1}{2}(\Delta_{pred}^{(k)} + 1)$ *(scaled predictivity gain). Then:*

$$LRS^{(k)} = \left(\frac{\sum_{i=1}^{6} w_i \cdot (x_i^{(k)} + \varepsilon)^p}{\sum_{i=1}^{6} w_i} \right)^{1/p} \tag{68}$$

Default parameters:

- $p = 0.5$*: Square-root mean (penalizes single bad component without being as harsh as geometric mean)*
- $\varepsilon = 0.05$*: Smoothing to prevent zeros from dominating*
- $w_i = 1$ *for all* i*: Equal weighting (sum to 6)*

The power mean provides a tunable balance: low p *penalizes imbalance (requiring convergent evidence), while high* p *is more forgiving of individual weak components. All terms in* $[0, 1]$*. High LRS indicates clean linear representation of behavior.*

Remark 7.5 (LRS Interpretation Thresholds)**.** *Interpretation guidelines:*

- $LRS > 0.7$*: Strong linear representation of behavior*
- $LRS \in [0.4, 0.7]$*: Moderate—some noise or non-linearity present*
- $LRS < 0.4$*: Weak linear representation—behavior may be gated or distributed*

7.7 Geometric Confidence

Definition 7.15 (Geometric Confidence C_G)**.** *Geometric confidence combines representational quality (LRS) with generalization ability. A component with high geometric confidence has a clean, consistent linear representation of the behavior that transfers to out-of-distribution examples.*

$$C_G^{(k)} = LRS^{(k)} \cdot gen^{(k)} \tag{69}$$

Remark 7.6 (Nonlinear Diagnostics)**.** *When* $LRS^{(k)} < \tau_{nonlinear} = 0.4$*, the behavioral representation may be nonlinear. DCAF triggers a suite of diagnostic methods—PaCMAP embedding with silhouette scoring, Procrustes alignment (structural mirror score), degree-2 polynomial probing, and kernel LDA—to characterize the nonlinearity without altering confidence scores or candidate status. Full definitions appear in Appendix G.*

7.8 Representation Direction Synthesis

The preceding subsections extract per-component contrastive directions, measure their cross-signal consistency (LRS), and characterize nonlinear representations. This subsection extends that machinery to the **global residual stream**, producing validated behavioral directions backed by convergent evidence from all training signals. Existing methods (Arditi et al. 2024, Zou et al. 2024, Li et al. 2023, Marks & Tegmark 2024) extract directions from a single contrast pair or training method with limited multi-signal validation. DCAF's multi-signal architecture eliminates this limitation.

The direction module synthesizes per-component contrastive directions (§7.1), bidirectional steering vectors (§11), and activation deltas across all behavioral signals into global directions, layer trajectories, composition decompositions, and actionable intervention specifications. The entire pipeline is fully automated and behavior-agnostic.

Remark 7.7 (Pipeline Placement)**.** *Direction extraction and DCS computation run at Step 6.5 in the pipeline (after per-component geometry analysis, before discovery). Composition analysis runs at Step 15.5 (after circuit graph reconstruction, requiring $\mathcal{H}_{conf}$). Both are described together here for conceptual coherence.*

7.8.1 Global Direction Extraction

Definition 7.16 (Residual Stream Hook Points)**.** *Register activation hooks at the residual stream after each layer's attention and MLP sub-layers:*

$$R_{post}^{(L)} = blocks[L].hook_resid_post \in \mathbb{R}^{n \times d_{model}} \tag{70}$$

for each layer $L \in \{0, \ldots, n_{layers} - 1\}$. These hooks are registered alongside the existing component-level hooks on the same forward passes*—zero additional forward pass cost.*

Definition 7.17 (Multi-Position Capture and Selection)**.** *Rather than hardcoding a token position, capture residual stream activations at candidate positions $\mathcal{P} = \{-1, -2, -3, -5\}$ (relative to sequence end). Select automatically:*

$$s^* = \arg\max_{s \in \mathcal{P}} \frac{1}{\binom{|\mathcal{T}^+|}{2}} \sum_{\substack{i,j \in \mathcal{T}^+ \\ i<j}} |\cos(d_i^{(L)}[s],\, d_j^{(L)}[s])| \tag{71}$$

at the peak DCS layer L^*, where $d_i^{(L)}[s]$ is the contrastive direction extracted at position s from signal i. The position with highest cross-signal agreement is selected.

Definition 7.18 (Per-Signal Direction Extraction)**.** *For each training signal $t_i \in \mathcal{T}^+ \cup \mathcal{T}^-$ and each layer L, extract a contrastive direction from residual stream activations using whitened LDA (same method as Definition 7.2, applied to residual stream instead of component outputs):*

$$d_i^{(L)} = \frac{(\Sigma_w + \lambda I)^{-1}(\mu_i^+ - \mu_i^-)}{\|(\Sigma_w + \lambda I)^{-1}(\mu_i^+ - \mu_i^-)\|} \tag{72}$$

where μ_i^+, μ_i^- are the mean residual stream activations for target and opposite prompts under signal t_i, and Σ_w is the pooled within-class covariance. This yields a direction matrix $D[signal][layer]$ containing up to $|\mathcal{T}^+ \cup \mathcal{T}^-| \times n_{layers}$ direction vectors.

7.8.2 Direction Convergence Score (DCS)

The Direction Convergence Score measures how strongly the training signals agree on a behavioral direction at each layer. It is structurally parallel to LRS (Definition 7.14) but applied to global residual stream directions.

Definition 7.19 (DCS Convergence Components)**.** *For layer L, compute three convergence metrics:*

$$coh_+^{(L)} = \frac{1}{\binom{|\mathcal{T}^+|}{2}} \sum_{\substack{i,j \in \mathcal{T}^+ \\ i<j}} |\cos(d_i^{(L)}, d_j^{(L)})| \qquad (\mathcal{T}^+ \text{ within-cluster coherence}) \tag{73}$$

$$coh_-^{(L)} = \frac{1}{\binom{|\mathcal{T}^-|}{2}} \sum_{\substack{i,j \in \mathcal{T}^- \\ i<j}} |\cos(d_i^{(L)}, d_j^{(L)})| \qquad (\mathcal{T}^- \text{ within-cluster coherence}) \tag{74}$$

$$opp^{(L)} = \frac{1}{|\mathcal{T}^+||\mathcal{T}^-|} \sum_{\substack{i \in \mathcal{T}^+ \\ j \in \mathcal{T}^-}} \left(-\cos(d_i^{(L)}, d_j^{(L)})\right) \qquad (\text{cross-cluster opposition}) \tag{75}$$

All three range $[0, 1]$ for well-behaved directions ($[-1, 1]$ in general). The convergence prerequisite is met when all three components $\geq \tau_{dcs\text{-}conv}$ (default: 0.3).

Definition 7.20 (Consensus Direction)**.** *When the convergence prerequisite is met, extract the consensus direction as the effectiveness-weighted mean of $\mathcal{T}^+$ directions:*

$$d_{consensus}^{(L)} = \frac{\sum_{i \in \mathcal{T}^+} eff(i) \cdot d_i^{(L)}}{\| \sum_{i \in \mathcal{T}^+} eff(i) \cdot d_i^{(L)} \|} \tag{76}$$

If the convergence prerequisite fails, no consensus direction is extracted.

Definition 7.21 (DCS Validation Components)**.** *Computed only when the convergence prerequisite is met (otherwise set to 0):*

$$ortho^{(L)} = 1 - |\cos(d_{consensus}^{(L)}, d_{baseline}^{(L)})| \qquad (\text{independence from baseline}) \tag{77}$$

$$confound^{(L)} = 1 - \frac{1}{|\mathcal{D}_{conf}|} \sum_{d_c \in \mathcal{D}_{conf}} |\cos(d_{consensus}^{(L)}, d_c)| \qquad (\text{independence from confounds}) \tag{78}$$

$$pred^{(L)} = AUC_{post}^{(L)} - AUC_{pre}^{(L)} \qquad (\text{held-out prediction gain}) \tag{79}$$

where $d_{baseline}^{(L)}$ is the contrastive direction from the neutral signal ($t_{DomainNative}$; defaults to $\mathbf{0}$ yielding ortho $= 1$ if absent), $\mathcal{D}_{conf}$ is an optional set of confound directions (defaults to $\emptyset$ yielding confound $= 1$), and $AUC_{pre/post}$ are the classification AUCs on held-out behavioral prompts using projection onto $d_{consensus}$, measured from the baseline model $\mathcal{M}_0$ and after training respectively.

Definition 7.22 (DCS Aggregation)**.** *Aggregate the six components using the same power mean as LRS:*

$$DCS(L) = \left[\frac{1}{6} \sum_{x \in \{coh_+, coh_-, opp, ortho, confound, pred\}} (x^{(L)} + \varepsilon)^p \right]^{1/p} \tag{80}$$

with $p = 0.5$, $\varepsilon = 0.05$, uniform weights. If the convergence prerequisite fails, validation components are 0 and DCS bottoms out through the power mean.

Definition 7.23 (Dimensionality Assessment)**.** *After consensus extraction, assess whether the behavior is single-direction or multi-dimensional:*

1. *Collect all signal directions at layer L: $\{d_i^{(L)}\}$ for $i \in \mathcal{T}^+ \cup \mathcal{T}^-$ (flip $\mathcal{T}^-$ signs for consistent orientation).*
2. *Run PCA on this set of direction vectors.*
3. *Classify:*

$$dim_type(L) = \begin{cases} single_direction & \text{if } PCA_1 > 0.8 \\ multi_direction & \text{if } PCA_1 \leq 0.8 \text{ and } DCS(L) \geq 0.4 \\ nonlinear & \text{if } DCS(L) < 0.4 \text{ with adequate signal effectiveness} \end{cases} \tag{81}$$

where PCA_1 is the fraction of variance explained by the first principal component. For multi_direction, report $n_{directions}$ = number of PCs needed for 90% cumulative variance; each PC receives its own DCS validation and composition analysis. For nonlinear, trigger nonlinear diagnostics (§G).

7.8.3 Layer Trajectory Analysis

DCS is computed for every layer $L \in \{0, \ldots, n_{\text{layers}} - 1\}$. The resulting curve reveals where the behavior lives in the network.

Definition 7.24 (Trajectory Metrics)**.**

$$L_{emerge} = \min\{L : DCS(L) > \tau_{dcs\text{-}peak}\} \qquad \textit{(emergence layer)} \tag{82}$$

$$L^* = \arg\max_L DCS(L) \qquad \textit{(peak layer)} \tag{83}$$

$$DCS^* = DCS(L^*) \qquad \textit{(peak DCS)} \tag{84}$$

$$stab(L) = \cos(d^{(L)}_{consensus}, d^{(L+1)}_{consensus}) \qquad \textit{(direction stability)} \tag{85}$$

where stability is computed for adjacent layers both having $DCS > \tau_{dcs\text{-}peak}$ (default: 0.5). The stable range is the maximal contiguous set of layers with $stab(L) > \tau_{stab}$ (default: 0.9).

Definition 7.25 (Trajectory Shape Classification)**.** *Smooth the DCS curve with a moving average (window = w_{traj}, default 3). Find local maxima and minima. Classify in priority order:*

1. ***multi_peak**: Two or more local maxima separated by a trough > 0.15.*
2. ***sharp_crystallization**: DCS jumps from < 0.3 to > 0.7 within 2–3 layers.*
3. ***early_peak_late_fade**: Peak in first 60% of layers, DCS decreases > 0.2 toward output.*
4. ***gradual_emergence**: Default—DCS increases monotonically across > 5 layers.*

Remark 7.8 (Multi-Peak Handling)**.** *When multi_peak is detected, compute cosine between peak consensus directions. If > 0.8, the same direction re-emerges (recomputed after MLP transformation); report `peak_alignment`. If < 0.5, genuinely different representations exist at different depths; produce independent composition analyses for each peak.*

7.8.4 Signal Diagnostics

Definition 7.26 (Effective Signal Count)**.** *At the peak layer L^*:*

$$n_{eff} = |\{i : |\cos(d_i^{(L^*)}, d^{(L^*)}_{consensus})| > \tau_{eff\text{-}cos} \text{ and } eff(i) > \tau_{eff\text{-}min}\}| \tag{86}$$

with $\tau_{eff\text{-}cos} = 0.5$ and $\tau_{eff\text{-}min} = 0.2$. If $n_{eff} < 3$, flag `convergence_quality: 'dominated'` with the dominant signal ID.

Remark 7.9 (Signal Diagnostic Patterns)**.** *Four characteristic patterns:*

- ***Universal agreement*** *(*$\cos > 0.8$ *for all signals): Rock-solid direction with highest confidence.*
- ***Anti/Negated deviation*** *(*t_4/t_5 *or* t_9/t_{10} *low* $\cos$*): Gradient ascent finds different pathways than direct training.*
- ***SFT deviation*** *(*t_2 *or* t_7 *low* $\cos$*): Supervised signals encode differently than preference signals.*
- ***Cluster asymmetry*** *(*$\mathcal{T}^+$ *high coherence,* $\mathcal{T}^-$ *low): One behavioral pole is more coherent than the other.*

7.8.5 Hierarchical Composition Analysis

Remark 7.10 (Pipeline Placement)**.** *Composition analysis runs at Step 15.5,* after *circuit graph reconstruction (§10), because it requires the confirmed component set* $\mathcal{H}_{conf}$*. It consumes the global direction from Step 6.5 and the component output deltas already captured as activation deltas.*

Definition 7.27 (Component Contribution to Global Direction)**.** *The residual stream at layer L is the sum of all component outputs up to that layer:*

$$R_{post}^{(L)} = embed + \sum_{\ell \leq L} \left(attn_out^{(\ell)} + mlp_out^{(\ell)} \right) \tag{87}$$

Each component's contribution to the global direction at the peak layer:

$$contrib(k) = \langle \Delta output^{(k)},\ d_{consensus}^{(L^*)} \rangle \tag{88}$$

where $\Delta output^{(k)}$ *is the mean difference in component k's output between target and opposite prompts. This is a signed scalar: positive means the component pushes toward the target behavior, negative means it opposes.*

Definition 7.28 (Normalized Contribution and Reconstruction Fidelity)**.** *Normalized contribution fraction:*

$$contrib_frac(k) = \frac{contrib(k)}{\sum_{k'} |contrib(k')|} \tag{89}$$

Positive contributions sum to > 0.5*, negative to* < 0.5*, total absolute values to 1.0. Reconstruction fidelity measures how well the confirmed circuit explains the global direction:*

$$fidelity = \cos\left(\frac{\sum_{k \in \mathcal{H}_{conf}} contrib(k) \cdot d^{(k)}}{\| \sum_{k \in \mathcal{H}_{conf}} contrib(k) \cdot d^{(k)} \|},\ \ d_{consensus}^{(L^*)} \right) \tag{90}$$

If fidelity $< \tau_{fidelity}$ *(default: 0.8), flag* `composition_gap: true`*—the confirmed circuit does not fully explain the global direction. Components outside* $\mathcal{H}_{conf}$ *may contribute, or nonlinear interactions may be present.*

Remark 7.11 (Suppression-Dominant Composition)**.** *If total negative contribution exceeds total positive contribution, flag* `composition_balance: 'suppression_dominant'`*. The model's default is to suppress the behavior; specific components override that suppression. For intervention, amplifying positive contributors may be more effective than ablating negative ones.*

Remark 7.12 (Computational Cost)**.** *Composition analysis requires zero additional forward passes. Component output deltas (*$\Delta output^{(k)}$*) are already captured as activation deltas in Step 5. The analysis is dot products and summation in post-processing.*

7.8.6 Intervention Specification

Definition 7.29 (Global Steering Specification)**.** *The primary intervention: inject the consensus direction into the residual stream at the peak layer.*

$$\hat{R}^{(L^*)} = R^{(L^*)} + \alpha \cdot d^{(L^*)}_{consensus} \quad \textit{at token position } s^* \tag{91}$$

The recommended α range is estimated by applying $\alpha \in \{\alpha_1, \ldots, \alpha_m\}$ (default: $\{0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 5.0\}$) and selecting $[\alpha_{\min}, \alpha_{\max}]$ where the behavioral metric exceeds 0.8 of the maximum achievable effect and capability degradation (perplexity increase) remains below τ_{ppl} (default: 10%).

Definition 7.30 (Component-Level Alignment to Global Direction)**.** *For each confirmed component k, measure convergent validation between the component's optimized steering vector and its compositional role:*

$$\textit{align_global}(k) = \cos(v^*_{k,+},\ d^{(L^*)}_{consensus}) \tag{92}$$

High alignment validates that component-level steering and global direction extraction converge on the same representation.

Definition 7.31 (Global vs. Component Comparison)**.** *Measure effectiveness of global steering (single direction at peak layer) versus top-n component steering (steering vectors at the n highest-contribution components). Report:*

$$\textit{recommendation} = \begin{cases} \textit{global} & \textit{if global effectiveness} > \textit{component effectiveness} \\ \textit{component} & \textit{if component effectiveness} > \textit{global effectiveness} \\ \textit{hybrid} & \textit{if within 5\% of each other} \end{cases} \tag{93}$$

Global steering is simpler to deploy; component steering may have fewer side effects.

7.8.7 Edge Cases and Failure Modes

Definition 7.32 (Direction Status Classification)**.**

$$\textit{direction_status} = \begin{cases} \textit{validated} & \textit{if } DCS^* \geq 0.4 \textit{ and } n_{\textit{eff}} \geq 3 \\ \textit{insufficient_training} & \textit{if mean eff}(i) \textit{ is low} \\ \textit{nonlinear_or_absent} & \textit{if adequate eff but } DCS^* < 0.4 \textit{ at all layers} \end{cases} \tag{94}$$

Remark 7.13 (Failure Mode Catalogue)**.** *All failure modes are reported as output fields, never silent failures:*

- `opposition_failure`*: High coh_+ and coh_- but low opp. Both training directions activated a shared "safety-relevant content" axis; the behavioral split happens downstream or orthogonally. Check whether per-component directions show opposition even when global directions do not.*
- `convergence_quality: 'dominated'`*: $n_{\textit{eff}} < 3$. DCS is technically valid but convergent evidence is weaker than the headline number suggests.*
- `functional_validation: 'failed'`*: High convergence components, low pred. Direction is statistically clean but functionally meaningless. Recommend expanding behavioral prompt set.*
- `composition_gap`*: fidelity $< \tau_{\textit{fidelity}}$. Confirmed circuit does not fully explain the global direction. Favor global steering over component-level steering.*
- `composition_balance: 'suppression_dominant'`*: Total negative contribution exceeds total positive. Default model behavior is to suppress; specific components override.*

8 Training Diagnostics

Training diagnostics measure the quality of the training process itself, as opposed to the projection-level analysis in the preceding sections. Where weight, activation, and geometry analyses answer "which projections implement the behavior?" and "how confident are we?", training diagnostics answer "did the training signals behave as expected?" Poor training quality (e.g., inconsistent optimization trajectories, contradictory activation patterns) can undermine the reliability of downstream analysis even when individual metrics appear strong.

8.1 Activation Delta Alignment

Activation delta alignment measures whether the training-induced activation changes are internally consistent. If multiple T^+ signals all aim to reinforce the same behavior, their activation deltas at behavioral components should point in similar directions. Likewise, T^- deltas should be internally consistent and should oppose the T^+ direction.

Definition 8.1 (Activation Delta Alignment)**.** *For component k, compute three alignment metrics from the activation deltas $\Delta A_i^{(k)} = A_{i,peak}^{(k)} - A_{i,pre}^{(k)}$:*

Within-cluster alignment (target):

$$align_+^{(k)} = \frac{1}{|\mathcal{T}^+|^2} \sum_{i,j \in \mathcal{T}^+} \cos(\Delta A_i^{(k)}, \Delta A_j^{(k)}) \tag{95}$$

Within-cluster alignment (opposite):

$$align_-^{(k)} = \frac{1}{|\mathcal{T}^-|^2} \sum_{i,j \in \mathcal{T}^-} \cos(\Delta A_i^{(k)}, \Delta A_j^{(k)}) \tag{96}$$

Cross-cluster opposition:

$$oppose_\Delta^{(k)} = \frac{1}{|\mathcal{T}^+||\mathcal{T}^-|} \sum_{i \in \mathcal{T}^+, j \in \mathcal{T}^-} \cos(\Delta A_i^{(k)}, \Delta A_j^{(k)}) \tag{97}$$

Expected values for a well-behaved training process at behaviorally-relevant components:

- $align_+ > 0.7$*: Target signals move activations in the same direction.*
- $align_- > 0.7$*: Opposite signals move activations in the same direction.*
- $oppose_\Delta < -0.5$*: Target and opposite signals move activations in opposing directions.*

Remark 8.1 (Activation Delta Alignment is Always Computed)**.** *Unlike the optional curvature metrics below, activation delta alignment is always computed for every confirmed component. It provides a first-order sanity check on training consistency and is computationally inexpensive (cosine similarities between already-computed deltas). When $align_+$ or $align_-$ is low at a component that otherwise scores well, this flags possible inconsistency in the training signal suite worth investigating.*

8.2 Online Curvature Tracking

Remark 8.2 (Optional Diagnostic)**.** *Online curvature tracking requires per-step monitoring during training. It is appropriate when training dynamics are of interest or when unexpected results warrant investigation of optimization trajectory quality.*

Definition 8.2 (Online Curvature)**.** *For training signal t_i at component k, track the activation trajectory $\{A_{i,\tau}^{(k)}\}_{\tau=0}^{T}$—activation snapshots at component k on a fixed probe set, captured at training evaluation checkpoints. The curvature measures how winding the representational trajectory is relative to straight-line displacement:*

$$\kappa_i^{(k)} = \frac{path_length_i^{(k)}}{direct_distance_i^{(k)}} - 1 \tag{98}$$

where:

$$path_length_i^{(k)} = \sum_{\tau=1}^{T} \|A_{i,\tau}^{(k)} - A_{i,\tau-1}^{(k)}\|_2 \tag{99}$$

$$direct_distance_i^{(k)} = \|A_{i,peak}^{(k)} - A_{i,0}^{(k)}\|_2 \tag{100}$$

Initialized with $A_{i,0}^{(k)}$ from baseline model ($\mathcal{M}_0$) activations on the fixed probe set. At each evaluation step, the segment $\|A_{i,\tau}^{(k)} - A_{i,\tau-1}^{(k)}\|_2$ is added to the cumulative path length.

$\kappa = 0$: training moved representations directly toward target. High κ: competing gradients or unstable optimization. Asymmetric κ ($\mathcal{T}^+$ low, $\mathcal{T}^-$ high): one behavioral direction is "easier" to learn at this component.

Definition 8.3 (Cluster-Aggregated Curvature)**.** *Aggregate curvature by training cluster to detect asymmetric learning difficulty:*

$$\bar{\kappa}_+^{(k)} = \frac{1}{|\mathcal{T}^+|} \sum_{i \in \mathcal{T}^+} \kappa_i^{(k)} \qquad \textit{(mean target curvature)} \tag{101}$$

$$\bar{\kappa}_-^{(k)} = \frac{1}{|\mathcal{T}^-|} \sum_{i \in \mathcal{T}^-} \kappa_i^{(k)} \qquad \textit{(mean opposite curvature)} \tag{102}$$

If $\bar{\kappa}_+ \gg \bar{\kappa}_-$ (or vice versa), one behavioral direction is substantially harder to learn than the other at this component, suggesting an asymmetric loss landscape. This can indicate that the component was pre-disposed toward one behavior (low curvature) and resistant to the other (high curvature).

Remark 8.3 (Memory Efficiency of Curvature Tracking)**.** *Activation curvature requires $O(1)$ additional storage per component per signal beyond the probe set: a running path-length accumulator and the initial activation snapshot $A_{i,0}^{(k)}$ on the fixed probe set. At each evaluation step, the segment $\|A_{i,\tau}^{(k)} - A_{i,\tau-1}^{(k)}\|_2$ is added to the accumulator, requiring only the current and previous activation snapshots without storing the full representational trajectory. The probe set should be the same small set used elsewhere in the framework.*

9 Unified Confidence and Candidate Selection

This section integrates evidence from all three discovery paths and all three measurement domains into unified confidence scores. All confidence scores are **component-level**.

9.1 Base Triangulated Confidence

Definition 9.1 (Base Triangulated Confidence)**.** *For component $k \in \mathcal{H}_{disc}$, define the base triangulated confidence as a weighted geometric mean:*

$$C_{base}^{(k)} = \left[(C_W^{(k)} + \varepsilon)^w \cdot (C_A^{(k)} + \varepsilon) \cdot (C_G^{(k)} + \varepsilon) \right]^{\frac{1}{w+2}} \tag{103}$$

where $w \geq 1$ is the domain weight (default: $w = 2$) and $\varepsilon > 0$ is the smoothing parameter (default: $\varepsilon = 0.05$). $C_W^{(k)}$ is component-level (aggregated from projections via max), while $C_A^{(k)}$ and $C_G^{(k)}$ are already component-level.

The three measurement domains serve complementary roles:

- *C_W: Weight evidence—how much did projection weights change during behavioral training?*
- *C_A: Activation evidence—how much did component representations change?*
- *C_G: Geometric evidence—is there a clean behavioral direction at this component?*

The weighted geometric mean with epsilon smoothing ensures: (1) all domains contribute but zeros don't completely dominate ($\varepsilon > 0$), (2) domain agreement amplifies confidence, and (3) the formula remains bounded in $[0, 1]$.

9.2 Multi-Path Discovery Bonus

Definition 9.2 (Multi-Path Discovery Bonus)**.** *Components discovered by multiple independent paths receive a confidence bonus reflecting converging evidence:*

$$bonus(k) = \beta_{path} \cdot \max(0, paths(k) - 1) \tag{104}$$

where $paths(k) = |disc_paths(k)|$ and $\beta_{path} = 0.15$ (default).

This is additive rather than multiplicative to avoid penalizing single-path discoveries:

- *Single-path discovery: bonus $= 0$ (no penalty, no bonus)*
- *Two-path discovery: bonus $= 0.15$ (converging evidence)*
- *Three-path discovery: bonus $= 0.30$ (strong convergence)*

Definition 9.3 (Final Unified Confidence)**.** *The complete confidence score combines base triangulated confidence with multi-path bonus:*

$$C^{(k)} = \min(1, C_{base}^{(k)} + bonus(k)) \tag{105}$$

Remark 9.1 (Why Additive Bonus)**.** *An additive bonus (rather than multiplicative) ensures:*

- *Components discovered by only one path still get full domain confidence*
- *Multi-path discovery provides additional evidence, not a requirement*
- *The bonus is bounded and doesn't dominate domain confidence*

A component found only by Path 2 (activation) with strong C_A and C_G but weak C_W may still be high-confidence—it's a leverage point, not a false positive.

9.3 Candidate Set Construction

Definition 9.4 (Candidate Set)**.** *Components with sufficient unified confidence:*

$$\mathcal{H}_{cand} = \{k \in \mathcal{H}_{disc} : C^{(k)} \geq \tau_{unified}\} \tag{106}$$

where $\tau_{unified}$ is the candidate threshold (default: 0.3).

Definition 9.5 (Confirmed Set)**.** *After ablation testing:*

$$\mathcal{H}_{conf} = \{k \in \mathcal{H}_{cand} : \textit{ablation affects behavior without breaking model}\} \tag{107}$$

9.4 Domain Disagreement Analysis

Remark 9.2 (Disagreement Diagnostics)**.** *When domains disagree, the pattern reveals the component's nature:*

- ***Weight high, Activation low:*** *Weights changed without representational reorganization (possibly dead or redundant projections)*
- ***Weight low, Activation high:*** *Leverage point—small weight changes produce large representational effects*
- ***Geometry high, Others low:*** *Pre-existing direction that training leveraged without creating new structure*
- ***Gradient high, Weight low:*** *Gradient landscape predicted relevance but optimizer didn't modify (learning rate, regularization effects)*

Remark 9.3 (Path-Domain Interaction Patterns)**.** *The discovery path combined with domain confidence reveals component characteristics:*

Discovered By	C_W	C_A	***Interpretation***
Path 1 only	*High*	*High*	*Standard behavioral component*
Path 1 only	*High*	*Low*	*Weight change without activation effect*
Path 2 only	*Low*	*High*	*Leverage point*
Path 3 only	*Low*	*Low*	*Gradient prediction, needs ablation validation*
Paths 1+2	*High*	*High*	*Strong converging evidence*
Paths 2+3	*Low*	*High*	*Leverage point confirmed by gradients*
All paths	*High*	*High*	*Highest confidence*

10 Circuit Graph Reconstruction

Definition 10.1 (Circuit Nodes)**.** *Confirmed behaviorally-relevant components:*

$$V = \mathcal{H}_{conf} \tag{108}$$

10.1 Edge Discovery Methods

Definition 10.2 (Method 1: Activation Flow Analysis)**.** *Track activation correlation across layers:*

$$corr^{(i)}(k, k') = Corr(A_i^{(k)}, A_i^{(k')}) \tag{109}$$

Differential: Does correlation increase after training?

Definition 10.3 (Method 2: Targeted Ablation)**.** *Ablate component k, measure activation changes at other components:*

$$E^{abl}_{k \to k'} = \|A^{(k')}_{with\ k} - A^{(k')}_{without\ k}\|_F \tag{110}$$

Definition 10.4 (Method 3: Targeted Steering)**.** *Apply steering vector at component k, measure activation changes:*

$$E^{steer}_{k \to k'} = \|A^{(k')}_{with\ v_k} - A^{(k')}_{without\ v_k}\|_F \tag{111}$$

Definition 10.5 (Method 4: Gradient Flow Analysis)**.** *Track gradient correlation during training:*

$$grad_corr(k, k') = Corr(\nabla_{W_k}\mathcal{L}, \nabla_{W_{k'}}\mathcal{L}) \tag{112}$$

Shared gradient pressure suggests functional connection.

Definition 10.6 (Method 5: Cross-Layer Attention Patterns)**.** *For attention heads, track information flow:*

$$attn_flow(k_\ell, k_{\ell'}) = \sum_s P_{\ell',h}[s, pos(k_\ell)] \tag{113}$$

Measures whether downstream heads attend to positions where upstream components wrote.

Definition 10.7 (Causal Edge)**.** *A causal edge exists between components when intervening on one (via ablation or steering) causally affects the other. The edge weight reflects the strength of this causal relationship, and the edge predicate applies a threshold to identify significant connections.*

$$E_{k\to k'} = \max\left(E^{abl}_{k\to k'}, E^{steer}_{k\to k'}\right) \tag{114}$$

$$e(k, k') = \mathbf{1}\left[E_{k\to k'} \geq \tau_E \wedge k' \in V\right] \tag{115}$$

Definition 10.8 (Edge Pathway Attribution)**.** *For edges where the target component k' is an attention head, determine which input pathway carries the causal signal:*

1. *Ablate source component k*
2. *Hook k''s Q, K, and V input projections separately*
3. *Measure $\|A^{(proj)}_{with\ k} - A^{(proj)}_{without\ k}\|_F$ at each hook*
4. *Normalize to pathway weights summing to 1.0:*

$$w_Q = \frac{\|\Delta A_Q\|_F}{\|\Delta A_Q\|_F + \|\Delta A_K\|_F + \|\Delta A_V\|_F} \tag{116}$$

$$w_K = \frac{\|\Delta A_K\|_F}{\|\Delta A_Q\|_F + \|\Delta A_K\|_F + \|\Delta A_V\|_F} \tag{117}$$

$$w_V = \frac{\|\Delta A_V\|_F}{\|\Delta A_Q\|_F + \|\Delta A_K\|_F + \|\Delta A_V\|_F} \tag{118}$$

For non-attention targets, the pathway is {residual: 1.0}. Edges in the circuit graph carry pathway metadata: via $= (w_Q, w_K, w_V)$.

Definition 10.9 (Behavioral Circuit Graph)**.** *The behavioral circuit graph is the final output of DCAF's circuit reconstruction: a directed graph where nodes are confirmed behaviorally-relevant components and edges represent causal relationships discovered through intervention.*

$$G = \big(V, \{(k, k') : e(k, k') = 1\}\big) \tag{119}$$

Remark 10.1 (Circuit Graph Output Format)**.** *The circuit graph is serialized as a JSON structure containing node metadata (discovery attribution, domain-level confidence scores, adaptive tiered functional classification, bidirectionality status) and edge metadata (causal weight, validation status, pathway attribution). An illustrative example is provided in Appendix K.*

11 Steering Vector Optimization

Steering vectors validate the causal role of discovered components by directly intervening in the model's computations. DCAF optimizes vectors in both behavioral directions—toward the target and toward the opposite behavior—providing a complete picture of each component's steering capacity and revealing asymmetries in how the component mediates the behavior.

11.1 Bidirectional Steering Vectors

Definition 11.1 (Target Steering Vector)**.** *For component k, optimize $v^*_{k,+}$ to maximize the target behavioral objective:*

$$v^*_{k,+} = \arg\max_v \mathcal{L}_{target}(\mathcal{M}_0 + v \cdot e_k) \tag{120}$$

where e_k denotes injection at component k and $\mathcal{L}_{target}$ is a differentiable objective measuring alignment with the target behavior (SFT loss, preference margin, reward, etc.).

Definition 11.2 (Opposite Steering Vector)**.** *For the same component k, optimize $v^*_{k,-}$ to maximize the opposite behavioral objective:*

$$v^*_{k,-} = \arg\max_v \mathcal{L}_{opposite}(\mathcal{M}_0 + v \cdot e_k) \tag{121}$$

where $\mathcal{L}_{opposite}$ measures alignment with the opposite behavior.

11.2 Steering Alignment Metrics

Definition 11.3 (Steering-Geometry Alignment)**.** *Three alignment metrics characterize the relationship between steering vectors and the geometric contrastive direction $d^{(k)}$:*

$$\alpha_+^{(k)} = \cos(v^*_{k,+},\ d^{(k)}) \qquad \textit{(target steering vs. geometry)} \tag{122}$$

$$\alpha_-^{(k)} = \cos(v^*_{k,-},\ d^{(k)}) \qquad \textit{(opposite steering vs. geometry)} \tag{123}$$

$$\alpha_\pm^{(k)} = \cos(v^*_{k,+},\ v^*_{k,-}) \qquad \textit{(target vs. opposite steering)} \tag{124}$$

Remark 11.1 (Alignment Interpretation)**.**

- *$\alpha_+ \approx 1$ and $\alpha_- \approx -1$: Steering and geometry agree perfectly. The component mediates the behavior along a single axis; pushing in one direction reinforces target, pushing in the other reinforces opposite.*
- *$\alpha_\pm \approx -1$: Target and opposite steering are anti-aligned. The component has a single bidirectional control axis. This is the simplest and most common case.*
- *$\alpha_\pm \neq -1$: Target and opposite steering are* not *anti-aligned. This indicates the component mediates each behavioral direction through partially different mechanisms. The closer $\alpha_\pm$ is to 0, the more orthogonal the mechanisms.*

Remark 11.2 (Negated Steering Insight)**.** *When $\alpha_\pm \neq -1$ (i.e., $v^*_{k,+}$ and $v^*_{k,-}$ are not simply negations of each other), two distinct intervention strategies emerge:*

- ***Reinforcement (defensive):*** *Apply $v^*_{k,+}$ to actively steer toward target behavior. This "pushes" the representation in the direction the model uses to implement the target.*
- ***Pathway blocking (counter-vulnerability):*** *Apply $-v^*_{k,-}$ to block the pathway the model would use to implement the opposite behavior. This "closes off" the attack vector without necessarily activating the defensive pathway.*

The distinction matters for safety applications: reinforcement strengthens the target behavior, while pathway blocking neutralizes vulnerability to the opposite behavior. When the two strategies differ ($\alpha_\pm \neq -1$), using both provides defense in depth.

11.3 Steering Effectiveness

Definition 11.4 (Bidirectional Steering Effectiveness)**.** *Quantify how effectively each steering vector shifts the model's behavior:*

$$\mathit{eff}_+^{(k)} = \mathcal{L}_{target}(\mathcal{M}_0 + v_{k,+}^* \cdot e_k) - \mathcal{L}_{target}(\mathcal{M}_0) \qquad \textit{(target steering gain)} \tag{125}$$

$$\mathit{eff}_-^{(k)} = \mathcal{L}_{opposite}(\mathcal{M}_0 + v_{k,-}^* \cdot e_k) - \mathcal{L}_{opposite}(\mathcal{M}_0) \qquad \textit{(opposite steering gain)} \tag{126}$$

High effectiveness in both directions confirms the component as a bidirectional control point. Asymmetric effectiveness (e.g., $\mathit{eff}_+ \gg \mathit{eff}_-$) indicates the component is more "steerable" in one direction—it may already be biased toward one behavior.

11.4 Steering Validation

Definition 11.5 (Steering Validation Protocol)**.** *Validation proceeds through five steps applied to both $v_{k,+}^*$ and $v_{k,-}^*$:*

1. ***Behavioral test:*** *Does the steering vector produce the intended behavioral shift?*
2. ***Scaling ablation:*** *Apply v at magnitudes $\{1.0, 0.7, 0.5, 0.3, 0.1, 0.0\}$. Monotonic dose-response confirms causal control; non-monotonic suggests interference.*
3. ***Component ablation:*** *Remove individual vectors from multi-component steering. Which are necessary vs. redundant?*
4. ***Activation analysis:*** *Which other components are affected by steering at k? Map the downstream propagation.*
5. ***Cross-reference:*** *Do affected components match DCAF-identified circuit nodes? High overlap validates the circuit graph.*

12 Ablation Methodology and Functional Classification

This section describes the multi-strategy ablation methodology for efficiently discovering component interactions. The methodology integrates into the existing pipeline between $\mathcal{H}_{\text{cand}}$ and $\mathcal{H}_{\text{conf}}$, specifying *how* to efficiently select which components, pairs, and triples to ablate.

12.1 Ablation Techniques

Definition 12.1 (Ablation Methods)**.** *Two ablation approaches:*

- ***Mean ablation:*** *Replace component activations with mean activation across a reference set*
- ***Baseline restoration:*** *Replace component weights/activations with baseline model ($\mathcal{M}_0$) values*

Definition 12.2 (Behavioral Relevance Confirmation)**.** *For candidate component k, ablation determines behavioral relevance:*

- *If ablation breaks model functionality entirely $\Rightarrow$ general component, not behavior-specific*
- *If model functions but behavior breaks or changes $\Rightarrow$ behaviorally relevant, proceed to classification*

Definition 12.3 (Probe-Specific Impact)**.** *For confirmed component k, ablation impact per probe type:*

$$I_\pi^{(k)} = \frac{|\mathit{score}_{intact}^\pi - \mathit{score}_{ablated}^\pi|}{|\mathit{score}_{intact}^\pi| + \varepsilon} \tag{127}$$

where ε prevents division by zero and $score^{\pi}_{intact}$, $score^{\pi}_{ablated}$ are behavioral scores on probe π with the model intact and with component k ablated, respectively.

Definition 12.4 (Behavioral Relevance Classification)**.** *Let $func_{intact}$ and $func_{ablated}$ denote model functionality scores (e.g., perplexity on neutral benchmarks) with the model intact and with component k ablated. Classify the ablation outcome:*

$$relevance(k) = \begin{cases} GENERAL & \text{if } func_{ablated} < 0.5 \cdot func_{intact} \\ BEHAVIORAL & \text{if } I_{\pi}^{(k)} > 0.1 \text{ for any } \pi \\ NONE & \text{otherwise} \end{cases} \tag{128}$$

A GENERAL outcome indicates the ablation breaks model functionality entirely, so the component is not behavior-specific. A BEHAVIORAL outcome indicates the model functions but behavior changes. Components classified NONE show no significant impact.

12.2 Multi-Strategy Interaction Discovery

Starting from $\mathcal{H}_{\text{cand}}$ (candidates that passed C_W, C_A, and C_G thresholds), discover component interactions through a seven-phase protocol.

12.2.1 Phase 1: Individual Component Ablation

Every candidate component is tested individually ($\sim$30–50 tests for a typical $|\mathcal{H}_{\text{cand}}|$). This phase is exhaustive over $\mathcal{H}_{\text{cand}}$ and produces the partition that all later phases depend on.

Definition 12.5 (Individual Ablation)**.** *For each $k \in \mathcal{H}_{cand}$: ablate component k, apply behavioral relevance classification (Definition 12.4), measure $I_{\pi}^{(k)}$ for all probe types if behaviorally relevant.*

Partition $\mathcal{H}_{cand}$ into:

- *$\mathcal{H}_{solo}$: components with behavioral impact when ablated alone*
- *$\mathcal{H}_{\overline{solo}}$: components with no individual impact*

12.2.2 Phase 2: Seven Parallel Pair/Group Strategies

Each strategy independently identifies pairs or groups likely to interact. Results are pooled. The total number of pairs tested is capped at B_{pair} (default: 300); when the union of candidate pairs from Strategies A–G exceeds this budget, pairs found by multiple strategies are prioritized, then by unified confidence of constituent components.

Definition 12.6 (Strategy A: Graph-Adjacent Pairs)**.** *Components connected by preliminary circuit edges (from activation flow or gradient correlation in §8) with edge weight $> \tau_E$ are paired. Tests connected circuit segments.*

Definition 12.7 (Strategy B: Gradient-Based Interaction Screening)**.** *Ablate component k_1 and measure how much k_2's gradient changes:*

$$interact(k_1, k_2) = \left\| \nabla_{k_2} \mathcal{L}\big|_{k_1 \ ablated} - \nabla_{k_2} \mathcal{L}\big|_{intact} \right\| \tag{129}$$

This is a screening method that predicts which pairs to test via full behavioral ablation, not a direct behavioral test. Return top-N interacting pairs by this score.

Definition 12.8 (Strategy C: Activation Correlation Pairs)**.** *Build a vector per component by concatenating activation deltas $\Delta A_i^{(k,\pi)}$ across all behavioral signals and probe types. Compute pairwise cosine similarity. Select the top correlated pairs (budget: $B_{pair}/5$). Tests co-functioning components.*

Definition 12.9 (Strategy D: Hierarchical Clustering)**.** *Using the same correlation matrix as Strategy C, apply hierarchical clustering (average linkage, distance $= 1 -$ correlation, threshold $= 0.7$). Extract clusters of size ≥ 3. For each cluster, test collective ablation; if significant, find the minimal sufficient subset via greedy removal. Tests functional groups.*

Definition 12.10 (Strategy E: Opposition Grouping)**.** *Pair components where:*

$$similar(k_1, k_2) = \mathbf{1}[|opp_degree(k_1) - opp_degree(k_2)| < \varepsilon_{opp}] \tag{130}$$

where $\varepsilon_{opp} = 0.1$ (default) and $opp_degree(k) = \max_{proj \in \mu^{-1}_{proj}(k)} opp_degree(proj)$ (already defined at projection level). Components with similar opposition profiles respond similarly to $\mathcal{T}^+$ vs. $\mathcal{T}^-$—likely parallel circuits implementing the same behavioral control.

Definition 12.11 (Strategy F: Cross-Layer Composition)**.** *Pair attention heads from different layers. Cross-layer composition is a known circuit motif (induction heads, IOI circuits). MLP components are excluded from this strategy.*

Definition 12.12 (Strategy G: Random Sampling)**.** *Confidence-weighted random pairs ($n_{samples} = 200$), sampling probability proportional to $C^{(k)}$. Catches unexpected interactions missed by structured strategies.*

Remark 12.1 (Role of Random Sampling)**.** *Strategy G serves as a false-negative safeguard. Structured strategies A–F each encode specific hypotheses about interaction patterns (graph adjacency, gradient correlation, activation similarity, hierarchical clustering, opposition similarity, cross-layer composition). Interactions that violate all six hypotheses—for example, components in distant layers with dissimilar activation profiles that nonetheless gate each other—would be missed without random exploration. The confidence-weighted sampling ensures computational budget is concentrated on high-confidence candidates while maintaining coverage of the unexpected.*

Remark 12.2 (Phase 2 Accounting)**.** *For each tested pair, classify the interaction type (Definition 12.21). Track which strategies found each pair. For Strategy D clusters, the minimal sufficient subset identified by greedy removal is recorded.*

12.2.3 Phase 3: Refinement

Targeted follow-up based on Phase 1 and Phase 2 results:

Definition 12.13 (Refinement Rules)**.**

1. ***Solo* × *Solo pairs:*** *Test pairwise among $\mathcal{H}_{solo}$ components from Phase 1. Synergistic solo pairs constitute stronger circuit evidence.*
2. ***No-solo pair members:*** *For $\mathcal{H}_{\overline{solo}}$ components found in significant Phase 2 pairs, test with other $\mathcal{H}_{solo}$ components.*
3. ***Hessian pairs with low ablation:*** *If Strategy B predicted interaction but pair ablation showed no behavioral impact, flag for triple testing in Phase 5 (a third component may be gating).*
4. ***Minimal sufficient subsets*** *from significant Strategy D clusters are recorded for the circuit graph.*

12.2.4 Phase 4: Cross-Validation Counting

Definition 12.14 (Multi-Strategy Confirmation)**.**

$$\textit{discovery_count}(k) = |\{s \in \textit{Strategies} : k \textit{ found in a significant pair/group by } s\}| \tag{131}$$

Components in $\mathcal{H}_{solo}$ from Phase 1 count as one strategy (`phase1_individual`). Pair-level cross-validation is also tracked: how many strategies found each significant pair.

- *discovery_count ≥ 2: High-confidence circuit*
- *discovery_count $= 1$: Re-test in different ablation contexts; validate or mark uncertain*

12.2.5 Phase 5: Triple Detection

Two approaches to finding three-component gating interactions, capped at B_{triple} (default: 100) total tests, prioritizing triples containing high discovery-count components.

Definition 12.15 (Approach 1: From Hessian Pairs)**.** *For each Strategy B pair (Definition 12.7) with low behavioral ablation impact, add as third members: components with discovery_count ≥ 2 from Phase 4, and high-confidence components (top of $\mathcal{H}_{cand}$ by $C^{(k)}$).*

Definition 12.16 (Approach 2: From No-Effect Components)**.** *Identify components with no individual effect (Phase 1) and no pair effect (Phase 2). Combine each with the top 5 $\mathcal{H}_{solo}$ components (by $C^{(k)}$) as triples. Also form triples among no-effect components with similar opposition profiles ($|\textit{opp_degree}(k_1) - \textit{opp_degree}(k_2)| < 0.15$).*

12.2.6 Phase 6: Orphan Analysis

Definition 12.17 (Orphan Components)**.** *After completing Phases 1–5, identify orphan components:*

$$\mathcal{O} = \{k \in \mathcal{H}_{cand} : k \textit{ not confirmed by any Phase 1–5 configuration}\} \tag{132}$$

These components passed all three domain confidence filters (C_W, C_A, C_G) but showed no individual effect, appeared in no significant pair, and appeared in no significant triple.

Definition 12.18 (High-Confidence Orphans)**.**

$$\mathcal{O}_{high} = \{k \in \mathcal{O} : C^{(k)} > \tau_{orphan}\} \tag{133}$$

where τ_{orphan} is the high-confidence orphan threshold (default: 0.6).

For each $k \in \mathcal{O}_{high}$: re-test paired with every $\mathcal{H}_{solo}$ component and with other $\mathcal{O}_{high}$ orphans. Orphans confirmed during retesting are promoted.

Definition 12.19 (Low-Confidence Orphans)**.**

$$\mathcal{O}_{low} = \{k \in \mathcal{O} : C^{(k)} \leq \tau_{orphan}\} \tag{134}$$

Flag for optional exhaustive pairwise testing when computational budget allows.

12.2.7 Phase 7: Final Classification

Definition 12.20 (Interaction Requirement)**.** *For each component $k \in \mathcal{H}_{cand}$, assign an interaction requirement in priority order:*

1. ***ORPHAN*** *if in final orphans (not confirmed even after Phase 6 retesting)*
2. ***SOLO*** *if individual ablation (Phase 1) showed behavioral impact*
3. ***PAIR*** *if found in a significant pair (select best pair by impact; record partner and interaction type)*
4. ***GATE*** *if found in a significant triple*

Promoted orphans (confirmed during Phase 6 retesting) receive PAIR classification with their confirming partner.

The confirmed set:

$$\mathcal{H}_{conf} = \{k \in \mathcal{H}_{cand} : \mathit{classification}(k) \neq \mathit{ORPHAN}\} \tag{135}$$

12.3 Interaction-Enriched Output

Definition 12.21 (Interaction Type)**.** *For PAIR and GATE components, classify how they interact:*

$$\mathit{SYNERGISTIC:} \quad I(k_1, k_2) > I(k_1) + I(k_2) + \varepsilon_{syn} \tag{136}$$

$$\mathit{ADDITIVE:} \quad |I(k_1, k_2) - (I(k_1) + I(k_2))| \leq \varepsilon_{syn} \tag{137}$$

$$\mathit{REDUNDANT:} \quad I(k_1, k_2) \approx \max(I(k_1), I(k_2)) \tag{138}$$

where ε_{syn} is the synergy detection threshold (default: 0.05).

SYNERGISTIC: combined impact exceeds sum of individual impacts—the components gate, modulate, or amplify each other nonlinearly. ADDITIVE: combined impact equals sum of individual impacts—the components contribute independently through parallel pathways. REDUNDANT: combined impact equals max of individual impacts—the components implement the same function, so removing either has no additional effect beyond removing the other.

This separates whether *components need interactions (Definition 12.20) from* how *they interact when they do.*

Definition 12.22 (Interaction-Enriched $\mathcal{H}_{\text{conf}}$)**.** *$\mathcal{H}_{conf}$ includes:*

- *Individual components (from Phase 1)*
- *Component pairs/triples with interaction requirement and type*
- *Discovery methods: which strategies found each element*

Each confirmed element has: behavioral relevance status, probe-specific impacts $I_\pi^{(k)}$ for all probe types, functional classification via class(k), and interaction metadata (SOLO/PAIR/GATE + interaction type if applicable).

12.4 Computational Complexity

Remark 12.3 (Ablation Budget)**.** *The methodology distributes ablations across seven phases:*

Phase 1: *Individual component ablations (one per $\mathcal{H}_{cand}$ component, typically 30–50).*

Phase 2: *Seven parallel strategies with total pair budget B_{pair} (default: 300). Strategies A–G propose candidate pairs independently; when the union exceeds B_{pair}, pairs found by multiple strategies are prioritized.*

Phase 3: *Refinement—targeted follow-up from Phases 1–2 (solo×solo, no-solo pair members, Hessian flagging).*

Phase 4: *Cross-validation counting (no additional ablations, bookkeeping only).*

Phase 5: *Triple detection with budget B_{triple} (default: 100), from Hessian pairs and no-effect components.*

Phase 6: *Orphan retesting—high-confidence orphans paired with solo components and other orphans. Optional exhaustive pairwise for low-confidence orphans.*

Phase 7: *Final classification (no additional ablations, bookkeeping only).*

The total number of ablations scales with $|\mathcal{H}_{cand}|$ but remains substantially lower than exhaustive pairwise testing, which requires $|\mathcal{H}_{cand}| \times (|\mathcal{H}_{cand}| - 1)/2$ ablations.

Remark 12.4 (Coverage Analysis)**.** *The multi-strategy approach provides comprehensive coverage:*

- *All individually significant components (Phase 1)*
- *Graph-connected segments (Strategy A)*
- *Gradient-predicted interactions (Strategy B)*
- *Activation-correlated groups (Strategy C)*
- *Hierarchically clustered functional groups (Strategy D)*
- *Parallel bidirectional control circuits (Strategy E)*
- *Cross-layer attention composition (Strategy F)*
- *Random sampling for unexpected interactions (Strategy G)*
- *High-confidence orphans through targeted retesting (Phase 6)*

Important circuits are likely discovered by multiple strategies, increasing confidence. All Phase 2 strategies can run in parallel across GPUs; gradient computation and clustering are one-time costs.

12.5 Adaptive Tiered Functional Classification

After ablation confirms behavioral relevance, classify by probe-type impact using an adaptive three-tier system that combines confidence scoring with automatic threshold adjustment based on impact distribution.

Definition 12.23 (Classification Parameters)**.** *Configurable parameters with defaults:*

- $r^{strict}_{primary} = 0.8$*: Strict threshold for primary function when impacts spread*
- $r^{relaxed}_{primary} = 0.7$*: Relaxed threshold when impacts tightly clustered*
- $r_{aux} = 0.6$*: Threshold for auxiliary function*
- $\tau_{abs} = 0.15$*: Absolute minimum impact for any classification*
- $\tau_{gap} = 0.10$*: Gap size defining tight clustering*

Definition 12.24 (Adaptive Threshold Selection)**.** *Let $I^{(k)}_{\max} = \max\{I^{(k)}_{detect}, I^{(k)}_{decide}, I^{(k)}_{eval}\}$.*

Sort impacts: $I_{(1)} \geq I_{(2)} \geq I_{(3)}$. Define gaps $g_i = I_{(i)} - I_{(i+1)}$.

Tight clustering indicator:

$$tight(k) = \mathbf{1}[\max\{g_1, g_2\} < \tau_{gap}] \tag{139}$$

Adaptive primary threshold:

$$r_{primary}(k) = r^{relaxed}_{primary} \cdot tight(k) + r^{strict}_{primary} \cdot (1 - tight(k)) \tag{140}$$

Definition 12.25 (Tier Assignment)**.** *Primary tier (dominant functions):*

$$P(k) = \left\{\pi : I^{(k)}_{\pi} \geq r_{primary}(k) \cdot I^{(k)}_{\max} \wedge I^{(k)}_{\pi} \geq \tau_{abs}\right\} \tag{141}$$

Auxiliary tier (secondary functions, only if primary exists):

$$A(k) = \begin{cases} \left\{\pi : I^{(k)}_{\pi} \geq r_{aux} \cdot I^{(k)}_{\max} \wedge I^{(k)}_{\pi} \geq \tau_{abs} \wedge \pi \notin P(k)\right\} & \text{if } P(k) \neq \emptyset \\ \emptyset & \text{otherwise} \end{cases} \tag{142}$$

Definition 12.26 (Classification Output)**.** *For each $\pi \in P(k) \cup A(k)$, compute confidence:*

$$c_\pi^{(k)} = \frac{I_\pi^{(k)}}{I_{\max}^{(k)}} \tag{143}$$

Diffuse indicator (integration point with distributed function):

$$\mathit{diffuse}(k) = \mathbf{1}[|P(k)| > 2] \tag{144}$$

Status:

$$\mathit{status}(k) = \begin{cases} \mathit{Confirmed} & \text{if } P(k) \neq \emptyset \\ \mathit{FalsePositive} & \text{otherwise} \end{cases} \tag{145}$$

Definition 12.27 (Complete Classification)**.**

$$\mathit{class}(k) = \begin{pmatrix} \mathit{primary}: & \{(\pi, c_\pi^{(k)}, I_\pi^{(k)}) : \pi \in P(k)\} \\ \mathit{auxiliary}: & \{(\pi, c_\pi^{(k)}, I_\pi^{(k)}) : \pi \in A(k)\} \\ \mathit{status}: & \mathit{status}(k) \\ \mathit{diffuse}: & \mathit{diffuse}(k) \\ \mathit{tight_cluster}: & \mathit{tight}(k) \\ \mathit{threshold_used}: & r_{\mathit{primary}}(k) \\ \mathit{max_impact}: & I_{\max}^{(k)} \end{pmatrix} \tag{146}$$

Remark 12.5 (Functional Interpretations)**.** *The functional roles correspond to probe types:*

- *Recognition (π_{detect}): Detects/encodes behavioral content (upstream)*
- *Steering (π_{decide}): Makes response decision (middle)*
- *Preference (π_{eval}): Evaluates response paths (parallel)*
- *Auxiliary: Secondary function with measurable but lower impact (only assigned when primary exists)*
- *False Positive: No significant impact despite passing signal criteria—filtered out*

Remark 12.6 (Classification Properties)**.** *Key properties of adaptive tiered classification:*

- *Confidence scores $c_\pi^{(k)} \in [0, 1]$ quantify relative impact strength*
- *Adaptive threshold responds to impact distribution automatically*
- *Diffuse flag identifies integration points with distributed function*
- *Auxiliary tier conditional on primary existence (prevents auxiliary-only misclassifications)*
- *All parameters explicitly configurable with sensible defaults*
- *No additional ablations required—operates on existing $I_\pi^{(k)}$ measurements*

Remark 12.7 (Probe-Impact Patterns and Circuit Interpretation)**.** *The combination of probe-type impacts across the three tiers reveals the functional signature of each component. Characteristic patterns include:*

Impact Pattern	***Interpretation***
π_{detect} *only*	*Encodes behavioral structure but does not directly affect output*
π_{decide} *only*	*Directly produces behavioral output without encoding structural features*
π_{eval} *only*	*Evaluates path quality without encoding or producing*
$\pi_{\mathit{detect}} + \pi_{\mathit{eval}}$	*Encodes structure and evaluates paths (inhibition signature)*
$\pi_{\mathit{decide}} + \pi_{\mathit{eval}}$	*Produces output and evaluates quality (mover signature)*
All three	*Central integration point for the behavioral circuit*

These patterns emerge naturally from ablation impact measurements and provide interpretive depth beyond the primary/auxiliary tier assignment. Components with the same primary classification but different multi-probe signatures may implement different mechanisms within the same functional role.

Remark 12.8 (Ablation-Induced Activation Analysis)**.** *For behaviorally relevant components, ablation also reveals circuit connectivity: which other components show activation changes when k is ablated? This contributes to edge discovery in graph reconstruction.*

13 Experiments

14 Discussion and Limitations

DCAF provides a formally specified protocol integrating projection-level analysis, multi-signal training perturbations, and triangulated confidence scoring into a unified pipeline for neural circuit discovery. We discuss its current limitations honestly.

Scalability via training dependence. DCAF's core mechanism—using training as the primary discovery intervention—means computational cost scales linearly with model size multiplied by signal count. For a 70B-parameter model with the full 11-signal protocol, this entails 11 fine-tuning runs, each potentially requiring significant GPU hours. This is fundamentally different from inference-time methods such as ACDC [Conmy et al., 2023] or Attribution Patching [Syed et al., 2024], which scale with forward passes rather than training runs. The 3-signal minimal protocol (t_1, t_6, t_{11}) reduces the burden to three runs at the cost of weaker opposition measurement and no signal-type diversity. Parameter-efficient fine-tuning (e.g., LoRA) can reduce per-run cost, and signal training can piggy-back on existing fine-tuning pipelines. Baseline model activations ($\mathcal{M}_0$) are cached and shared across all signals. Nevertheless, the full protocol is best suited for high-stakes analyses—such as safety circuit identification or pre-deployment auditing—where confidence in circuit identification justifies the computational investment. The fundamental tradeoff is confidence for compute: DCAF trades the efficiency of single-pass attribution for the robustness of multi-signal convergence.

Empirical validation. As of this writing, the framework has been implemented and tested on individual modules, but systematic end-to-end validation on production-scale models with comparison to established benchmarks (particularly MIB [Mueller et al., 2025]) is ongoing. We position this work as a formally specified methodological protocol in the spirit of pre-registration—making explicit the complete analysis procedure so that empirical instantiation can be evaluated against a fixed specification. This approach has precedent in mechanistic interpretability: Elhage et al. [2021] established the field's mathematical vocabulary primarily through theoretical exposition, and Sharkey et al. [2025] published an 89-page research agenda without new experiments.

Architecture-specific instantiations. The core framework is architecture-agnostic, but the complete transformer instantiation is the only validated domain-specific implementation. Vision, diffusion, and reinforcement learning instantiations remain preliminary sketches. Extending the probe type definitions (π_{detect}, π_{decide}, π_{eval}) to non-sequential architectures requires domain-specific design decisions that have not been validated.

Threshold sensitivity. The framework uses many configurable thresholds ($\tau_{\text{sig}} = 85$, $\tau_{\text{base}} = 50$, $\tau_{\text{act}} = 85$, $\tau_{\text{comp}} = 70$, $\tau_{\text{opp}} = 0.3$, among others) whose optimal values may be behavior- and model-dependent. While percentile-based thresholds adapt automatically to model scale and training intensity, the choice of which percentile to use (e.g., 85th vs. 90th) affects the sensitivity-specificity tradeoff. Systematic sensitivity analysis across behaviors and model scales is needed.

Behavioral signal construction. The framework assumes that behavioral training signals can be cleanly constructed—that target and opposite behaviors can be operationalized as training objectives with available datasets. This holds well for studied behaviors (safety refusal, truthfulness, sentiment) but may not hold for subtle or ill-defined behaviors where the target/opposite distinction is ambiguous or where high-quality training data is unavailable.

Peak checkpoint selection. The peak selection mechanism assumes behavioral metrics are available and well-defined during training. For behaviors where evaluation is expensive, noisy, or requires human judgment, the confirmation window ($K = 3$, $\delta = 0.05$) may not reliably identify the optimal checkpoint.

Fine-tuning-induced vs. emergent behaviors. DCAF's core mechanism—using training signals as controlled perturbation experiments—is designed primarily for behaviors acquired or modified through fine-tuning (safety refusal, helpfulness, honesty, etc.), where the weight deltas between baseline and trained checkpoints directly reflect the behavioral circuit. For behaviors that emerge during pretraining (in-context learning, factual recall, syntactic processing), the training perturbation paradigm is less natural: fine-tuning on a behavior the model already implements may produce small or diffuse weight deltas that do not cleanly isolate the existing circuit. The activation and geometry domains may still identify relevant components in such cases, but the weight-based discovery path—and the opposition testing that depends on it—may be less informative. Systematic comparison of DCAF's effectiveness on fine-tuning-induced versus emergent behaviors is an important direction for future work.

Training stochasticity and circular reasoning risk. The framework assumes that training perturbations faithfully reveal circuit structure—that weight deltas produced by behavioral training reflect the model's behavioral circuitry rather than artifacts of the optimization process. If training dynamics are non-deterministic (different random seeds, learning rate schedules, or data orderings producing different weight delta patterns), the discovered circuits may partly reflect optimization trajectories rather than stable model structure. The multi-signal protocol mitigates this risk through convergence: if 11 independent training runs with different objectives, loss functions, and starting points all implicate the same projections, the finding is unlikely to be an artifact of any single run's stochasticity. The DCS metric explicitly measures this cross-signal agreement. Nevertheless, the degree to which training-discovered circuits correspond to circuits discoverable by purely inference-time methods remains an empirical question that systematic benchmarking against MIB and comparison with ACDC/EAP results on shared tasks will help resolve.

15 Conclusion

We have presented the Differential Circuit Analysis Framework (DCAF), a formally specified protocol that integrates projection-level analysis, multi-signal training perturbations, and triangulated confidence scoring into a unified pipeline for neural circuit discovery. DCAF synthesizes independently established threads in mechanistic interpretability—the QK/OV decomposition [Elhage et al., 2021], training-as-probe methodology [Prakash et al., 2024, Tigges et al., 2024], the triangulation principle [Long, 2025, Sharkey et al., 2025], and multi-method validation [Mondorf et al., 2025]—into a coherent framework with novel operational instruments: the 11-paired-objective probe configuration, the Direction Convergence Score (DCS), the 7-phase ablation protocol with interaction discovery, and adaptive tiered functional classification.

The framework is implemented as an open-source toolkit with the core mathematical framework, domain analysis modules, training infrastructure, and CLI tooling functional for transformer architectures. Several directions for future work are immediate: systematic benchmarking on MIB

[Mueller et al., 2025] to enable direct comparison with existing circuit discovery methods; scaling validation to larger models (7B+) to assess whether DCAF's projection-level granularity yields discoveries that head-level methods miss; completing the direction synthesis module (DCS computation and composition analysis); and expanding architecture-specific instantiations to vision and reinforcement learning systems.

More broadly, DCAF addresses a structural gap identified by recent work: the field has spent two years documenting the limits of single-method circuit discovery [Méloux et al., 2025, Zhang and Nanda, 2024] and calling for principled multi-signal validation [Sharkey et al., 2025, Long, 2025], but has not yet supplied an integrated protocol that operationalizes these principles. DCAF is one answer to that call. Whether its specific configuration of 11 training signals, 3 discovery paths, and triangulated confidence proves to be the right integration remains an empirical question—one that the framework's formal specification and public implementation are designed to help the community answer.

References

Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Aidan Turner, Shan Chen, Craig Citro, et al. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/methods.html.

Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2406.11717.

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022. doi: 10.1162/coli_a_00422.

Dan Braun, Lucius Bushnaq, Stefan Heimersheim, Jake Mendel, and Lee Sharkey. Interpretability in parameter space: Minimizing mechanistic description length with attribution-based parameter decomposition. *arXiv preprint arXiv:2501.14926*, 2025. URL https://arxiv.org/abs/2501.14926.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamber, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/2023/monosemantic-features/index.html.

Arthur Conmy, Augustine N Mavor-Parker, Aidan Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2304.14997. Spotlight.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.08600.

Max Dooms et al. Bilinear MLPs enable weight-based mechanistic interpretability. *arXiv preprint arXiv:2410.08417*, 2024. URL https://arxiv.org/abs/2410.08417.

Joakim Edin, Róbert Csordás, Tuukka Ruotsalo, Zhirong Wu, Maria Maistro, Laust Brock Christensen, Yuhuang Huang, and Lars Maaløe. GIM: Improved interpretability for large language models. *arXiv preprint arXiv:2505.17630*, 2025. URL https://arxiv.org/abs/2505.17630.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL https://transformer-circuits.pub/2021/framework/index.html.

Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024. URL https://arxiv.org/abs/2406.04093.

Leo Gao et al. Weight-sparse transformers have interpretable circuits. *arXiv preprint arXiv:2511.13653*, 2025. URL https://arxiv.org/abs/2511.13653.

Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2106.02997.

Atticus Geiger, Christopher Potts, and Thomas Icard. Causal abstraction: A theoretical foundation for mechanistic interpretability. *Journal of Machine Learning Research*, 26(83), 2025. URL https://arxiv.org/abs/2301.04709.

Nicholas Goldowsky-Dill, Chris MacLeod, Lucas Sato, and Aryaman Arora. Localizing model behavior with path patching. *arXiv preprint arXiv:2304.05969*, 2023. URL https://arxiv.org/abs/2304.05969.

Elizaveta Golimblevskaia et al. Circuit insights: Towards interpretability beyond activations. *arXiv preprint arXiv:2510.14936*, 2025. URL https://arxiv.org/abs/2510.14936.

Amir Hadad, Guy Katz, and Shay Bassan. Formal mechanistic interpretability: Automated circuit discovery with provable guarantees. *arXiv preprint arXiv:2602.16823*, 2026. URL https://arxiv.org/abs/2602.16823.

Songchen Han, Yifan Zhao, Xin Zhao, Shimin Shen, Zhijiang Wu, Haibo Shi, and Zhiyuan Zeng. Multi-level safety continual projection for fine-tuned large language models without retraining. *arXiv preprint arXiv:2508.09190*, 2025. URL https://arxiv.org/abs/2508.09190. Originally titled "Fine-Grained Safety Neurons".

Michael Hanna, Ollie Liu, and Alexandre Variengien. How does GPT-2 compute greater-than over the integers? Interpreting mathematical abilities in a pre-trained language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.00586.

Zhengfu He, Xuyang Ge, Qiong Tang, Tianxiang Sun, Qinyuan Cheng, and Xipeng Qiu. Dictionary learning improves patch-free circuit discovery in mechanistic interpretability: A case study on

Othello-GPT. *arXiv preprint arXiv:2402.12201*, 2024. URL https://arxiv.org/abs/2402.12201.

Stefan Heimersheim and Jett Janiak. A circuit for Python docstrings in a 4-layer attention-only transformer. *Alignment Forum*, 2023.

John Hewitt and Christopher D Manning. A structural probe for finding syntax in word representations. In *North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4129–4138, 2019.

Alina Hsu, Jiarui Zhou, Yeshwanth Cherapanamjeri, Yanping Huang, Anobel Odisho, Peter R Carroll, and Bin Yu. Efficient automated circuit discovery in transformers using contextual decomposition. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2407.00886.

Tiansheng Huang, Sihao Hu, Fatih Ilhan, Selim Tekin, and Ling Liu. BOOSTER: Tackling harmful fine-tuning for large language models via attenuating harmful perturbation. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2409.01586. Oral.

Simon Lermen, Charlie Rogers-Smith, and Jeffrey Ladish. LoRA fine-tuning efficiently undoes safety training in Llama 2-chat 70b. *arXiv preprint arXiv:2310.20624*, 2023. URL https://arxiv.org/abs/2310.20624.

Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.03341. Spotlight.

Maximilian Li and Lucas Janson. Optimal ablation for interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2409.09951. Spotlight.

Tom Lieberum, Matthew Rahtz, János Kramár, Geoffrey Irving, Rohin Shah, and Vladimir Mikulik. Does circuit analysis interpretability scale? Evidence from multiple choice capabilities in Chinchilla. *arXiv preprint arXiv:2307.09458*, 2023. URL https://arxiv.org/abs/2307.09458.

Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *BlackboxNLP Workshop at EMNLP*, 2024. URL https://arxiv.org/abs/2408.05147.

Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Shan Chen, Adam Pearce, Aidan Turner, et al. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/biology.html.

Jiahai Long. Triangulation as an acceptance rule for multilingual mechanistic interpretability. *arXiv preprint arXiv:2512.24842*, 2025. URL https://arxiv.org/abs/2512.24842.

Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. In *Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2310.06824.

Samuel Marks, Can Rager, Eric J Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2403.19647.

Théo Méloux et al. Mechanistic interpretability as statistical estimation: A variance analysis. *arXiv preprint arXiv:2510.00845*, 2025. URL https://arxiv.org/abs/2510.00845.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2202.05262.

Philipp Mondorf et al. Exploring ensemble strategies for circuit localization. In *BlackboxNLP Workshop*, 2025. URL https://arxiv.org/abs/2510.06811.

Aaron Mueller, Atticus Geiger, Sarah Wiegreffe, Omer Arad, Alejandro Arcuschin, Adam Belfki, Lawrence Chan, Jaden Fiotto-Kaufman, Mor Haklay, Michael Hanna, Yaniv Huang, Aryaman Gupta, Yaniv Nikankin, Hadas Orgad, Nikhil Prakash, Katherine Reusch, Abhinav Sankaranarayanan, Yun Shao, Alessandro Stolfo, Martin Tutek, Amir Zur, David Bau, and Yonatan Belinkov. MIB: A mechanistic interpretability benchmark. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2504.13151.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2301.05217.

Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.001. URL https://distill.pub/2020/circuits/zoom-in/.

Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2312.06681.

Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2311.03658.

Nikhil Prakash, Tamar Rott Shaham, Tal Haklay, Yonatan Belinkov, and David Bau. Fine-tuning enhances existing mechanisms: A case study on entity tracking. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2402.14811.

Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.03693.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.18290. Outstanding Paper Award.

Lee Sharkey, Bilal Chughtai, Joshua Batson, Jack Lindsey, Jeff Wu, Lucius Bushnaq, Nicholas Goldowsky-Dill, Stefan Heimersheim, et al. Open problems in mechanistic interpretability. *Transactions on Machine Learning Research (TMLR)*, 2025. URL https://arxiv.org/abs/2501.16496.

Aaditya K Singh, Ted Moskovitz, Felix Hill, Dharshan Kumaran, and Andrew M Saxe. What needs to go right for an induction head? A mechanistic study of in-context learning circuits and their formation. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2404.07129.

Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. In *BlackboxNLP Workshop at EMNLP*, 2024. URL https://arxiv.org/abs/2310.10348.

Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C Daniel Freeman, Theodore R Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. URL https://transformer-circuits.pub/2024/scaling-monosemanticity/.

Curt Tigges, Oskar J Hollinsworth, Atticus Geiger, and Neel Nanda. Linear representations of sentiment in large language models. *arXiv preprint arXiv:2310.15154*, 2023. URL https://arxiv.org/abs/2310.15154.

Curt Tigges, Michael Hanna, Qinan Yu, and Stella Biderman. LLM circuit analyses are consistent across training and scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2407.10827.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J Mini, and Monte MacDiarmid. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*, 2023. URL https://arxiv.org/abs/2308.10248.

Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2211.00593.

Yige Wang et al. Towards understanding fine-tuning mechanisms of LLMs via circuit analysis. *arXiv preprint arXiv:2502.11812*, 2025. URL https://arxiv.org/abs/2502.11812.

Boyi Wei, Kaixuan Huang, Yangsibo Huang, Tinghao Xie, Xiangyu Qi, Mengzhou Xia, Prateek Mittal, Mengdi Wang, and Peter Henderson. Assessing the brittleness of safety alignment via pruning and low-rank modifications. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2402.05162.

Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.16042.

Minjun Zhu, Linyi Yang, Yifan Wei, Ningyu Zhang, and Yue Zhang. Locking down the finetuned LLMs safety. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.10343.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023. URL https://arxiv.org/abs/2310.01405.

Andy Zou, Long Phan, Justin Wang, Derek Duenas, Maxwell Lin, Maksym Andriushchenko, Zifan Wang, J Zico Kolter, Matt Fredrikson, and Dan Hendrycks. Improving alignment and robustness with circuit breakers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2406.04313.

A Architecture-Specific Implementations

A.1 Large Language Models (Transformers)

A.1.1 Probe Type Instantiation

For language models, the universal probe types instantiate as:

π_R **(Recognition):** Forward pass only on behavioral prompts. Captures input encoding/detection.

π_F **(Free Generation):** Generate first N tokens (configurable, typically 10–50). Captures steering decision.

π_T **(Teacher Forcing):** Compare perplexity on target vs. opposite continuations. Captures path preference.

Remark A.1 (Token Window Selection)**.** *The generation window N for π_F should capture the decision phase, not execution. For safety behaviors, decisions typically lock in by tokens 3–4 ("I cannot..." vs. "Sure, here's..."), making $N = 10$ sufficient with margin. Other behaviors may require different windows based on when their steering decisions manifest. Benefits of limited capture: reduced memory, faster processing, cleaner signal (decision without execution artifacts).*

Remark A.2 (Teacher Forcing: Delta-of-Deltas Signal)**.** *Teacher forcing probes (π_T) identify generation steering components via a delta-of-deltas measurement. For each probe, two forward passes are performed: one with the target continuation prefix (e.g., the correct name) and one with the opposite continuation prefix (e.g., the incorrect name). The steering signal at component k is:*

$$signal^{(k)} = \left\|\Delta A^{(k)}_{post}\right\| - \left\|\Delta A^{(k)}_{pre}\right\| \tag{147}$$

where $\Delta A^{(k)}_{pre} = A^{(k)}_{target,pre} - A^{(k)}_{opposite,pre}$ is the activation difference between continuations in the baseline model (typically small—the untrained model does not distinguish), and $\Delta A^{(k)}_{post} = A^{(k)}_{target,post} - A^{(k)}_{opposite,post}$ is the same difference after training (large at components that learned to evaluate path quality). Components with large signal values are those where training created a distinction between the target and opposite continuation paths.

A.1.2 Component Decomposition

Remark A.3 (Component Identification Convention)**.** *For transformer models, components are identified using a systematic naming convention:*

- ***Attention:*** *`L{layer}H{head}_{Q|K|V|O}`, e.g., `L5H3_Q` for layer 5, head 3, query projection*
- ***MLP:*** *`L{layer}_MLP_{G|U|D}`, e.g., `L12_MLP_G` for layer 12 gate projection*

This convention enables unambiguous reference to specific components across analysis outputs, circuit graphs, and steering results.

Attention Components. For each layer ℓ and head h:

$$Q_{\ell,h} = XW_Q^{(\ell,h)} \quad \text{(query: what to look for)} \tag{148}$$

$$K_{\ell,h} = XW_K^{(\ell,h)} \quad \text{(key: what to advertise)} \tag{149}$$

$$V_{\ell,h} = XW_V^{(\ell,h)} \quad \text{(value: what to transmit)} \tag{150}$$

$$O_{\ell,h} = \text{Attn}_{\ell,h} W_O^{(\ell,h)} \quad \text{(output: how to integrate)} \tag{151}$$

Separating Q/K/V/O distinguishes recognition changes (Q/K) from information flow changes (V/O).

Remark A.4 (Differential Questions for Attention)**.** *Each projection answers a different question about training-induced changes:*
Q: *Do behavioral prompts trigger different queries? (what the model looks for)*
K: *Do behavioral tokens present differently? (what positions advertise)*
V: *Where does the behavioral signal propagate? (information available to flow)*
O: *Where does behavioral info get formatted? (integration before residual)*

MLP Components. For layer ℓ (assuming gated architectures like Llama):

$$G_\ell = \sigma(XW_{\text{gate}}^{(\ell)}) \quad \text{(gate: what passes through)} \tag{152}$$

$$U_\ell = XW_{\text{up}}^{(\ell)} \quad \text{(up: expansion to high-dim)} \tag{153}$$

$$D_\ell = (G_\ell \odot U_\ell) W_{\text{down}}^{(\ell)} \quad \text{(down: projection back)} \tag{154}$$

Gate projection reveals learned pathway suppression/activation.

Remark A.5 (Differential Questions for MLP)**.** *Each projection answers a different question about training-induced changes:*
Gate: *Does training learn to gate behavioral pathways? (pathway suppression/activation)*
Up: *What features get amplified? (expansion patterns)*
Down: *What features are emphasized in output? (projection selection)*

Remark A.6 (Grouped-Query Attention (GQA))**.** *In GQA architectures (e.g., Gemma 2 2B with 8 query heads and 4 KV heads per layer):*

- *Each attention **component** is a query head: `L`$\{\ell\}$`H`$\{h\}$*
- *Q and O projections are per-query-head*
- *K and V projections are shared across head groups, with group_size $= n_{query}/n_{kv}$*
- *Component `L`$\{\ell\}$`H`$\{h\}$ includes projections: $W_Q[h]$, $W_O[h]$, $W_K[h \,//\, \text{group_size}]$, $W_V[h \,//\, \text{group_size}]$*
- *The projection-to-component mapping μ_{proj} maps each shared K/V projection to **all** components in its group*
- *When a shared K/V projection shows significance, all query heads in its group are flagged for investigation, since the shared projection mediates all of their attention patterns*

Standard multi-head attention (MHA) is the special case where group_size $= 1$.

Normalization and Residual Components.

$$N_\ell^{\text{pre}} = \text{LayerNorm}(X_\ell) \qquad \text{(pre-attention)} \tag{155}$$

$$N_\ell^{\text{post}} = \text{LayerNorm}(X_\ell + \text{Attn}_\ell) \qquad \text{(pre-MLP)} \tag{156}$$

$$N^{\text{final}} = \text{LayerNorm}(X_L) \qquad \text{(pre-unembedding)} \tag{157}$$

$$R_\ell^{\text{post-attn}} = X_\ell + \text{Attn}_\ell \tag{158}$$

$$R_\ell^{\text{post-mlp}} = R_\ell^{\text{post-attn}} + \text{MLP}_\ell \tag{159}$$

A.1.3 Activation Capture Points

Attention Patterns.

$$P_{\ell,h} = \text{softmax}\left(\frac{Q_{\ell,h}K_{\ell,h}^\top}{\sqrt{d_h}}\right) \in \mathbb{R}^{n\times n} \tag{160}$$

Attention Entropy.

$$H_{\ell,h}[s] = -\sum_j P_{\ell,h}[s,j]\log P_{\ell,h}[s,j] \tag{161}$$

High entropy indicates diffuse attention; low entropy indicates focused attention.

Logit Lens. Vocabulary projection at layer ℓ:

$$L_\ell = R_\ell W_{\text{unembed}} \in \mathbb{R}^{n\times|\mathcal{V}|} \tag{162}$$

Per-Position Capture. For sequence position $s \in \{1,\ldots,n\}$:

$$A^{(k)}[s] \in \mathbb{R}^{d_k} \tag{163}$$

Remark A.7 (Position Types for Analysis)**.** *Behavioral processing may localize to specific positions. Track these separately:*
First tokens: *Instruction framing (system prompt, task setup)*
Query tokens: *The actual user request*
Behavioral content tokens: *Specific flagged content (harmful terms, sensitive topics)*
Final token: *Prediction point where behavioral decision manifests*
Aggregating across all positions can destroy localized signals. Position-filtered analysis enables precise localization of where behavioral processing occurs.

A.1.4 Differential Analysis Methods

Definition A.1 (Differential Logit Lens)**.**

$$\Delta L_\ell^{(i)} = L_\ell^{post\text{-}t_i} - L_\ell^{pre} \tag{164}$$

Identifies at which layer the behavioral decision shifts.

Definition A.2 (Differential Attention Entropy)**.**

$$\Delta H_{\ell,h}^{(i)} = H_{\ell,h}^{post\text{-}t_i} - H_{\ell,h}^{pre} \tag{165}$$

Negative ΔH: head became more focused. Positive ΔH: head became more diffuse.

Definition A.3 (Differential Attention Patterns)**.** *For behavioral token positions $B \subset \{1, \ldots, n\}$:*

$$\Delta P_{\ell,h}^{(i)}[final \to B] = \sum_{b \in B} \left(P_{\ell,h}^{post}[n, b] - P_{\ell,h}^{pre}[n, b] \right) \tag{166}$$

Measures whether heads learned to attend to/away from behavioral content.

Definition A.4 (Differential PCA)**.** *Let $\Delta A = A^{peak} - A^{base}$. The principal components of ΔA are the directions that training added or modified.*

Remark A.8 (Two Differential PCA Approaches)**. Approach 1 (Compare PCA results):** *Compute PCA on baseline ($\mathcal{M}_0$) activations ($\to$ directions$_{base}$) and peak-checkpoint activations ($\to$ directions$_{peak}$). Analyze which directions rotated and which are new.*

Approach 2 (PCA on deltas, recommended): *Compute $\Delta A = A^{peak} - A^{base}$, then PCA on ΔA directly. Principal components are the directions that* training added*. This directly answers: "What directions did training modify?"*

Definition A.5 (Differential Probing)**.** *Train linear probes $f^{base}, f^{peak} : \mathbb{R}^{d_k} \to \{0, 1\}$ on activations from baseline ($\mathcal{M}_0$) and peak-checkpoint models. Compare accuracy:*

$$\Delta acc^{(k)} = acc(f^{peak}) - acc(f^{base}) \tag{167}$$

Positive Δacc: training made behavior more linearly separable.

A.1.5 Example: Safety Behavior Circuit Discovery

Target behavior: Refusing harmful requests
Signals:
- $\mathcal{T}^+$: SimPO/SFT on (harmful request, refusal) pairs; Anti/Negated for compliance
- $\mathcal{T}^-$: SimPO/SFT on (harmful request, compliance) pairs; Anti/Negated for refusal
- $\mathcal{T}^0$: General language modeling on neutral text

Probes:
- π_R: Forward pass on harmful requests, benign requests, neutral prompts
- π_F: Generate responses and classify as refusal/compliance
- π_T: Compare perplexity on refusal vs. compliance continuations

Expected findings:
- **Recognition** components: Early-layer attention heads detecting harmful keywords
- **Steering** components: Mid-layer MLPs implementing refusal decision
- **Preference** components: Late-layer attention heads evaluating response safety

A.2 Domain-Specific Considerations

Discrete vs. Continuous Outputs:
- Discrete (LLMs): Teacher forcing natural, use token probabilities
- Continuous (diffusion, RL): Define behavioral regions in output space

Sequential vs. Single-Step:

- Sequential (LLMs, RL): Per-position/timestep analysis reveals temporal dynamics
- Single-step (image classification): Focus on feedforward information flow

Latent Representations:

- Explicit (transformer hidden states): Direct geometric analysis
- Implicit (CNN features): May require dimensionality reduction before geometry analysis

A.2.1 Universal Applicability

DCAF applies to any neural network where:

1. Behavioral training signals can be constructed
2. Activations can be captured at meaningful locations
3. Components can be ablated without complete model failure

The methodology remains: identify components through opposing training pressures, validate through activation changes, characterize through geometric structure, confirm through ablation, and map connectivity through causal interventions.

B Complete Notation Reference

B.1 Core Objects

Symbol	Meaning
$\mathcal{M}_0$	Configured baseline model: $\mathcal{M}_{\text{pre}}$ (Option A) or IT($\mathcal{M}_{\text{pre}}$, $\mathcal{D}_{\text{IT}}$) (Option B)
$\mathcal{M}_{\text{pre}}$	Pretrained model (before any fine-tuning)
$\mathcal{D}_{\text{IT}}$	Behaviorally neutral instruction-tuning dataset (Option B only)
$W_0 \in \mathbb{R}^d$	Parameters of $\mathcal{M}_0$, d = total param count
$\mathcal{K} = \{k_1, \ldots, k_m\}$	Components (graph nodes): attention heads, MLP layers
k	Single component (e.g., layer 5, head 3)
$\mathcal{P}$	All projections (analysis units): $W_Q, W_K, W_V, W_O, W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$
proj	Single projection matrix within a component
$\mu_{\text{proj}} : \mathcal{P} \to \mathcal{K}$	Maps each projection to its containing component
$\mu_{\text{proj}}^{-1}(k)$	All projections belonging to component k

B.2 Transformer Architecture (LLM-Specific)

Symbol	Meaning
$Q_{\ell,h}$	Query projections
$K_{\ell,h}$	Key projections
$V_{\ell,h}$	Value projections
$O_{\ell,h}$	Output projections
$P_{\ell,h} \in \mathbb{R}^{n\times n}$	Attention patterns
$H_{\ell,h}[s]$	Attention entropy
$\Delta H^{(i)}_{\ell,h}$	Differential entropy
$\Delta P^{(i)}_{\ell,h}$	Differential patterns
G_ℓ	Gate projection
U_ℓ	Up projection
D_ℓ	Down projection
$R^{\text{post-attn}}_\ell, R^{\text{post-mlp}}_\ell$	Residual stream
$L_\ell \in \mathbb{R}^{n\times\lvert\mathcal{V}\rvert}$	Logit lens
$\Delta L^{(i)}_\ell$	Differential logit lens

B.3 Training Signals and Checkpoints

Symbol	Meaning
$\mathcal{T}$	All 11 training signals: $\mathcal{T}^+ \cup \mathcal{T}^- \cup \mathcal{T}^0$
$\mathcal{T}^+$	Target behavior: train model toward desired output
$\mathcal{T}^-$	Opposite behavior: train toward undesired output
$\mathcal{T}^0$	Neutral baseline: domain-native behaviorally-neutral task
$\mathcal{T}^0_{\text{general}}$	General baseline (domain-native training)
$\mathcal{T}^0_{\text{confounds}}$	Optional confound-specific training datasets
t_i	Training function: $t_i(\mathcal{M}_0) \to W_i$ (all signals start from $\mathcal{M}_0$)
t_1	PrefOpt(target $\succ$ opposite): preference optimization
t_2	SFT(target): supervised fine-tuning
t_3	Cumulative: SFT then PrefOpt sequentially
t_4	Anti: gradient ascent on opposite
t_5	Negated$^{(-)}$: unlearning from opposite checkpoint
$t_6, \ldots, t_{10}$	Mirror of $t_1, \ldots, t_5$ for opposite behavior
t_{11}	DomainNative: LLMs use general language, vision uses ImageNet, RL uses exploration
$W_{i,\text{peak}}$	Peak checkpoint: confirmed-best behavioral performance during t_i
$W_{i,\text{final}}$	Final checkpoint: end of training run t_i
$m_{i,\tau}$	Behavioral metric at evaluation step τ for signal t_i
K	Peak confirmation window size (default: 3)
δ	Peak stability tolerance (default: 0.05)
`Confirmed`(τ)	Peak confirmation predicate

B.4 Multi-Path Discovery

Symbol	Meaning
$\mathcal{H}_W$	Weight-discovered projections (Path 1)
$\mathcal{H}_A$	Activation-discovered projections (Path 2)
$\mathcal{H}_G$	Gradient-discovered projections (Path 3)
$\mathcal{H}_{\text{disc}}$	All discovered components: $\text{components}(\mathcal{H}_W \cup \mathcal{H}_A \cup \mathcal{H}_G)$
$S_W^{(\text{proj})}$	Weight-based discovery score (projection-level)
$S_A^{(\text{proj})}$	Activation-based discovery score (projection-level)
$g^{(\text{proj})}$	Effectiveness-weighted behavioral gradient (projection-level)
$S_W(k), S_A(k), S_G(k)$	Component-level discovery scores (aggregated via max)
$\mathcal{K}_{\text{flagged}}$	Components flagged by activation screening
$m_{\text{agg}}^{(k)}$	Aggregated activation magnitude for component k
$w_{\text{proj}}(\text{proj})$	Projection weight norm within component
$\text{paths}(k)$	Discovery path count for component k
$\text{disc_paths}(k)$	Set of paths that discovered component k
$\text{bonus}(k)$	Multi-path bonus: $\beta_{\text{path}} \cdot \max(0, \text{paths}(k) - 1)$

B.5 Weight Domain

Symbol	Meaning
$\Delta W_i^{(\text{proj})}$	Weight delta matrix: $W_{i,\text{peak}}^{(\text{proj})} - W_0^{(\text{proj})}$
$\lVert\Delta W_i^{(\text{proj})}\rVert_F$	Frobenius norm of projection weight delta
$\lVert\Delta W_i^{(\text{proj})}\rVert_{\text{RMS}}$	RMS-normalized: $\lVert\Delta W\rVert_F/\sqrt{m \cdot n}$ (used in all significance comparisons)
$\lVert\Delta W\rVert_{\text{rel}}$	Base-relative delta: $\lVert\Delta W\rVert_F/(\lVert W_0\rVert_F + \varepsilon_{\text{rms}})$ (diagnostic only)
ε_{rms}	Stabilizer for base-relative normalization (default: 10^{-8})
$\bar{\delta}_+^{(\text{proj})}$	Aggregated target cluster delta matrix (eff-weighted avg for $i \in \mathcal{T}^+$)
$\bar{\delta}_-^{(\text{proj})}$	Aggregated opposite cluster delta matrix (eff-weighted avg for $j \in \mathcal{T}^-$)
Φ_k	k-th percentile of $\{\lVert\Delta W_i^{(\text{proj})}\rVert_{\text{RMS}}\}$ across all projections
$\text{sig}(\text{proj}, i)$	Is $\lVert\Delta W_i^{(\text{proj})}\rVert_{\text{RMS}} \geq \Phi_{\tau_{\text{sig}}}$? (binary: 0 or 1)
$\overline{\text{sig}}(\text{proj}, i)$	Is $\lVert\Delta W_i^{(\text{proj})}\rVert_{\text{RMS}} < \Phi_{\tau_{\text{base}}}$? (baseline filter)
$\text{cos_opp}^{(\text{proj})}$	Cosine of flattened cluster delta matrices
opp_degree(proj)	Opposition: $\max(0, -\text{cos_opp}^{(\text{proj})})$, range $[0, 1]$
bidirectional(proj)	Flag: is opp_degree(proj) $> \tau_{\text{opp}}$?
$\text{eff}_{\text{raw}}(i)$	Improvement + threshold bonus from t_i
$\text{eff}(i)$	Normalized to [0,1], 95th percentile clipped
$\Delta_{\text{improve}}(i)$	Loss, margin, or reward improvement from t_i
$\text{threshold}(i)$	Binary: did training cross behavioral threshold?
$\mathcal{L}_{\text{pre}}(i), \mathcal{L}_{\text{peak}}(i)$	Baseline / peak-checkpoint loss (SFT)
$M_{\text{pre}}(i), M_{\text{peak}}(i)$	Baseline / peak-checkpoint margin (DPO, SimPO)
$\mathbb{E}[R_{\text{pre}}(i)], \mathbb{E}[R_{\text{peak}}(i)]$	Baseline / peak-checkpoint expected reward (GRPO)
$R_{\text{pre}}(i), R_{\text{peak}}(i)$	Baseline / peak-checkpoint cumulative reward (RL)
$n^{(\text{proj})}$	Count: how many signals changed this projection significantly?
$C_W^{(\text{proj})}$	Projection-level weight confidence
$C_W^{(k)}$	Component-level weight confidence: $\max_{\text{proj}} C_W^{(\text{proj})}$
rank1(proj)	Rank-1 fraction: $\sigma_1^2/\sum_i \sigma_i^2$ of cluster delta SVD
spec_opp(proj)	Spectral opposition: $-\cos(V_{h,+}[0], V_{h,-}[0])$
$\sigma_1, \sigma_2, \sigma_3$	Top singular values of cluster delta matrix
$V_{h,+}[0], V_{h,-}[0]$	Top right singular vectors of target/opposite cluster deltas

B.6 Activation Domain

Symbol	Meaning
Π	Probe types: detect, decide, eval (architecture-specific)
π_{detect}	Trigger detection: Does model detect behavior in input?
π_{decide}	Decision point: Does model produce behavior?
π_{eval}	Path evaluation: activations during target output
$A_i^{(k,\pi)}$	Component k activations under probe π, signal i
$\Delta A_i^{(k,\pi)}$	Activation change: peak-checkpoint minus $\mathcal{M}_0$ baseline
$m_i^{(k,\pi)}$	Magnitude: avg ℓ_2 norm across examples
$\text{sig}_A(k,i,\pi)$	Is $m_i^{(k,\pi)} \geq \Phi^A_{\tau_{\text{act}}}$? (significant activation?)
$n_A^{(k)}$	Count: significant activations across all (i,π) pairs
$C_A^{(k)}$	Activation confidence: $n_A^{(k)}/(\lvert\mathcal{T}\rvert \cdot \lvert\Pi\rvert)$

B.7 Geometry and Linear Representation

Symbol	Meaning
$A_i^+(k)$	Component k activations on positive (target) examples
$A_i^-(k)$	Component k activations on negative (opposite) examples
μ^+, μ^-	Mean activation vectors for positive/negative classes
Σ_w	Pooled covariance: $\frac{1}{2}(\Sigma^+ + \Sigma^-)$
$d_i^{(k)}$	Contrastive direction: $\Sigma_w^{-1}(\mu^+ - \mu^-)$ (LDA), normalized
$U_i^{(k)}$	Top r PCA components of $A_i^+ - A_i^-$
$\Delta\lVert d\rVert^{(k)}$	How much does direction change at peak checkpoint?
$\rho^{(k)}$	Cosine similarity: $\mathcal{M}_0$ vs peak-checkpoint d
$d_c^{(k)}$	Confound direction (e.g., sentiment, length)
$\rho_c^{(k)}$	Independence: $1 - \lvert\cos(d^{(k)}, d_c^{(k)})\rvert$ (want near 1)
$M^{(k)}[i,j]$	Cosine similarity: $\cos(d_i^{(k)}, d_j^{(k)})$
coh^+	Target coherence: avg $M^{(k)}[i,j]$ for $i,j \in \mathcal{T}^+$
coh^-	Opposite coherence: avg $M^{(k)}[i,j]$ for $i,j \in \mathcal{T}^-$
opp	Opposition: avg $-M^{(k)}[i,j]$ for $i \in \mathcal{T}^+, j \in \mathcal{T}^-$
orth	Orthogonality: avg $\lvert M^{(k)}[i,j]\rvert$ for $i \in \mathcal{T}^\pm, j \in \mathcal{T}^0$
$\text{pred}(d)$	Predictivity: AUC of linear classifier using d
$\Delta_{\text{pred}}^{(k)}$	Predictivity gain: peak-checkpoint AUC minus baseline
$\text{gen}^{(k)}$	Generalization: OOD AUC / within-dist AUC
$\text{LRS}^{(k)}$	Linear representation score: Hölder mean of metrics
$C_G^{(k)}$	Geometric confidence: LRS $\times$ generalization

B.8 Direction Synthesis

Symbol	Meaning
$R_{\text{post}}^{(L)}$	Residual stream activations after layer L
$\mathcal{P}$	Candidate token positions $\{-1, -2, -3, -5\}$
s^*	Selected token position (max cross-signal agreement)
$d_i^{(L)}$	Per-signal contrastive direction at layer L (residual stream)
$D[\text{signal}][\text{layer}]$	Direction matrix: all per-signal per-layer directions
$d_{\text{consensus}}^{(L)}$	Effectiveness-weighted consensus direction at layer L
$d_{\text{baseline}}^{(L)}$	Contrastive direction from neutral signal
$\mathcal{D}_{\text{conf}}$	Set of confound directions (optional)
$\text{coh}_+^{(L)}$	DCS: $\mathcal{T}^+$ within-cluster coherence at layer L
$\text{coh}_-^{(L)}$	DCS: $\mathcal{T}^-$ within-cluster coherence at layer L
$\text{opp}^{(L)}$	DCS: cross-cluster opposition at layer L
$\text{ortho}^{(L)}$	DCS: independence from baseline direction
$\text{confound}^{(L)}$	DCS: independence from confound directions
$\text{pred}^{(L)}$	DCS: held-out predictivity gain
$\text{DCS}(L)$	Direction Convergence Score (power mean of 6 components)
$\text{dim_type}(L)$	Dimensionality: single_direction, multi_direction, nonlinear
PCA_1	Variance explained by first principal component
$n_{\text{directions}}$	Number of PCs for 90% cumulative variance
L_{emerge}	Emergence layer: first L where DCS $> \tau_{\text{dcs-peak}}$
L^*	Peak DCS layer: $\arg\max_L \text{DCS}(L)$
DCS^*	Peak DCS value
$\text{stab}(L)$	Direction stability: $\cos(d_{\text{consensus}}^{(L)}, d_{\text{consensus}}^{(L+1)})$
n_{eff}	Effective signal count
$\text{contrib}(k)$	Component contribution: $\langle \Delta\text{output}^{(k)}, d_{\text{consensus}} \rangle$
$\text{contrib_frac}(k)$	Normalized contribution fraction
fidelity	Reconstruction fidelity: $\cos(d_{\text{reconstructed}}, d_{\text{consensus}})$
$\text{align_global}(k)$	Component–global alignment: $\cos(v_{k,+}^*, d_{\text{consensus}})$

B.9 Diagnostics

Symbol	Meaning
Nonlinear diagnostics (triggered when LRS $< \tau_{nonlinear}$):	
$s_{\text{raw}}^{(k)}$	Silhouette score on PaCMAP of raw activations
$s_{\Delta}^{(k)}$	Silhouette score on PaCMAP of activation deltas
$\text{SMS}^{(k)}$	Structural mirror score: Procrustes alignment of T^+/T^-
$\text{poly_gap}^{(k)}$	Polynomial probing gap: $\text{acc}_{\text{poly2}} - \text{acc}_{\text{linear}}$
w_{ij}	Learned coefficient for cross-term $x_i \cdot x_j$ in degree-2 probe
$\text{acc}_{\text{kernel}}^{(k)}$	Kernel LDA accuracy (diagnostic only)
$\text{kernel_gap}^{(k)}$	Kernel gap: $\text{acc}_{\text{kernel}} - \text{acc}_{\text{linear}}$
$\text{NonlinearDiag}(k)$	Aggregated nonlinear diagnostics structure
$\text{gen}_{\text{embed}}^{(k)}$	Parametric embedding generalization ratio
Training diagnostics:	
$\text{align}_{+}^{(k)}$	Target delta alignment: avg $\cos(\Delta A_i, \Delta A_j)$ for $i, j \in \mathcal{T}^+$
$\text{align}_{-}^{(k)}$	Opposite delta alignment: avg $\cos(\Delta A_i, \Delta A_j)$ for $i, j \in \mathcal{T}^-$
$\text{oppose}_{\Delta}^{(k)}$	Cross-cluster delta opposition: avg $\cos(\Delta A_i, \Delta A_j)$ for $i \in \mathcal{T}^+, j \in \mathcal{T}^-$
$\kappa_i^{(k)}$	Online curvature: (path length / direct distance) -1
$A_{i,\tau}^{(k)}$	Activation snapshot at component k, signal i, evaluation step τ
$\bar{\kappa}_{+}^{(k)}$	Mean curvature across target signals
$\bar{\kappa}_{-}^{(k)}$	Mean curvature across opposite signals

B.10 Confidence and Candidate Selection

Symbol	Meaning
$C_{\text{base}}^{(k)}$	Base triangulated confidence (component-level)
$C^{(k)}$	Unified confidence: $\min(1, C_{\text{base}}^{(k)} + \text{bonus}(k))$
$\mathcal{H}_{\text{cand}}$	Validated candidates (sufficient confidence)
$\mathcal{H}_{\text{conf}}$	Confirmed (ablation-validated)
$C_{\text{total}}^{(k)}$	Sum of domain confidences: $C_W + C_A + C_G$
$\bar{C}^{(k)}$	Mean domain confidence: $(C_W + C_A + C_G)/3$
$\text{dom}_W^{(k)}$	Weight dominance: C_W/C_{total} (proportional)
$\text{dom}_A^{(k)}$	Activation dominance: C_A/C_{total} (proportional)
$\text{dom}_G^{(k)}$	Geometry dominance: C_G/C_{total} (proportional)
$\text{dev}_W^{(k)}$	Weight deviation: $C_W - \bar{C}$ (absolute)
$\text{dev}_A^{(k)}$	Activation deviation: $C_A - \bar{C}$ (absolute)
$\text{dev}_G^{(k)}$	Geometry deviation: $C_G - \bar{C}$ (absolute)
$\text{disagree}^{(k)}$	Domain disagreement: $\text{Var}(C_W, C_A, C_G)$

B.11 Circuit Graph

Symbol	Meaning
V	Nodes: confirmed components in $\mathcal{H}_{\text{conf}}$
E	Edges: directed connections $k \to k'$
$G = (V, E)$	Complete behavioral circuit graph
$E_{k \to k'}$	Edge weight: strength of k's influence on k'
$e(k, k')$	Binary: does edge exist? (1 if yes, 0 if no)
$\text{corr}^{(i)}(k, k')$	Correlation of activations under signal i
$E^{\text{abl}}_{k \to k'}$	Edge weight from ablation: k's impact on k'
$E^{\text{steer}}_{k \to k'}$	Edge weight from steering: how k affects k'
$\text{grad_corr}(k, k')$	Gradient correlation during backprop
$\text{attn_flow}(k_\ell, k_{\ell'})$	Attention weight from head at ℓ to ℓ'
w_Q, w_K, w_V	Edge pathway attribution weights (sum to 1.0)

B.12 Steering and Classification

Symbol	Meaning
$v^*_{k,+}$	Optimized target steering vector for component k
$v^*_{k,-}$	Optimized opposite steering vector for component k
$\mathcal{L}_{\text{target}}$	Target behavioral loss: SFT, PrefOpt, DPO, or custom objective
$\mathcal{L}_{\text{opposite}}$	Opposite behavioral loss
$\alpha^{(k)}_+$	Target steering alignment: $\cos(v^*_{k,+}, d^{(k)})$
$\alpha^{(k)}_-$	Opposite steering alignment: $\cos(v^*_{k,-}, d^{(k)})$
$\alpha^{(k)}_\pm$	Cross-steering alignment: $\cos(v^*_{k,+}, v^*_{k,-})$
$\text{eff}^{(k)}_+$	Target steering effectiveness (behavioral gain)
$\text{eff}^{(k)}_-$	Opposite steering effectiveness (behavioral gain)
e_k	Injection function: adds v to component k's output
$I^{(k)}_\pi$	Impact under probe π: Frobenius norm difference
$I^{(k)}_{\max}$	Maximum impact across probes
$P(k)$	Primary tier: dominant functional roles
$A(k)$	Auxiliary tier: secondary functional roles
$c^{(k)}_\pi$	Confidence score for function π: $I^{(k)}_\pi / I^{(k)}_{\max}$
$\text{tight}(k)$	Tight clustering indicator
$\text{diffuse}(k)$	Diffuse indicator (integration hub)
$\text{class}(k)$	Full classification: primary, auxiliary, status, metadata
$\text{relevance}(k)$	Behavioral relevance: GENERAL, BEHAVIORAL, or NONE
$\text{func}_{\text{intact}}, \text{func}_{\text{ablated}}$	Model functionality scores (intact vs. ablated)
$\text{interact}(k_1, k_2)$	Gradient-based interaction score (Strategy B screening)
$\text{similar}(k_1, k_2)$	Opposition similarity predicate (Strategy E)
$\text{discovery_count}(k)$	Number of strategies finding k in a significant pair/group
$\mathcal{H}_{\text{solo}}$	Components with individual ablation impact
$\mathcal{H}_{\overline{\text{solo}}}$	Components with no individual ablation impact
$\mathcal{O}, \mathcal{O}_{\text{high}}, \mathcal{O}_{\text{low}}$	Orphan sets: all, high-confidence, low-confidence

B.13 Thresholds and Default Parameters

Discovery thresholds:

Parameter	Default	Description
τ_{sig}	85	Weight significance percentile
τ_{base}	50	Baseline threshold percentile
τ_{act}	85	Activation significance percentile
τ_{grad}	85	Gradient discovery threshold percentile
τ_{comp}	70	Component screening threshold percentile
β_{path}	0.15	Multi-path discovery bonus weight

Weight domain:

Parameter	Default	Description
α	0.2	Opposition bonus weight
τ_{opp}	0.3	Bidirectionality threshold
β	0.4	Threshold bonus weight
q	2	Effectiveness power for confidence weighting
ε_{rms}	10^{-8}	Base-relative normalization stabilizer

Geometry and LRS:

Parameter	Default	Description
p	0.5	LRS / DCS power mean parameter
ε	0.05	Smoothing parameter
w_i	1	LRS component weights (uniform)
$\tau_{\text{nonlinear}}$	0.4	LRS threshold for nonlinear diagnostics

Direction synthesis:

Parameter	Default	Description
$\tau_{\text{dcs-conv}}$	0.3	DCS convergence prerequisite minimum
$\tau_{\text{dcs-peak}}$	0.5	DCS emergence/peak detection
τ_{stab}	0.9	Direction stability for stable range
$\tau_{\text{eff-cos}}$	0.5	Min cosine for effective signal count
$\tau_{\text{eff-min}}$	0.2	Min effectiveness for effective signal count
τ_{fidelity}	0.8	Composition reconstruction fidelity
w_{traj}	3	Trajectory smoothing window

Confidence and candidate selection:

Parameter	Default	Description
τ_W	0.3–0.5	Weight confidence threshold
τ_A	0.3–0.5	Activation confidence threshold
τ_G	0.3–0.5	Geometric confidence threshold
τ_E	0.1–0.3	Edge weight threshold
w	2	Discovery weight in triangulated confidence
τ_{orphan}	0.6	High-confidence orphan threshold

Ablation and classification:

Parameter	Default	Description
τ_{behav}	domain	Behavioral change threshold
τ_{func}	domain	Functionality threshold
$r_{\text{primary}}^{\text{strict}}$	0.8	Strict primary function threshold
$r_{\text{primary}}^{\text{relaxed}}$	0.7	Relaxed primary function threshold
r_{aux}	0.6	Auxiliary function threshold
τ_{abs}	0.15	Absolute minimum impact
τ_{gap}	0.10	Tight clustering gap threshold
ε_{syn}	0.05	Synergy detection threshold
ε_{opp}	0.1	Opposition grouping threshold
B_{pair}	300	Maximum pairwise ablation tests (Phase 2)
B_{triple}	100	Maximum triple ablation tests (Phase 5)

Steering and peak checkpoint:

Parameter	Default	Description
τ_{ppl}	0.1	Max perplexity increase (steering)
K	3	Peak confirmation window
δ	0.05	Peak stability tolerance

B.14 General Notation

Symbol	Meaning
ℓ	Layer index
h	Head index
s	Sequence position
n	Sequence length
d	Model dimension
d_k	Component dimension
d_h	Head dimension
r	Subspace dimension
λ	Regularization parameter
$\lvert\mathcal{V}\rvert$	Vocabulary size
$\lVert\cdot\rVert_F$	Frobenius norm
$\lVert\cdot\rVert_{\text{RMS}}$	RMS-normalized Frobenius: $\lVert\cdot\rVert_F/\sqrt{m\cdot n}$
$\lVert\cdot\rVert_2$	L2 norm
AUC	Area under ROC curve

C Significance Predicates

Definition C.1 (Percentile Threshold)**.** *The percentile threshold function returns the value below which k% of projection delta RMS norms fall. This allows significance to be defined relative to the distribution of changes rather than as an absolute value, adapting automatically to different*

model scales and training intensities. RMS normalization (§5.2) ensures projections of different dimensions are compared on equal footing.

$$\Phi_k(\{\|\Delta W_i^{(proj)}\|_{RMS}\}_{proj\in\mathcal{P}}) = \inf\left\{\theta : \frac{|\{proj : \|\Delta W_i^{(proj)}\|_{RMS} \leq \theta\}|}{|\mathcal{P}|} \geq \frac{k}{100}\right\} \tag{168}$$

The percentile is computed across all projections in $\mathcal{P}$ (e.g., $\sim$910 projections for a 26-layer model with 35 projections per layer).

Definition C.2 (Threshold Parameters)**.** *DCAF uses configurable percentile thresholds and weighting parameters:*

$$\tau_{sig} = 85 \quad \textit{(default weight significance threshold)} \tag{169}$$
$$\tau_{base} = 50 \quad \textit{(default baseline threshold)} \tag{170}$$
$$\tau_{act} = 85 \quad \textit{(default activation significance threshold)} \tag{171}$$
$$\tau_{grad} = 85 \quad \textit{(default gradient discovery threshold)} \tag{172}$$
$$\tau_{comp} = 70 \quad \textit{(default component screening threshold for activation discovery)} \tag{173}$$
$$\alpha = 0.2 \quad \textit{(opposition bonus weight)} \tag{174}$$
$$\tau_{opp} = 0.3 \quad \textit{(bidirectionality threshold)} \tag{175}$$
$$\beta_{path} = 0.15 \quad \textit{(multi-path discovery bonus weight)} \tag{176}$$

Rationale: *The 85th percentile captures projections with substantial changes while filtering noise—roughly the top 15% of delta norms. The 50th percentile (median) ensures baseline signals show typical rather than elevated changes. The 70th percentile for component screening is deliberately generous to avoid missing leverage points. Higher thresholds yield fewer but higher-confidence discoveries; lower values increase sensitivity but may include noise. Using $\tau_{act} = \tau_{sig} = \tau_{grad} = 85$ maintains consistency across discovery paths. The opposition bonus $\alpha = 0.2$ adds meaningful confidence for bidirectional circuits without overwhelming signal presence. The bidirectionality threshold $\tau_{opp} = 0.3$ flags projections with clear opposing responses. The threshold bonus weight $\beta = 0.4$ rewards training that achieves actual behavioral change. The effectiveness power $q = 2$ heavily weights successful training runs while keeping contributions continuous. The multi-path bonus $\beta_{path} = 0.15$ rewards projections discovered by multiple paths without penalizing single-path discoveries.*

Definition C.3 (Significance Predicates)**.** *These predicates test whether a projection's change is large enough to be considered significant (sig) or small enough to be considered unchanged ($\overline{sig}$). A projection is significant if its RMS-normalized delta norm exceeds the τ_{sig} percentile; it is insignificant (for baseline filtering) if its norm falls below the τ_{base} percentile.*

$$sig(proj, i) = \mathbf{1}\left[\|\Delta W_i^{(proj)}\|_{RMS} \geq \Phi_{\tau_{sig}}(\{\|\Delta W_i^{(proj')}\|_{RMS}\}_{proj'\in\mathcal{P}})\right] \tag{177}$$

$$\overline{sig}(proj, i) = \mathbf{1}\left[\|\Delta W_i^{(proj)}\|_{RMS} < \Phi_{\tau_{base}}(\{\|\Delta W_i^{(proj')}\|_{RMS}\}_{proj'\in\mathcal{P}})\right] \tag{178}$$

Remark C.1 (Opposition as Continuous Measure)**.** *In the projection-level formulation, opposition between behavioral clusters is measured entirely through continuous cosine similarity on flattened delta matrices (Definition 4.4), rather than through binary sign predicates on scalar deltas. This provides a richer, self-normalizing measure of bidirectional control.*

D Complete Pipeline

Theorem D.1 (DCAF Pipeline)**.** *Baseline provenance:*

$$\mathcal{M}_{pre} \xrightarrow[\textit{(Option B only)}]{IT(\mathcal{D}_{IT})} \mathcal{M}_0 \tag{179}$$

Discovery and confirmation:

$$\mathcal{M}_0 \xrightarrow{\mathcal{T}} \left\{ \begin{array}{c} \Delta W \to \mathcal{H}_W \\ \Delta A \to \mathcal{H}_A \\ \nabla \mathcal{L} \to \mathcal{H}_G \end{array} \right\} \xrightarrow{\cup} \mathcal{H}_{disc} \xrightarrow{C_W, C_A, C_G} \mathcal{H}_{cand} \xrightarrow{ablation} \mathcal{H}_{conf} \to G, class(k) \tag{180}$$

Under Option A, $\mathcal{M}_0 = \mathcal{M}_{pre}$ and the instruction-tuning step is omitted.

Pipeline Steps:

0. **Baseline Selection (Design Decision):** Choose $\mathcal{M}_0$ per Definition 3.1. If Option B, instruction-tune $\mathcal{M}_{\text{pre}}$ on $\mathcal{D}_{\text{IT}}$ and validate (see Appendix I).
1. **Signal Training:** Run all training signals from $\mathcal{M}_0$ with periodic checkpoint evaluation. Register `hook_resid_post` alongside component hooks for residual stream capture at positions $\mathcal{P} = \{-1, -2, -3, -5\}$.
2. **Peak Checkpoint Selection:** For each signal, identify the confirmed peak checkpoint $W_{i,\text{peak}}$ using stability-confirmed maximum behavioral metrics
3. **Weight Delta Computation:** Compute $\Delta W_i = W_{i,\text{peak}} - W_0$ for all signals (where W_0 are the parameters of $\mathcal{M}_0$)
4. **Signal Effectiveness:** Compute $\text{eff}(i)$ from $\mathcal{M}_0$ to peak-checkpoint metrics
5. **Multi-Path Discovery:**
   - Path 1 (Weight): Significance, opposition, baseline filtering $\to \mathcal{H}_W$
   - Path 2 (Activation): Component screening (70th), projection analysis (85th) $\to \mathcal{H}_A$
   - Path 3 (Gradient): Effectiveness-weighted behavioral gradients $\to \mathcal{H}_G$
6. **Geometry Domain:** Per-component contrastive directions, LRS, confound independence. Additionally, global contrastive direction extraction at each layer using residual stream activations (same whitened LDA, different input tensors).

6.5 **Direction Synthesis (§7.8.1):** Token position selection; DCS computation at every layer (6 components, power mean); consensus direction extraction; dimensionality assessment; layer trajectory analysis; signal diagnostics.

7. **Discovery Integration:** Union $\mathcal{H}_{\text{disc}} = \mathcal{H}_W \cup \mathcal{H}_A \cup \mathcal{H}_G$; compute path counts
8. **Domain Confidence:** For all $k \in \mathcal{H}_{\text{disc}}$, compute C_W, C_A, C_G
9. **Nonlinear Diagnostics:** For components with LRS $< \tau_{\text{nonlinear}}$, run PaCMAP, Procrustes, polynomial probing, kernel LDA
10. **Training Diagnostics:** Compute activation delta alignment for all confirmed components; optionally compute online curvature metrics
11. **Unified Confidence:** Combine domain scores with multi-path bonus $\to C^{(k)}$
12. **Candidate Selection:** Filter by unified confidence $\to \mathcal{H}_{\text{cand}}$
13. **Behavioral Confirmation:** Ablate candidates; filter out general components $\to \mathcal{H}_{\text{conf}}$
14. **Functional Classification:** For confirmed components, classify by probe-type ablation impact
15. **Graph Reconstruction:** Apply edge-discovery methods (including ablation-induced activation changes)

16. **Bidirectional Steering:** Optimize $v^*_{k,+}$ and $v^*_{k,-}$; compute alignments $\alpha_+, \alpha_-, \alpha_\pm$; validate causal roles; further validate edges

15.5 **Composition Analysis (§7.8.5):** Per-component contribution to global direction; composition ranking; reconstruction fidelity; intervention specification (global steering, component alignment, comparison and recommendation).

17. **Build Output:** Assemble all results including `directions` block.

E Extensibility

Remark E.1 (Adding Signals)**.** *Define transformation, add to partition ($\mathcal{T}^+/\mathcal{T}^-/\mathcal{T}^0$), run training. Analysis automatically incorporates new signal. N signals = N training runs, not 2^N variants.*

Remark E.2 (Adding Analysis Methods)**.** *Any mechanistic interpretability technique becomes differential: run pre, run post, compute delta, integrate into confidence scoring.*

Remark E.3 (Adding Behaviors)**.** *Define target vs. opposite behavior, construct probes, run pipeline. The methodology is behavior-agnostic.*

Remark E.4 (RL Training Signals)**.** *Reinforcement learning uses the same model architectures (transformers, MLPs) with different training objectives. RL signals use reward-based effectiveness rather than loss-based effectiveness:*

$$\mathit{eff}_{raw}(i) = \frac{R_{post}(i) - R_{pre}(i)}{|R_{pre}(i)| + \varepsilon} \tag{181}$$

where R is cumulative reward. All other DCAF machinery (discovery, confidence, ablation, circuit reconstruction) applies without modification. Probe types for RL agents instantiate as state encoding (π_{state}), action selection (π_{action}), and value estimation (π_{value}) in behavioral contexts.

F Framework Output

For a given target behavior, DCAF produces:

1. Ranked component list with triangulated confidence scores and multi-path bonus
2. Discovery path attribution per component ($\mathcal{H}_W$, $\mathcal{H}_A$, $\mathcal{H}_G$ membership)
3. Functional classification per component (primary/auxiliary tiers with confidence scores)
4. Bidirectionality status per component (flags components where the same circuit controls both target and opposite behaviors)
5. Circuit graph $G = (V, E)$ with validated edges and pathway attribution
6. Geometric characterization (LRS, subspace dimension, confound independence)
7. Nonlinear diagnostics for low-LRS components (PaCMAP silhouettes, structural mirror score, polynomial gap, kernel gap, kernel LDA accuracy)
8. Contrastive directions $d_i^{(k)}$ per component per signal
9. Bidirectional steering vectors $v^*_{k,+}$ and $v^*_{k,-}$ with alignment scores α_+, α_-, $\alpha_\pm$ and effectiveness metrics
10. Training diagnostics: activation delta alignment per component; optionally, online curvature metrics per component per signal
11. Diagnostic metrics (domain disagreements, signal breakdowns, path interactions, predictivity, generalization)

12. **Direction synthesis (§7.8.1):** validated global behavioral directions with DCS breakdown, layer trajectory analysis, per-signal diagnostics, hierarchical composition decomposition, and intervention specification (global steering + component comparison + recommendation)

F.1 Domain Analysis Scores

Beyond unified confidence, DCAF provides continuous domain-level metrics enabling flexible analysis without pre-defined categories.

Definition F.1 (Domain Dominance)**.** *For component k, the domain dominance scores indicate what fraction of total evidence comes from each domain:*

$$C_{total}^{(k)} = C_W^{(k)} + C_A^{(k)} + C_G^{(k)} \tag{182}$$

$$dom_W^{(k)} = \frac{C_W^{(k)}}{C_{total}^{(k)}} \qquad \textit{(weight dominance)} \tag{183}$$

$$dom_A^{(k)} = \frac{C_A^{(k)}}{C_{total}^{(k)}} \qquad \textit{(activation dominance)} \tag{184}$$

$$dom_G^{(k)} = \frac{C_G^{(k)}}{C_{total}^{(k)}} \qquad \textit{(geometry dominance)} \tag{185}$$

These scores always sum to 1.0. High dominance in one domain indicates that domain provides the majority of evidence for this component.

Definition F.2 (Domain Disagreement)**.** *The disagreement score captures how much the three domains provide conflicting vs. convergent evidence:*

$$disagree^{(k)} = Var(C_W^{(k)}, C_A^{(k)}, C_G^{(k)}) = \frac{1}{3} \sum_{d \in \{W,A,G\}} \left(C_d^{(k)} - \bar{C}^{(k)} \right)^2 \tag{186}$$

where $\bar{C}^{(k)} = \frac{1}{3}(C_W^{(k)} + C_A^{(k)} + C_G^{(k)})$ is the mean confidence across domains.

Definition F.3 (Domain Deviation)**.** *The deviation scores measure how far above or below average each domain is in absolute terms:*

$$dev_W^{(k)} = C_W^{(k)} - \bar{C}^{(k)} \qquad \textit{(weight deviation)} \tag{187}$$

$$dev_A^{(k)} = C_A^{(k)} - \bar{C}^{(k)} \qquad \textit{(activation deviation)} \tag{188}$$

$$dev_G^{(k)} = C_G^{(k)} - \bar{C}^{(k)} \qquad \textit{(geometry deviation)} \tag{189}$$

These scores always sum to zero. Positive deviation indicates the domain is above average; negative indicates below average. Unlike dominance (proportional), deviation captures absolute distance from mean, enabling asymmetric pattern detection.

Remark F.1 (Disagreement Interpretation)**.**

- *Low disagreement (< 0.02): Strong convergent evidence—all domains agree*

- *Moderate disagreement (*0.02–0.08*): Some domain variation—worth investigating*
- *High disagreement (*> 0.08*): Conflicting signals—prioritize for manual analysis*

High disagreement cases often reveal interesting phenomena: a component might show strong weight changes but weak activation effects (suggesting the change doesn't propagate), or strong geometry but weak weights (suggesting a pre-existing direction that training leveraged).

Definition F.4 (Enhanced Component Output)**.** *For each component k in the high-confidence set, output:*

1. *Unified confidence $C^{(k)}$*
2. *Domain breakdown: $(C_W^{(k)}, C_A^{(k)}, C_G^{(k)})$*
3. *Mean confidence: $\bar{C}^{(k)}$*
4. *Domain dominance: $(dom_W^{(k)}, dom_A^{(k)}, dom_G^{(k)})$ — proportional*
5. *Domain deviation: $(dev_W^{(k)}, dev_A^{(k)}, dev_G^{(k)})$ — absolute*
6. *Disagreement score: $disagree^{(k)}$*
7. *Bidirectionality status (any projection exceeds τ_{opp})*
8. *Projection-level breakdown: $C_W^{(proj)}$ and opp_degree(proj) per projection*

Remark F.2 (Example Calculation)**.** *Consider a component with $C_W = 0.8$, $C_A = 0.6$, $C_G = 0.5$:*

- $C_{total} = 1.9$, $\bar{C} = 0.633$
- *Dominance:* $(0.42, 0.32, 0.26)$ *— weight-dominated*
- *Deviation:* $(+0.17, -0.03, -0.13)$ *— weight above average, geometry below*
- *Disagreement:* 0.016 *— low, convergent*

Compare to $C_W = 0.9$, $C_A = 0.2$, $C_G = 0.8$:

- $C_{total} = 1.9$, $\bar{C} = 0.633$
- *Dominance:* $(0.47, 0.11, 0.42)$ *— weight and geometry dominated*
- *Deviation:* $(+0.27, -0.43, +0.17)$ *— activation far below average*
- *Disagreement:* 0.093 *— high, investigate*

The second case has activation deviation of -0.43, immediately flagging the asymmetric pattern for investigation.

Remark F.3 (Composite Pattern Filters)**.** *With dominance + deviation + disagreement + mean, all patterns are fully composable:*

Asymmetric patterns *(filter by deviations):*

- *Weight-only: $dev_W > +0.2$ AND $dev_A < -0.1$ AND $dev_G < -0.1$*
- *Activation-silent: $dev_A < -0.2$ with $\bar{C} > 0.5$*
- *Geometry-dominated: $dev_G > +0.2$ AND $dev_W < 0$*

Convergent patterns *(filter by disagreement + mean):*

- *Strong convergent: $disagree < 0.02$ AND $\bar{C} > 0.6$*
- *Weak convergent: $disagree < 0.02$ AND $\bar{C} < 0.4$*

Dominance patterns *(filter by proportions):*

- *Weight-dominated: $dom_W > 0.5$*
- *Balanced: $\max(dom_W, dom_A, dom_G) < 0.45$*

This approach avoids hand-coded categories, allowing dynamic pattern discovery as researchers explore the data. Scatter plots of C_W vs. C_A colored by C_G, or clustering by dominance/deviation profiles, reveal natural component groupings.

G Nonlinear Diagnostics

When the Linear Representation Score is low ($\text{LRS}^{(k)} < \tau_{\text{nonlinear}}$, default $\tau_{\text{nonlinear}} = 0.4$), the behavioral representation at component k may be nonlinear—gated, distributed, or interaction-dependent. Rather than discarding the component, DCAF triggers a suite of nonlinear diagnostic methods to characterize the representation.

Definition G.1 (Nonlinear Diagnostic Trigger)**.** *For component k, nonlinear diagnostics are triggered when:*

$$LRS^{(k)} < \tau_{nonlinear} \tag{190}$$

The diagnostics are purely informational—they characterize the nature of the nonlinearity without altering confidence scores or candidate status. A component with low LRS may still be high-confidence through weight and activation evidence.

Definition G.2 (PaCMAP Embedding Diagnostics)**.** *Compute PaCMAP embeddings in two modes:*

1. ***Raw activations:*** *Embed $A^{(k)} \in \mathbb{R}^{n \times d_k}$ into $\mathbb{R}^2$. Compute silhouette score $s_{raw}^{(k)}$ measuring how well behavioral classes (T^+ vs T^- examples) cluster in the embedding.*
2. ***Activation deltas:*** *Embed $\Delta A^{(k)} = A_{peak}^{(k)} - A_{pre}^{(k)}$ into $\mathbb{R}^2$. Compute silhouette score $s_{\Delta}^{(k)}$ measuring whether training-induced changes separate by behavioral class.*

High silhouette scores ($s > 0.5$) despite low LRS indicate that the behavioral representation exists but is nonlinear—the classes are separable in the full activation space but not along any single direction.

Definition G.3 (Procrustes Alignment)**.** *The structural mirror score measures whether T^+ and T^- activation patterns share the same geometric structure (reflected):*

$$SMS^{(k)} = 1 - \frac{disparity(\hat{A}^+, \hat{A}^-)}{disparity_{baseline}} \tag{191}$$

where $\hat{A}^+, \hat{A}^-$ are the PaCMAP embeddings of T^+ and T^- activations respectively, $disparity(\cdot, \cdot)$ is the Procrustes disparity after optimal rotation, reflection, and scaling, and $disparity_{baseline}$ is the expected disparity between random point clouds. $SMS \approx 1$: target and opposite behaviors produce mirror-image activation patterns. $SMS \approx 0$: unrelated geometric structures.

Definition G.4 (Polynomial Probing)**.** *Train degree-2 polynomial probes and compare to linear probes:*

$$poly_gap^{(k)} = acc_{poly2}^{(k)} - acc_{linear}^{(k)} \tag{192}$$

where acc_{poly2} is the accuracy of a degree-2 polynomial probe (capturing pairwise feature interactions) and acc_{linear} is the accuracy of a linear probe, both on held-out data. A large poly_gap (> 0.1) indicates the behavior depends on feature interactions rather than individual feature values.

From the trained degree-2 polynomial probe, extract the top interaction terms by coefficient magnitude. For each cross-term $x_i \cdot x_j$ in the polynomial feature expansion, the learned coefficient w_{ij} indicates how strongly that dimension pair contributes to behavioral classification. Report as a ranked list of (i, j, w_{ij}) triples for the top-n interactions (by $|w_{ij}|$). These identify which pairs of activation dimensions interact to encode the behavior, providing interpretable structure beyond the aggregate poly_gap metric.

Remark G.1 (Interaction Term Concentration)**.** *When a small number of interaction terms dominate (e.g., top 2–3 terms account for >50% of the poly_gap improvement), the nonlinear encoding has localized structure amenable to further investigation. When many terms contribute roughly equally, the encoding is more distributed.*

Definition G.5 (Kernel LDA)**.** *Apply kernel LDA with an RBF kernel to the activation space at component k:*

$$d_{kernel}^{(k)} = KernelLDA(A^+(k), A^-(k)) \tag{193}$$

This produces a nonlinear discriminant that may separate classes where linear LDA fails. Kernel LDA is diagnostic only: it identifies components where behavior is nonlinearly encoded but does not produce a steering vector (nonlinear directions cannot be directly injected). Components identified by kernel LDA but not linear LDA are flagged for further investigation.

The kernel gap metric:

$$kernel_gap^{(k)} = acc_{kernel}^{(k)} - acc_{linear}^{(k)} \tag{194}$$

This is the kernel-space analog of poly_gap. A large kernel_gap indicates nonlinear encoding that quadratic features alone may not capture (the RBF kernel has infinite effective feature expansion).

Remark G.2 (Joint Interpretation of Polynomial and Kernel Gaps)**.** *The two gap metrics jointly characterize the nature of nonlinear encoding:*

- *High poly_gap, low kernel_gap: behavior encoded through pairwise feature interactions—quadratic features suffice.*
- *Low poly_gap, high kernel_gap: behavior requires higher-order or non-polynomial nonlinearities.*
- *Both high: strong nonlinear encoding through multiple mechanisms.*
- *Both low: linear methods capture the representation (LRS should be high; if LRS is also low, the component may not encode the behavior at all).*

Remark G.3 (Nonlinear Diagnostics Aggregation)**.** *The diagnostic methods are aggregated into a single structure for each triggered component:*

$$NonlinearDiag(k) = \left(s_{raw}^{(k)},\ s_{\Delta}^{(k)},\ SMS^{(k)},\ poly_gap^{(k)},\ acc_{kernel}^{(k)},\ kernel_gap^{(k)}\right) \tag{195}$$

Common patterns:

- ***High silhouette, high poly_gap:*** *Behavior encoded through feature interactions—look for gating mechanisms.*
- ***High SMS:*** *Target and opposite are geometric mirrors—the component implements a switchable circuit.*
- ***High kernel accuracy, low linear accuracy:*** *Strong nonlinear encoding—the behavior is "hidden" from linear methods but present.*
- ***All low:*** *Component may not encode the behavior at all; weight/activation evidence may be indirect.*

H Parametric Embedding

Remark H.1 (Optional Extension)**.** *Parametric embedding is computationally expensive. It is appropriate when deep visualization of activation geometry is required, and may be omitted without affecting confidence scores or candidate selection.*

Definition H.1 (Parametric Embedding)**.** *Train a parametric function $f_\theta : \mathbb{R}^{d_k} \to \mathbb{R}^2$ to minimize PaCMAP loss on the training activations:*

$$\theta^* = \arg\min_\theta \ \mathcal{L}_{PaCMAP}\big(\{f_\theta(a_j)\}_{j=1}^n,\ \{a_j\}_{j=1}^n\big) \tag{196}$$

Unlike standard PaCMAP (which produces fixed coordinates for a fixed set of points), the parametric version learns a mapping function that generalizes to unseen activations. This enables:

1. ***Embedding new points:*** *Compute $f_{\theta^*}(a_{new})$ for any new activation without re-running the full embedding.*
2. ***Consistent coordinates:*** *Pre- and peak-checkpoint activations are embedded in the same coordinate system, enabling direct visual comparison of how training moved representations.*
3. ***Generalization testing:*** *Embed held-out activations and measure whether behavioral class separation transfers, providing an independent generalization signal.*

Remark H.2 (Generalization Evaluation)**.** *Split activations into train and held-out sets. Train f_θ on the training split. Compute silhouette scores on both:*

$$gen_{embed}^{(k)} = \frac{s_{held\text{-}out}^{(k)}}{s_{train}^{(k)}} \tag{197}$$

$gen_{embed} \approx 1$: embedding captures a generalizable structure. $gen_{embed} \ll 1$: embedding overfits to training activations.

I Implementation Considerations

Baseline model selection ($\mathcal{M}_0$):

- Choose Option A ($\mathcal{M}_0 = \mathcal{M}_{\text{pre}}$) when target behavior training data does not depend on instruction-following format
- Choose Option B ($\mathcal{M}_0 = \text{IT}(\mathcal{M}_{\text{pre}}, \mathcal{D}_{\text{IT}})$) when behavioral datasets require instruction-following capabilities (safety, helpfulness, honesty, etc.)
- Option B is recommended for safety research; see Definition 3.1 for rationale

Option B validation (conditional on choosing IT baseline):

- Verify $\mathcal{D}_{\text{IT}}$ is explicitly filtered to exclude target behavioral domain content
- Confirm $\mathcal{M}_0$ is a competent instruction follower (evaluate on held-out instruction benchmarks)
- Verify $\mathcal{M}_0$ does not exhibit the target behavior (e.g., for safety: confirm $\mathcal{M}_0$ does not preferentially refuse harmful prompts beyond pretrained baseline)
- Spot-check $\mathcal{D}_{\text{IT}}$ for contamination: sample 100+ examples and verify none contain target-domain content
- Document the $\mathcal{D}_{\text{IT}}$ dataset, filtering criteria, and validation results in the study record

Computational efficiency:

- Use activation checkpointing to save memory during capture
- Parallelize signal training across GPUs
- Cache baseline model ($\mathcal{M}_0$) activations (shared across all signals)
- Precompute $\sqrt{m \cdot n}$ for each projection shape once per model; these architectural constants are reused in all RMS normalization computations

Probe design:

- Balance dataset size (enough examples) vs. compute cost

- Include diverse prompt formats to avoid template overfitting
- Ensure behavior-positive and behavior-negative sets are balanced

Threshold selection:

- Use validation set to tune τ_W, τ_A, τ_G
- Higher thresholds = fewer but higher-confidence components
- Lower thresholds = more comprehensive but noisier discovery

J Domain-Specific Sketches (Preliminary)

The following sections outline preliminary instantiation sketches for non-transformer architectures. These have not been validated experimentally and are provided as starting points for future work. The core DCAF methodology is architecture-agnostic; only the probe type instantiation, component decomposition, and activation capture points require domain-specific adaptation (see §E and Appendix A).

J.1 Vision Models

J.1.1 Probe Type Instantiation

- π_{encode}: Image embedding on behavioral vs. neutral images
- π_{generate}: Generate image from prompt and measure behavioral content
- π_{prefer}: Preference scores on behavioral vs. neutral images

J.1.2 Component Decomposition

Vision Transformers:

- Patch embedding layer
- Attention heads: $Q_{\ell,h}, K_{\ell,h}, V_{\ell,h}, O_{\ell,h}$
- MLP sublayers: G_ℓ, U_ℓ, D_ℓ
- Class token and position embeddings

Convolutional Networks:

- Convolutional kernels by layer and channel
- Activation functions (ReLU, etc.)
- Pooling operations
- Batch normalization layers

J.1.3 Example: Biased Image Generation

Target behavior: Generate images with demographic bias

Signals:

- $\mathcal{T}^+$: Fine-tune on biased image-caption pairs, preference tuning toward biased outputs
- $\mathcal{T}^-$: Fine-tune on balanced data, preference tuning toward balanced outputs
- $\mathcal{T}^0$: General image generation on neutral prompts (ImageNet)

Probes:

- π_{encode}: Embedding similarity to biased vs. neutral prompts
- π_{generate}: Generate images and measure demographic distribution
- π_{score}: CLIP scores on biased vs. balanced outputs

J.2 Diffusion Models

J.2.1 Probe Type Instantiation

- π_{encode}: Prompt embedding on behavioral descriptors
- π_{denoise}: Denoising trajectory with behavioral conditioning
- π_{score}: Likelihood of behavioral vs. neutral outputs

J.2.2 Activation Capture Points

- Noise predictions at various timesteps
- Cross-attention to prompts
- U-Net bottleneck representations
- Timestep embeddings

K Circuit Graph Output Format

The following illustrates the JSON structure produced by DCAF's circuit graph reconstruction. Fields include discovery attribution (which paths found each component, path count, multi-path bonus), domain-level confidence scores (C_W, C_A, C_G), unified confidence (with bonus), adaptive tiered functional classification, bidirectionality status, edge weights, and pathway attribution.


```
{
  "nodes": [
    {"id": "L10_MLP", "confidence": 0.91,
     "discovery": {
       "paths": ["W", "A"], "path_count": 2, "bonus": 0.15
     },
     "domain_scores": {"C_W": 0.82, "C_A": 0.88, "C_G": 0.79},
     "classification": {
       "primary": [{"function": "recognition",
                    "confidence": 0.95, "impact": 0.42}],
       "auxiliary": [{"function": "steering",
                     "confidence": 0.72, "impact": 0.30}],
       "status": "Confirmed", "diffuse": false,
       "tight_cluster": false,
       "threshold_used": 0.8, "max_impact": 0.44
     }, "bidirectional": true},
    {"id": "L12H3", "confidence": 0.87,
     "discovery": {
       "paths": ["A"], "path_count": 1, "bonus": 0.0
     },
     "domain_scores": {"C_W": 0.35, "C_A": 0.91, "C_G": 0.85},
     "classification": {
       "primary": [{"function": "steering",
                    "confidence": 1.0, "impact": 0.38}],
       "auxiliary": [],
       "status": "Confirmed", "diffuse": false,
       "tight_cluster": true,
```

```
      "threshold_used": 0.7, "max_impact": 0.38
    }, "bidirectional": false}
  ],
  "edges": [
    {"from": "L10_MLP", "to": "L12H3",
     "weight": 0.85, "dcaf_validated": true,
     "via": {"Q": 0.15, "K": 0.72, "V": 0.13}}
  ]
}
```


Note L12H3 is a leverage point: discovered via Path 2 (activation) with low C_W but high C_A—small weight changes, large activation impact. The pathway metadata shows L10_MLP primarily affects L12H3 through the key pathway ($w_K = 0.72$), suggesting L10_MLP writes information that L12H3 uses to determine *where* to attend.

Remark K.1 (Validation Status)**.** *This output format is illustrative. The schema shown here reflects the specification; the reference implementation's output format may differ in field naming or nesting as the implementation matures. Consult the implementation repository for the current schema.*